

Chapter 5.

From Algorithms to Architectures



You Will Learn ...

- ➡ Options for tailoring hardware to data/signal processing algorithms
- ➡ Techniques for optimizing VLSI architectures
 - ➡ Iterative decomposition, pipelining, replication, time sharing
 - ➡ Algebraic transforms
 - ➡ Retiming
 - ➡ Loop unfolding, pipeline interleaving
- ➡ Options for temporary storage of data
- ➡ Some other architectural concepts
 - ➡ Bit-serial architectures, distributed arithmetic
 - ➡ Computing in semirings



Goals of Architecture Design

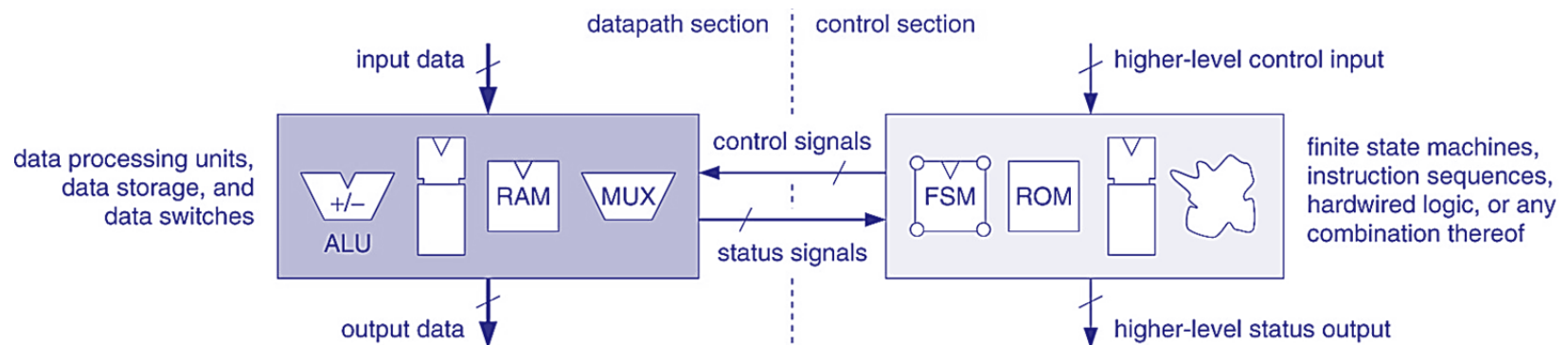
- ➡ Decide on the necessary hardware resources for carrying out computations from data and/or signal processing
- ➡ Organize their interplay such as to meet target specifications
- ➡ Major Concerns
 - ➡ Functional correctness
 - ➡ Performance targets (throughput, operation rate, etc.)
 - ➡ Circuit size
 - ➡ Energy efficiency
 - ➡ Agility (e.g. evolving needs, changing specs, and future standards)
 - ➡ Engineering effort and time to market



Architectural Solution Space

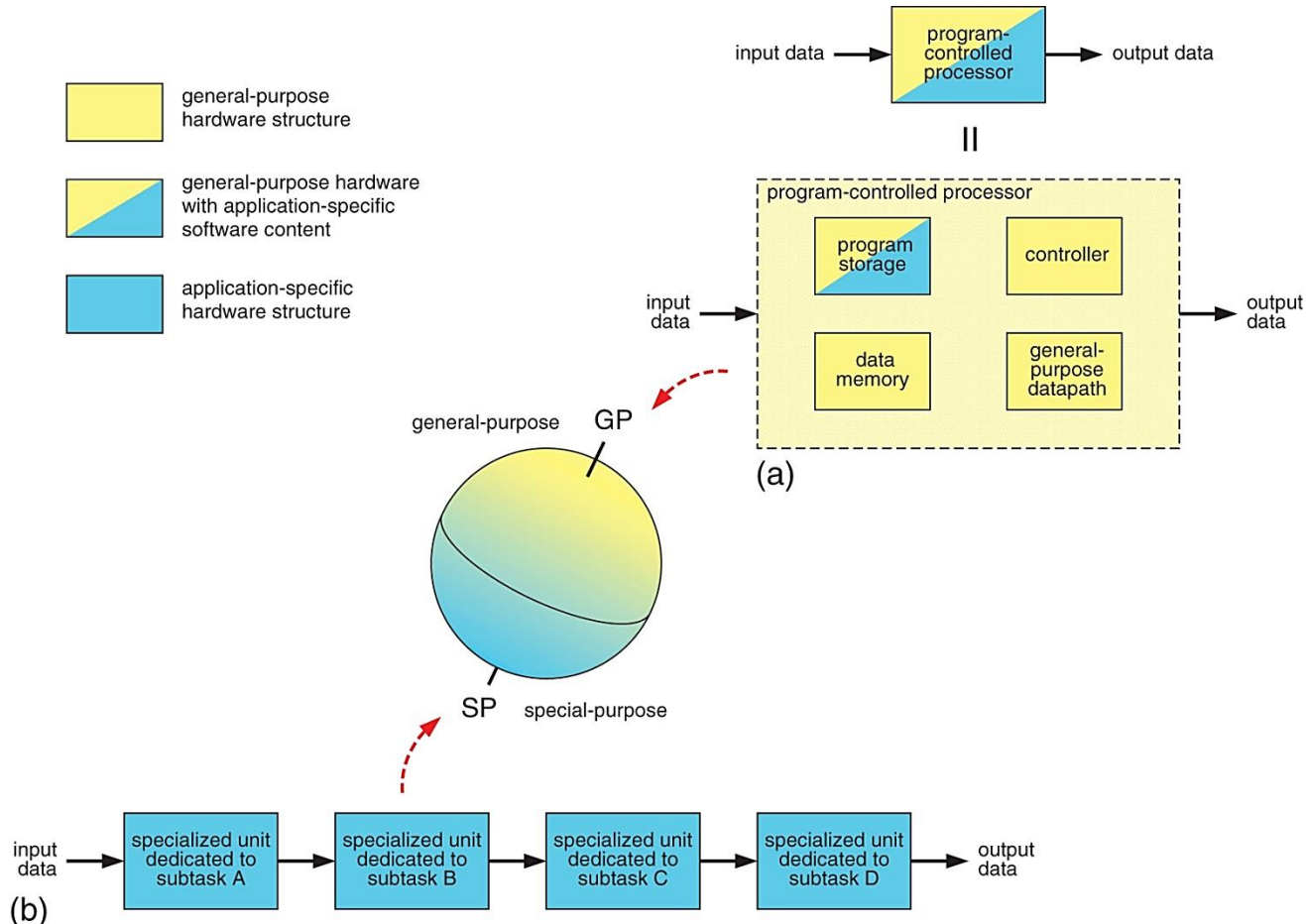
➔ Some thoughts about microprocessors

- ➔ **Instruction set processors** execute one program instruction after the other in consecutive fetch-load-execute-store cycles
- ➔ **ALU (arithmetic-logic unit)** carries out data manipulations
- ➔ **Datapath vs. Control path**
- ➔ **Von Neumann architecture** (common memory space) vs. **Harvard architecture** (separate memory spaces for data and program codes)



The Architectural Antipodes (1/3)

➡ General-purpose processor and dedicated hardware



The Architectural Antipodes (2/3)

➔ The architectural antipodes comparisons

	Hardware architecture	
	General purpose	Special purpose
Algorithm	any, not known a priori	fixed, must be known
Architecture	instruction set processor	dedicated, no single pattern
Execution model	fetch-load-execute-store “instruction-oriented”	process data item and pass on “dataflow-oriented”
Datapath	ALU(s) plus memory	customized design
Controller	with program microcode	typically hardwired
Performance indicator	instructions per second, run time of benchmarks	data throughput, can be anticipated analytically
Strengths	highly flexible, immediately available, routine design flow, low up-front costs	room for max. performance, highly energy-efficient, lean circuitry



The Architectural Antipodes (3/3)

➡ **Guideline:** Before embarking in ASIC design, find out:

- ➡ Does an architecture dedicated to the application at hand make sense or
- ➡ Is a program-controlled general-purpose processor more adequate?

➡ **Observations:**

- ➡ Opting for commercial microprocessors and/or FPL sidesteps many technical problems that absorb much attention when a custom IC is to be designed instead.
- ➡ Conversely, it is precisely the focus on the payload computation, the absence of programming and configuration overhead, and the full control over architecture, circuit, and layout details that make it possible to optimize performance and energy efficiency.



Viterbi Decoder – Example (1/2)

Architecture	General purpose		Special purpose	
Key component	DSP		ASIC	
	TI TMS320C6455 without / with Viterbi coprocessor VCP2		sem03w6 ETH	sem05w1 ETH
Number of chips	1	1	1	1
CMOS process	90 nm	90 nm	250 nm	250 nm
Program code	187 Kbyte	242 Kbyte	none	none
Circuit size	n.a.	n.a.	73 kGE	46 kGE
Max. throughput	45 kbit/s	9 Mbit/s	310 Mbit/s	54 Mbit/s
@ clock	1 GHz	1 GHz	310 MHz	54 MHz
Power dissipation	2.1 W	2.1 W	1.9 W	50 mW
Year	2005	2005	2004	2006

- DSP optimized for sustained multiply-accumulates, word width 32 bits.
- Viterbi algorithm arranged to do without multiplication, with words of 6 bit or less and dedicated architectures for parallelism.



AES Block Cipher – Examples (2/2)

Architecture	General purpose	Special purpose		
Key component	CISC Proc. Pentium III	FPGA Xilinx Virtex-II	ASIC (ETH) CryptoFun	ASIC (UCLA) Rijndael core
Number of chips	motherboard	1 + config.	1	1
CMOS process	n.a.	150 nm	180 nm	180 nm
Max. throughput	648 Mbit/s	1.32 Gbit/s	2.0 Gbit/s	1.6 Gbit/s
@ clock	1.13 GHz	n.a.	172 MHz	125 MHz
Power dissipation	41.4 W	490 mW	n.a.	56 mW
@ supply	n.a.	1.5 V	1.8 V	1.8 V
Year	2000	≈ 2002	2007	2002

- ➡ Multiple LUTs included in hardware for S-Box function and inverse.
- ➡ Ciphering and subkey preparation carried out by concurrent units.
- ➡ Rijndael algorithm designed with Pentium III architecture (MMX instructions, LUTs that fit into cache memory, etc.)

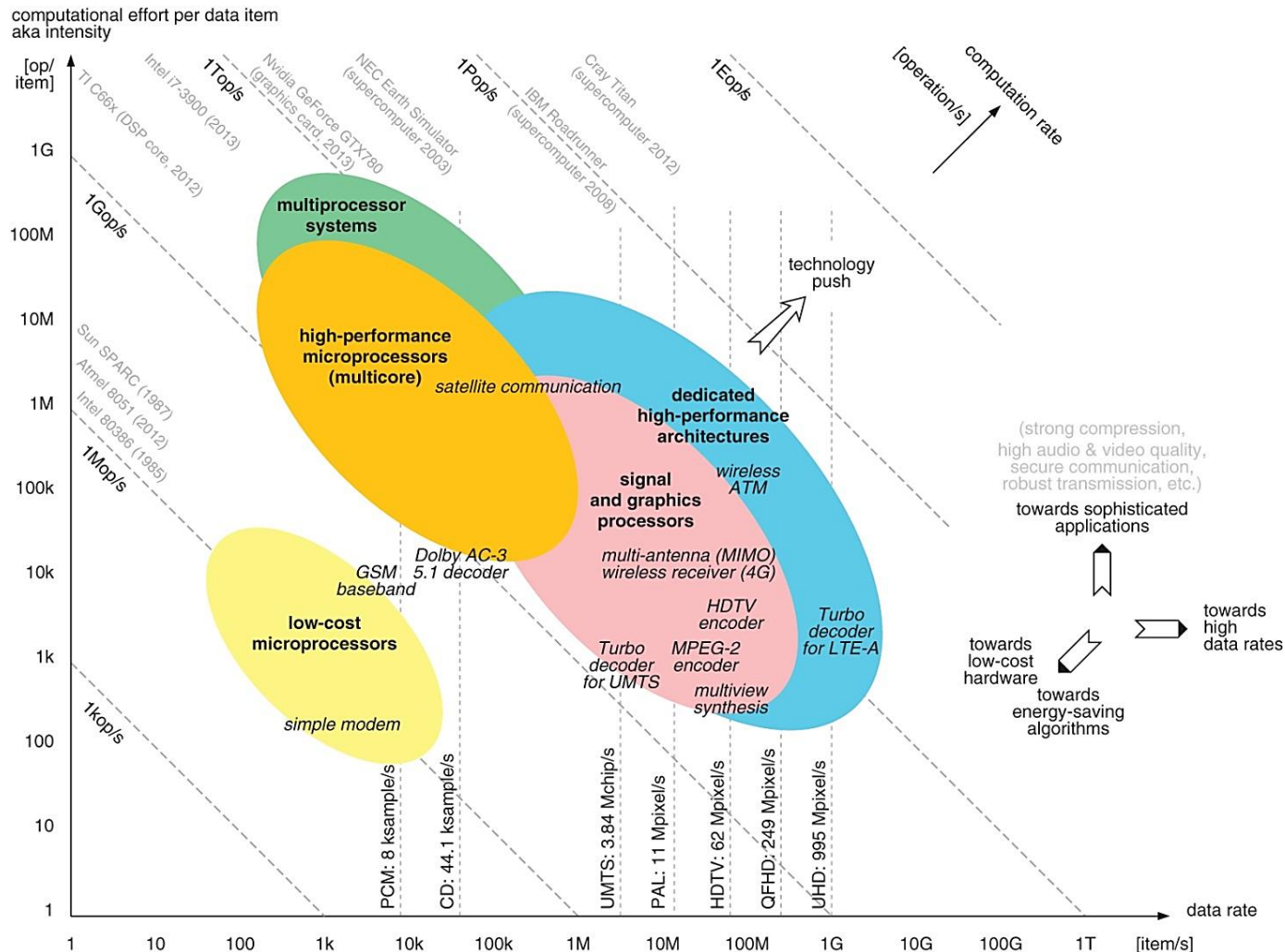


Why Dedicated Architectures?

- ➡ Dedicated architectures are favored by real-time applications such as
 - ➡ Data, audio and video (de)compression, error correction coding, digital modulation & demodulation, adaptive channel equalization, computer graphics and video rendering, multimedia ...
- ➡ Decisions: Yes or no?
 - ➡ Dedicated architectures outperform program-controlled processors by orders of **magnitude (wrt throughput and energy efficiency)** in many **transformational** systems where data streams get processed in fairly **regular** ways.
 - ➡ Dedicated architectures can not rival the agility and economy of processor-type designs in applications where the computation is primarily **reactive**, very **irregular**, highly data-dependent, or memory-hungry.



Computational Needs & Capabilities



Algorithm Selection (1/3)

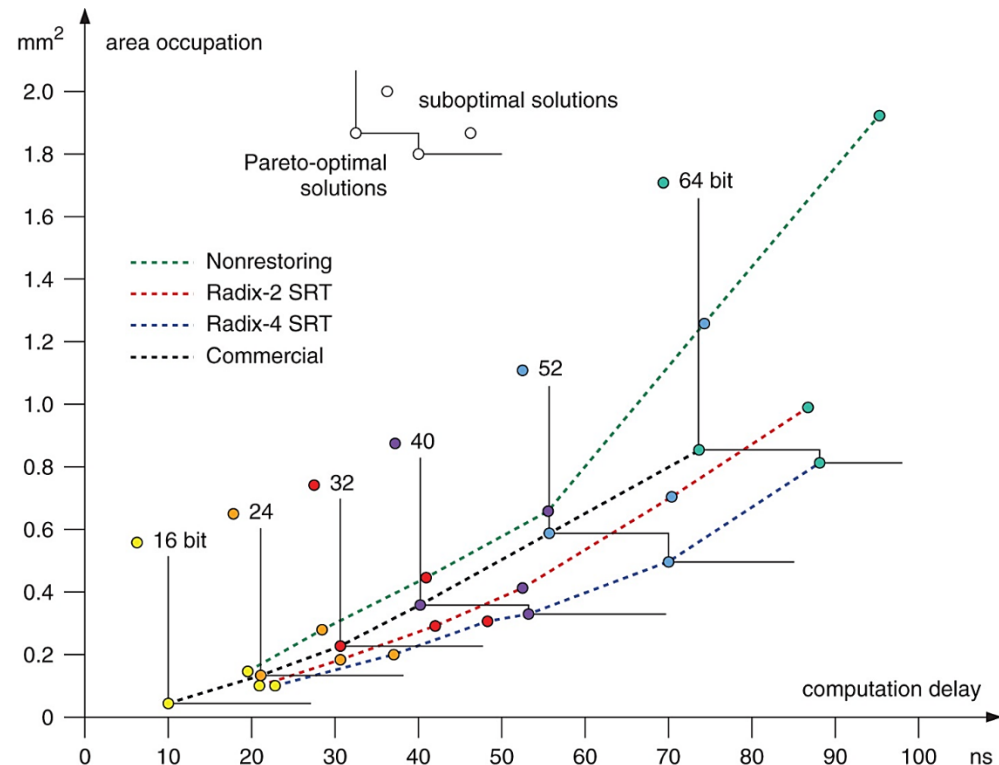
- ➡ What makes an algorithm suitable for dedicated VLSI architectures?
 - ➡ Costs are not the same in hardware as in software
- ➡ Desired characteristics
 - ➡ **Loose coupling between major processing tasks**: well-defined functional specification for each task, and manageable interactions between them.
 - ➡ **Simple control flow**: course of operation does not depend on the data being processed, and no need for overly many modes of operations, data formats, etc
 - ➡ **Regular data flow, recurrence of a few identical operations**: for sharing hardware resources in an efficient way
 - ➡ **Reasonable storage requirements**: renders on-chip memories economically possible



Algorithm Selection (2/3)

➡ Desired characteristics (cont'd)

- ➡ **Compatible with finite precision arithmetics:** insensitive to effects from finite precision, no need for floating-point arithmetics
- ➡ **Area, logic delay, interconnect length, parasitic capacitances, and energy dissipation grow with word width.**
- ➡ **Pre-layout dividers for 180 nm CMOS under worst-case PVT conditions.**



Algorithm Selection (3/3)

➡ Desired characteristics (cont'd)

- ➡ **Non-recursive linear time-invariant computation over some algebraic field**: opens a door for reorganizing the data processing in many ways
- ➡ **No transcendental functions**: roots, logarithmic, exponential, or trigonometric functions, translations between incompatible number systems are expensive in hardware
- ➡ **Extensive usage of operations unavailable from instruction sets**: replace lengthy instruction sequences by dedicated computational units; fixed arguments often allow for some form of preprocessing
- ➡ **No divisions and multiplications on very wide data words**: much more expensive than addition and subtraction; vast numerical range of results gives rise to scaling issues
- ➡ **Throughput rather than latency is what matters**: tight latency requirements rule out pipelining



Land Between the Antipodes (1/2)

➡ **Problem:** how to combine computational efficiency with flexibility?

➡ **Examples:** blu-ray player and smartphone

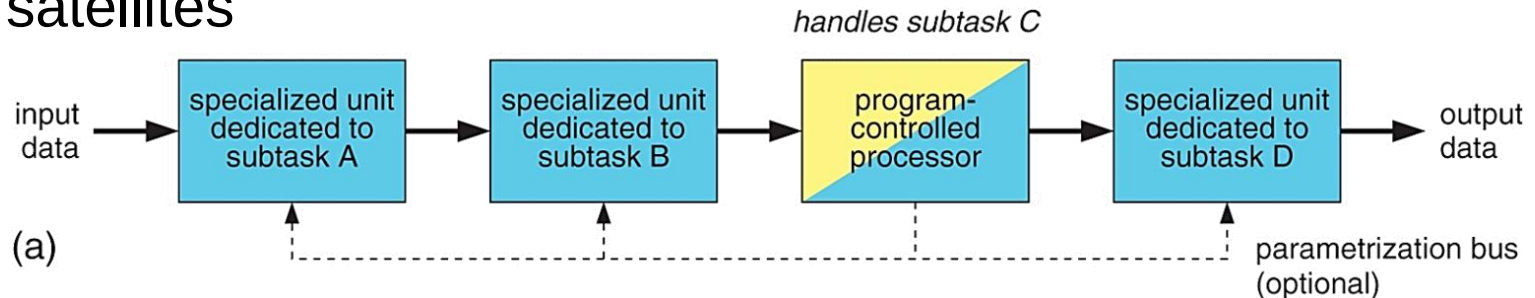
Applications	Subfunctions primarily characterized by	
	irregular control flow and/or need for flexibility	repetitive control flow and need for computing efficiency
Blu-ray player	user interface, track seeking, tray and spindle control, processing of non-video data (directory, title, author, subtitles, region codes)	16-to-8 bit demodulation, error correction, MPEG-2 decompression, deciphering (AACS AES-128), video signal processing
Smartphone	user interface, SMS/MMS, directory management, battery monitoring, communication protocol, channel allocation, roaming, accounting	intermediate frequency filtering, (de)modul., channel (de)coding, error correction (de)coding, (de)ciphering, speech and video (de)compression, display graphics

⇒ Segregate the needs for computational efficiency from those of agility

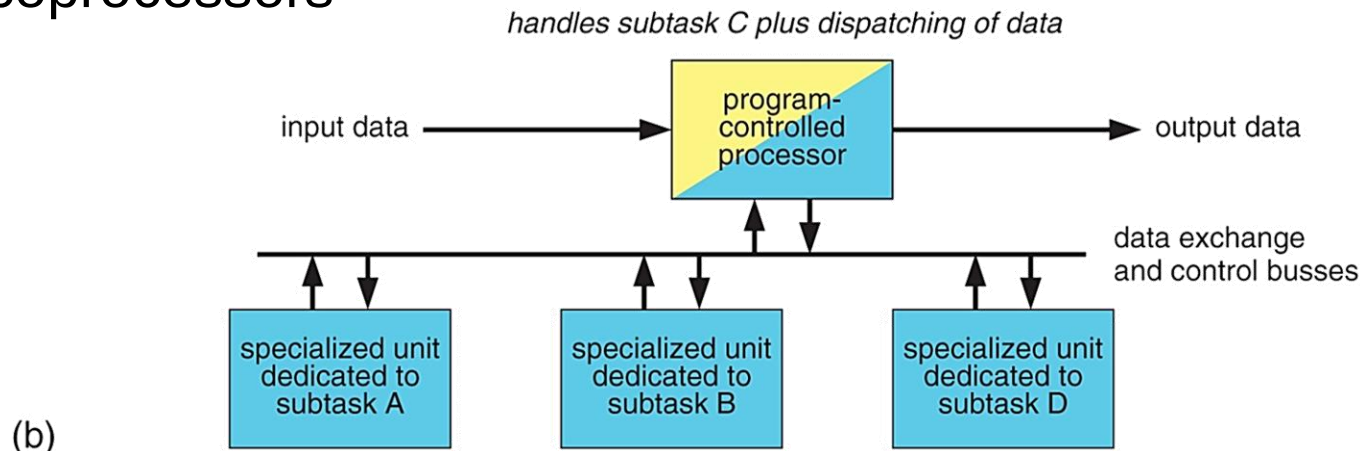


Land Between the Antipodes (2/2)

- ➔ Chain of general-purpose processor and dedicated satellites

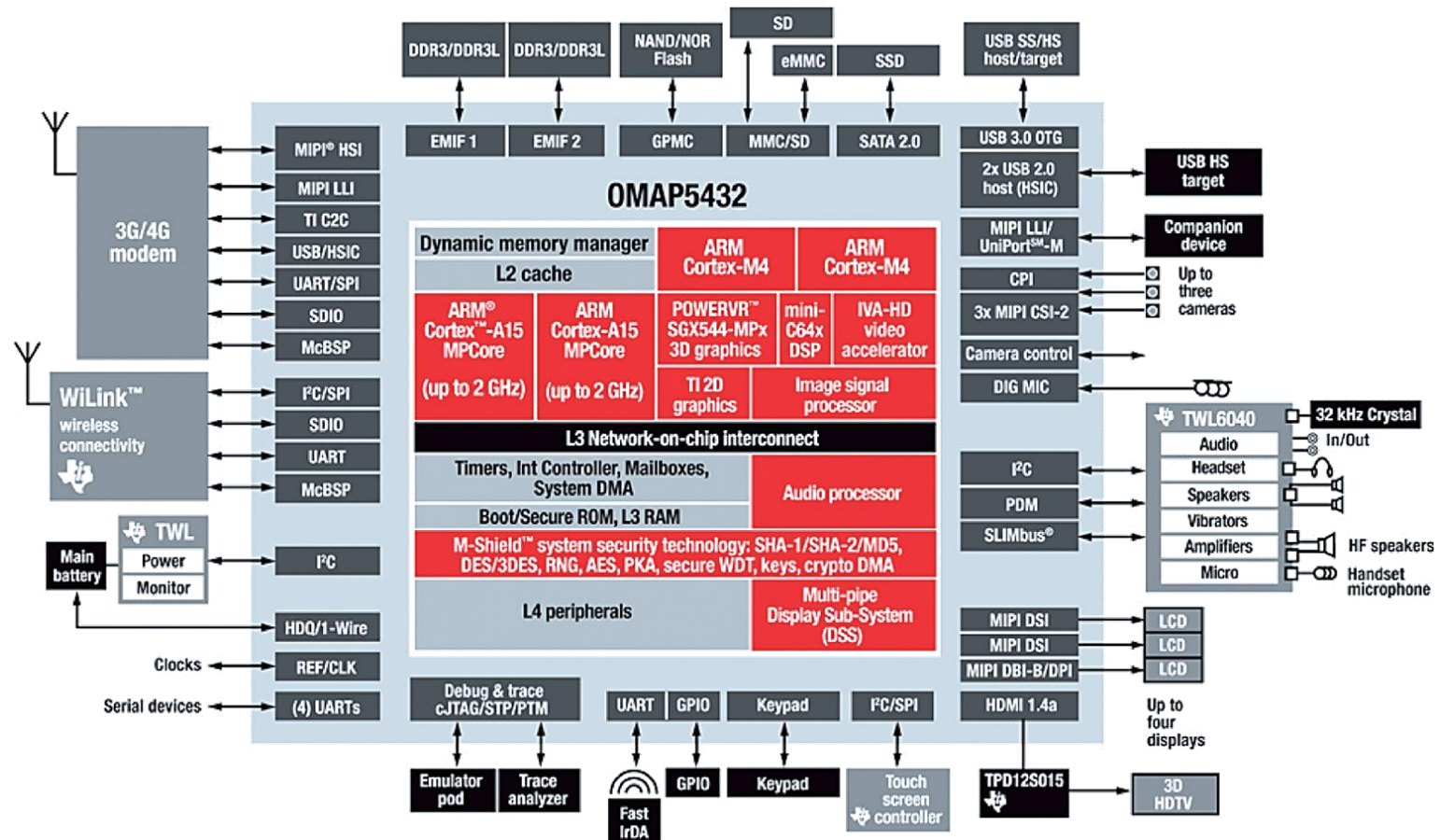


- ➔ A host computer with specialized fixed-function blocks or coprocessors



SoC for TI Smartphones

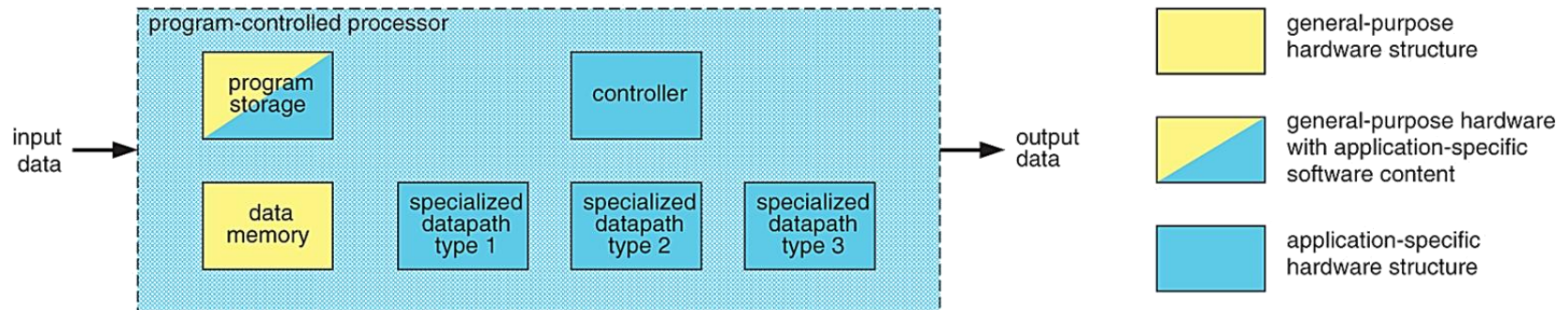
➔ Host computer with helper engineer



Application-Specific Instruction Set Proc.

- ➡ Application-specific instruction set processors (ASIP)
 - ➡ Program-controlled operation is highly flexible.
 - ➡ Application-specific features are confined to datapath circuitry.
 - ➡ Single thread of execution (concurrency limited to SIMD), easily extended to multiple threads (by including multiple ASIP cores)
 - ➡ The design flow essentially includes: (1) define the most adequate instruction set for a target application, (2) refine the architecture into a cycle-accurate model, and (3) cast your architecture into an RTL-type model (plus supporting tool chain)

handles subtasks A, B, C and D with the aid of multiple specialized datapaths



ASIP – An Example

➔ AES cipher encrypter/decrypter - revisited

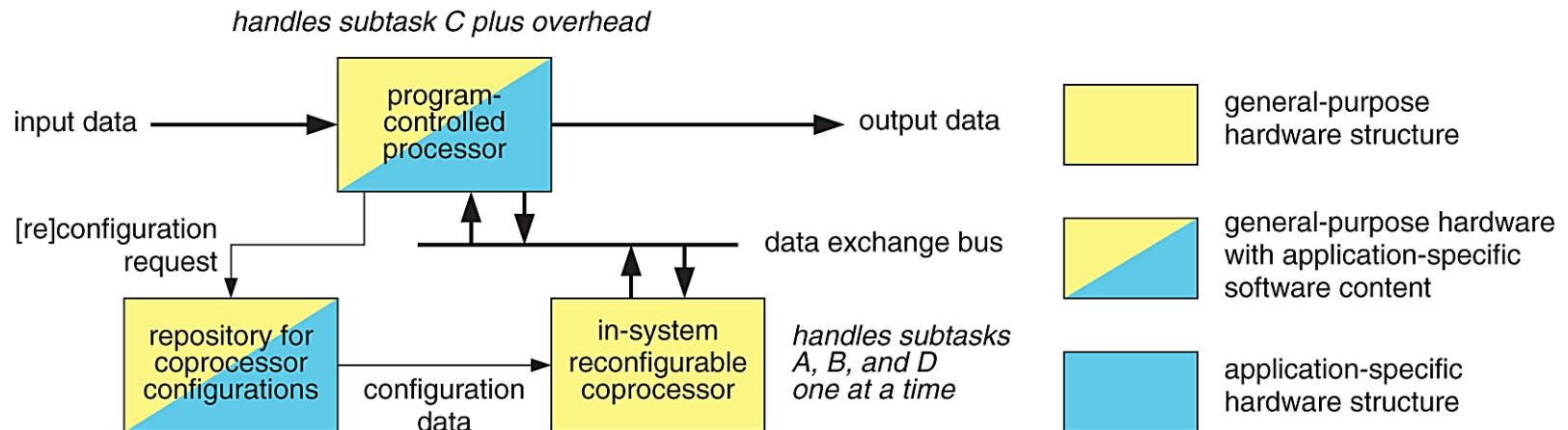
- ➔ ASIP combines excellent throughput and low power with the agility of a program-controlled architecture - **proprietary instruction set with special assembler, libraries, debuggers, ...**

General sol.	Special purposes			ASIP
CISC Proc. Pentium III	FPGA Xilinx Virtex-II	ASIC (ETH) CryptoFun	ASIC (UCLA) Rijndael core	Cryptoprocess or core UCLA
motherboard	1 + config.	1	1	1
assembler	none	none	none	Assembler
n.a.	n.a.	76 kGE	173 kGE	73.2 kGE
n.a.	150 nm	180 nm	180 nm	180 nm
648 Mb/s	1.32 Gb/s	2.0 Gb/s	1.6 Gb/s	3.43 Gb/s
1.13 GHz	n.a.	172 MHz	125 MHz	295 MHz
41.4 W	490 mW	n.a.	56 mW	86 mW
n.a.	1.5 V	1.8 V	1.8 V	1.8 V
2000	≈ 2002	2007	2002	2004



Reconfigurable Computing

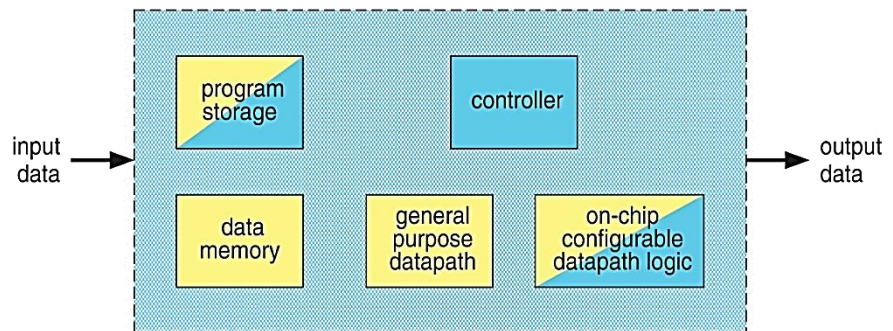
- ➡ General-purpose processor with juxtaposed reconfigurable coprocessor
 - ➡ Designers come up with a specific circuit structure for each major piece of suitable computation.
 - ➡ All configurations get stored in memory.
 - ➡ Whenever the host encounters a call to one of the computations, it downloads the pertaining configuration file into the FPL
 - ➡ Host feeds the coprocessor with data and fetches results



Extendable Instruction Set Processors

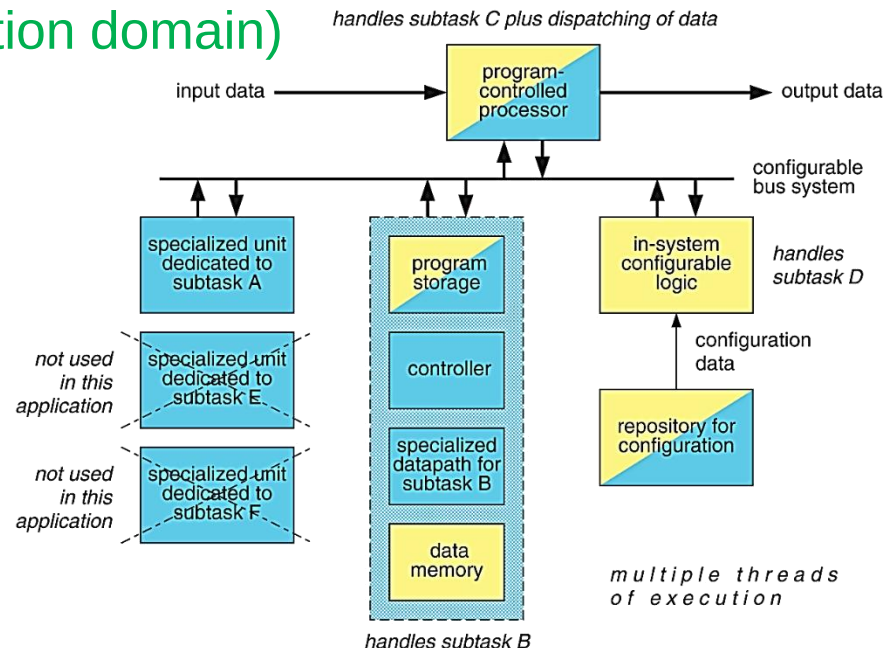
- ➔ Combine ideas from ASIP design and reconfigurable computing.
- ➔ General procedures:
 - ➔ System developers write application programs in C or C++.
 - ➔ Proprietary EDA tools identify **instruction sequences** that are executed many times over (**hot spots**).
 - ➔ For each such sequence, reconfigurable logic is synthesized into a parallel computation network that completes within one clock cycle.
 - ➔ Each occurrence of the original instruction sequence gets **replaced by a function call** that activates the custom-made logic.

handles subtasks A, B, C, and D with a combination of fixed and configurable datapaths



Platform ICs (DSPP)

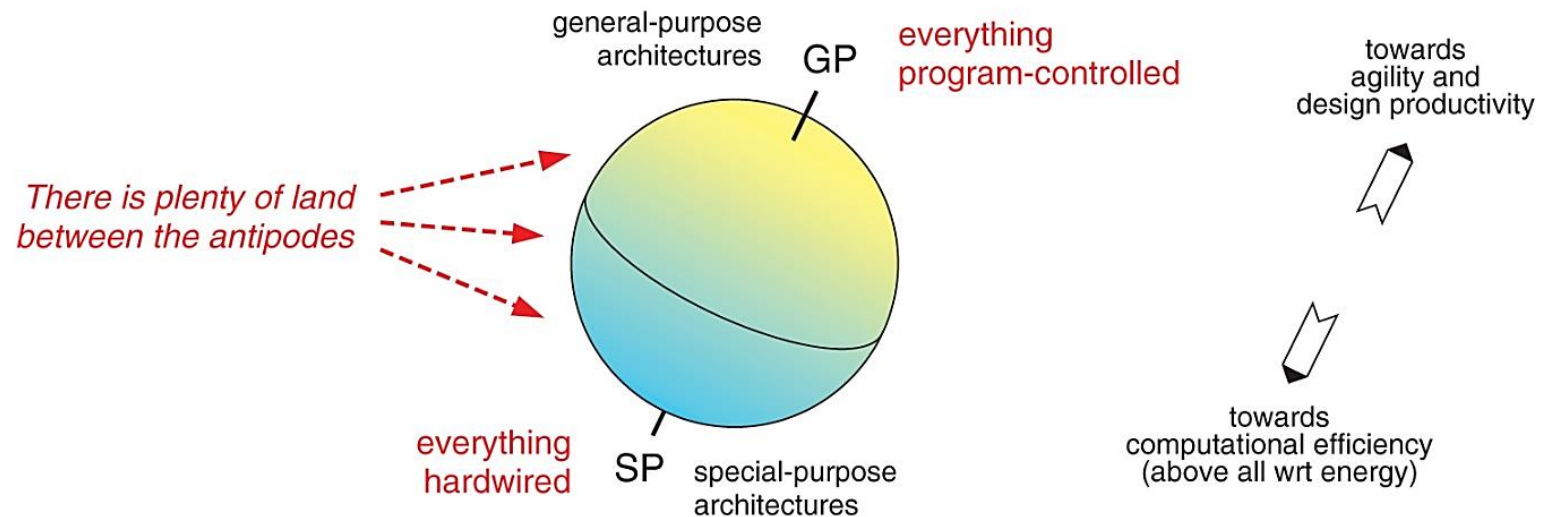
- ➔ Domain-specific programmable platform (DSPP)
 - ➔ Generous and heterogeneous circuit resources in one malleable platform.
 - ➔ Instruction-set-programmable processors (CPUs, GPUs, ASIPs)
 - ➔ Hardwired circuits (fixed-function blocks that are pretty standard in one application domain)
 - ➔ Electrically configurable fabrics (FPL)
 - ➔ Various types of on-chip memories (SRAM, DRAM, Flash)
 - ➔ Example: Xilinx Zynq all programmable SoC architecture.



Concluding Remarks (1/3)

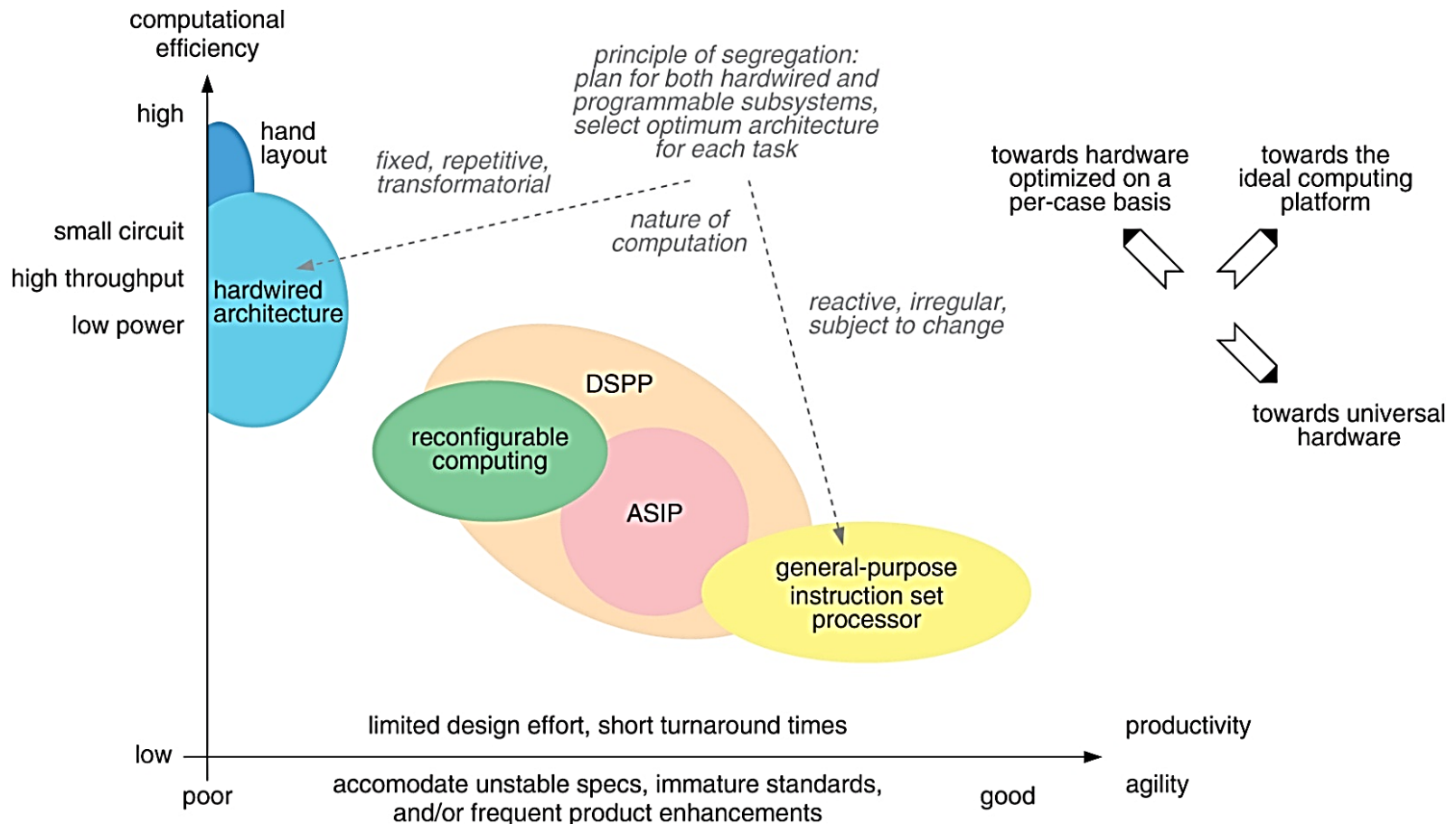
➡ Lesson Learned

- ➡ Rely on dedicated hardware only for those subfunctions that are called many times and are unlikely to change.
- ➡ Keep the rest programmable (via software or reconfiguration).



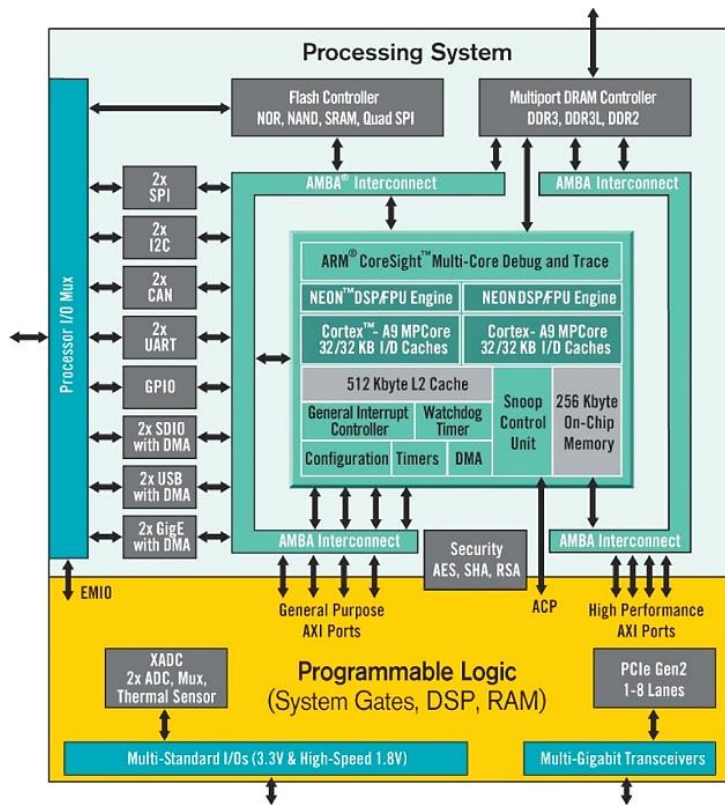
Concluding Remarks (2/3)

➔ Options of architecture design

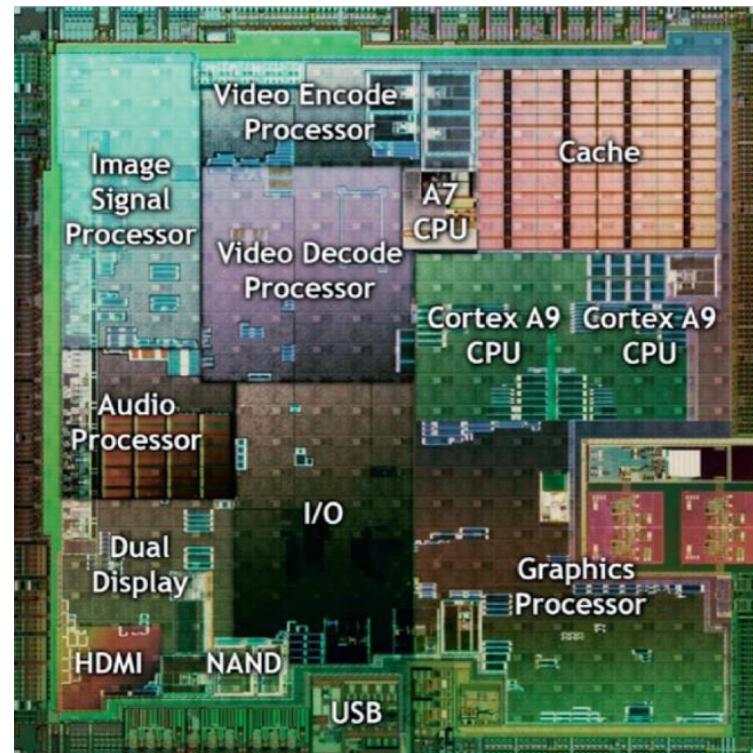


Concluding Remarks (3/3)

➔ Two Examples:



Zynq (Xilinx)



Tegra 2 SoC (Nvidia, 2010)



VLSI Architecture Design (1/6)

- ➡ Why do we focus on dedicated architectures?
 - ➡ A **priori knowledge of a computational problem** offers room for ideas that do not apply to instruction set processors.
 - ➡ Utmost **performance requirements** often ask for special-purpose designs.
 - ➡ The same holds for **energy efficiency**.
 - ➡ Industry provides us with a vast selection of micro- and signal processors, making proprietary designs hard to justify.
- ➡ How to decide on the necessary hardware resources for solving a given computational problem and how to best organize them?
 - ➡ **Most data and signal processing algorithms would lead to inefficient solution if they were implemented in hardware as they are.**



VLSI Architecture Design (2/6)

- ➡ Most processing algorithms must be reworked for hardware design. For example,
- ➡ **Digital filter:** Tolerate a somewhat lower stopband suppression in exchange for a reduced computational burden (e.g., lower order, smaller coefficients replaced by zeros).
 - ➡ **Viterbi decoder (for convolutional codes):** Sacrifice 0.1 dB or so of coding gain for the benefit of doing computations in a more economic way (e.g., truncated dynamic range and restricted traceback).
 - ➡ **Autocorrelation function:** Replace the computation of

$$ACF_{xx}(k) = r_{xx}(k) = \sum_{n=-\infty}^{\infty} x(n) \cdot x(n+k)$$

with the average magnitude difference function

$$AMDF_{xx}(k) = r'_{xx}(k) = \sum_{n=0}^{N-1} |x(n) - x(n+k)|$$



VLSI Architecture Design (3/6)

➔ Most processing algorithms must be reworked for hardware design. (cont'd)

➔ Approximations for computing magnitudes:

- ➔ Replace by l^1 and l^∞ -norms.
- ➔ Approximated with shift, add, and subtract
- ➔ Be care of **implementation loss** (e.g., re-implement floating point arithmetics with fixed-point arithmetics)

Name	aka	Formula
lesser	$l^{-\infty}$ -norm	$l = \min(a , b)$
sum	l^1 -norm	$s = a + b $
magnitude (reference)	l^2 -norm	$m = \sqrt{a^2 + b^2}$
greater	l^∞ -norm	$g = \max(a , b)$
Approximation 1		$m \approx \frac{3}{8}s + \frac{5}{8}g$
Approximation 2		$m \approx \max(g, \frac{7}{8}g + \frac{1}{2}l)$



VLSI Architecture Design (4/6)

➡ Finding an optimal hardware organization

- ➡ There is room for re-modeling computations in two distinct domains: **processing algorithm** and **hardware architecture**.
- ➡ **Alternative choices in the algorithmic domain:** How to tailor an algorithm such as to cut the computational burden, to trim down memory requirements, and/or to speed up calculations **without incurring unacceptable implementation losses**.

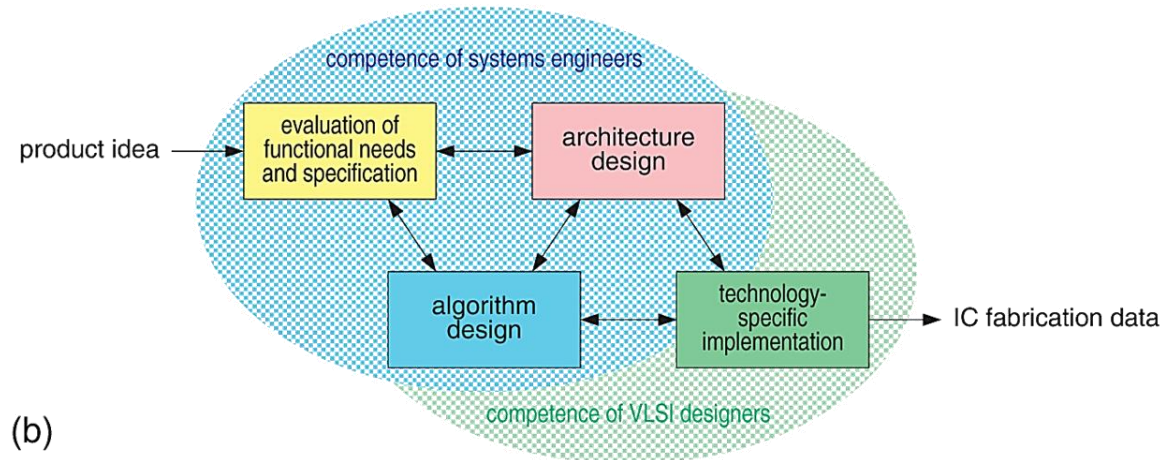
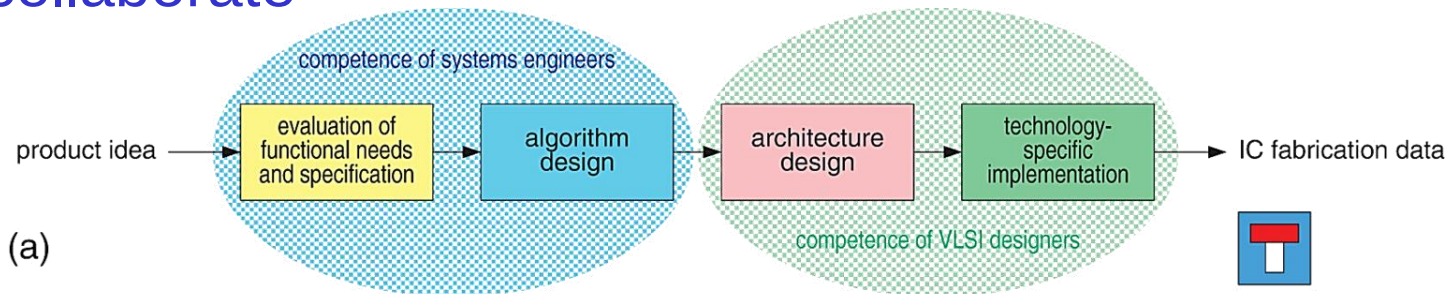
- ➡ **Equivalence transforms in the architectural domain:** How to (re)organize a computation such as to optimize throughput, circuit size, energy efficiency and overall costs while **leaving the input-to-output relationship unchanged except, possibly, for latency**.

⇒ There is more to VLSI design than accepting a given algorithm and turning that into hardware with the aid of some HDL synthesizer.



VLSI Architecture Design (5/6)

⇒ Systems engineers and VLSI designers must collaborate



VLSI Architecture Design (6/6)

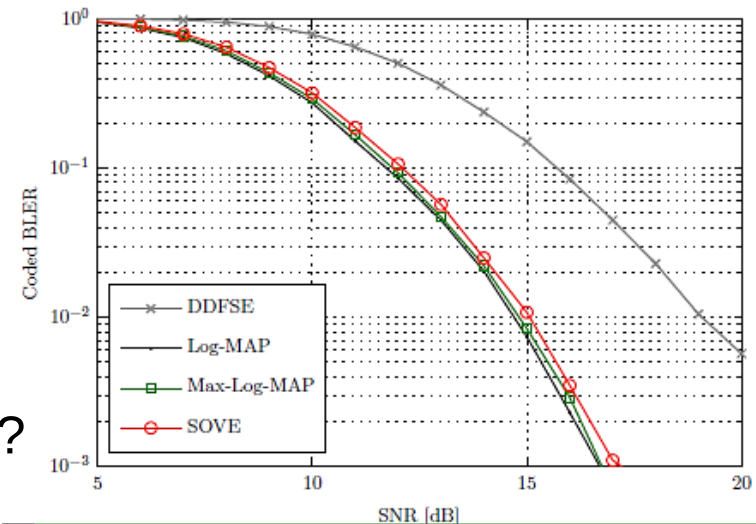
➔ Example: Sequence estimation for EDGE receiver

Algorithm	Delayed Decision Feedback	Max-log-MAP	Soft Output Viterbi Equalizer
Soft output	no	yes	yes
Forward recursion	yes	yes	yes
Backward recursion	no	yes	no
Backtracking step	yes	no	no
Memory requirements	1x	50x	0.13x

➔ Key design targets:

- ➔ soft output
- ➔ less than 577 us per burst
- ➔ small circuit and low power
- ➔ min. block error rate at any given signal-to-noise ratio

➔ Which one is your preference?



Data Flow Representation

➔ Data Dependency Graphs (DDG)

Definitions

vertex  memoryless operation

edge  transport weight indicates latency in computation cycles

Shorthand notations

introduced for convenience

$\downarrow 0 =$ 

 fan out expressed as "no operation" vertex

 illegal!

$\downarrow 1 =$ 

$\downarrow 2 =$ 

$c \rightarrow$ 

constant input expressed as constant data source

$x(k) \rightarrow$ 

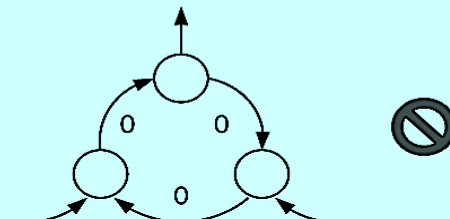
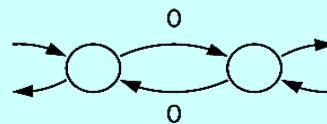
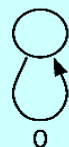
variable input expressed as time-varying data source

 $y(k)$

output expressed as data sink

Danger of race conditions

circular paths of edge weight zero are not admitted!

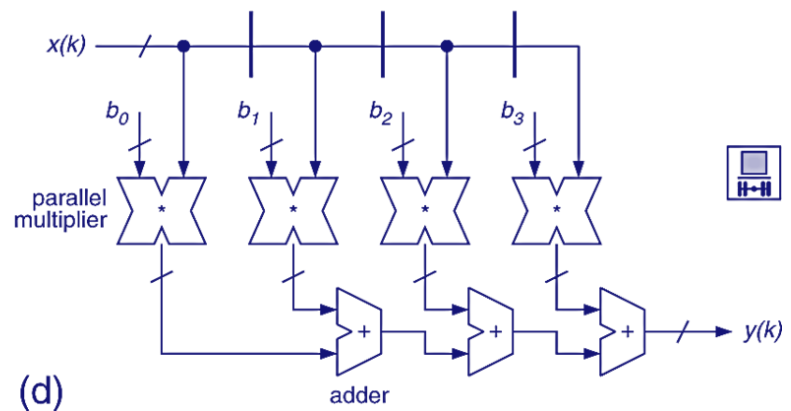
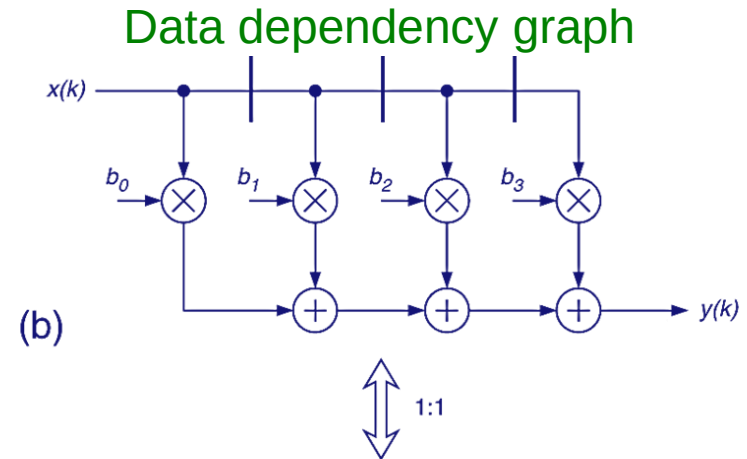
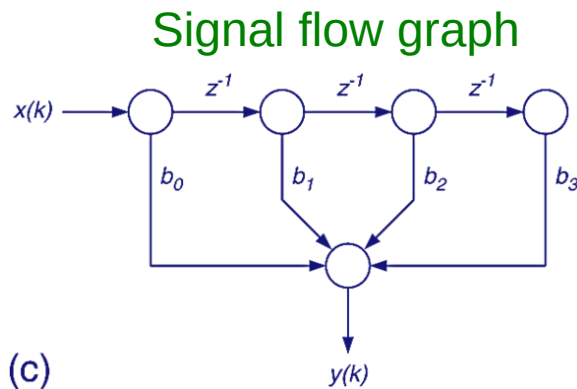


The Isomorphic Architectures

➡ Example: A third-order transversal filter in various notations

(a)
$$y(k) = \sum_{n=0}^{N=3} b_n x(k-n)$$

Mathematical function



Isomorphic hardware architecture



Figures of Merit (FoM) (1/3)

⇒ Figures of merit for hardware architectures

- ⇒ **Cycles per data item Γ** : the number of computation cycles between releasing two subsequent data items.
- ⇒ **Longest path delay t_{lp}** : the lapse of time required for data to propagate along the longest path. A circuit cannot function correctly unless $t_{lp} \leq T_{cp}$.
- ⇒ **Time per data item T** : the lapse of time between releasing two subsequent data items, e.g. in $\mu\text{s/sample}$, ms/frame , or s/computation . $T = \Gamma \cdot T_{cp} \geq \Gamma \cdot t_{lp}$.
- ⇒ **Data throughput $\Theta = \frac{1}{T} = \frac{f_{cp}}{\Gamma}$** : expressed in pixel/s, sample/s, frame/s, record/s, FFT/s, or the like.
- ⇒ **Latency L** : the number of computation cycles from a data item entering a circuit until the pertaining result becomes available.



Figures of Merit (FoM) (2/3)

⇒ Figures of merit for hardware architectures

- ⇒ **Circuit size A** : expressed in mm^2 , F^2 or GE (gate equivalent).
- ⇒ **Size-time product AT** : the hardware resources spent to obtain a given throughput $AT = \frac{A}{\Theta}$.
- ⇒ **Energy per data item E** : the amount of energy dissipated for a given computation on a data item, e.g. in pJ/MAC , nJ/sample , $\mu\text{J}/\text{datablock}$, or mWs/frame .

Can also be understood as power-per-throughput ratio $E = \frac{P}{\Theta}$ measured in $\frac{\text{mW}}{\text{Mbit/s}}$ or $\frac{\text{W}}{\text{Gop/s}}$.

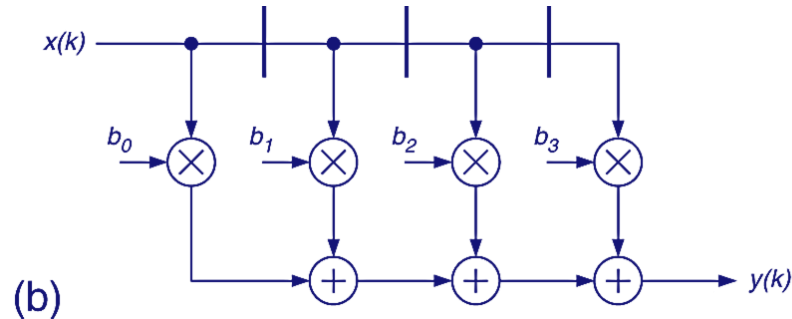
- ⇒ **Energy-time product ET** : indicates how much energy gets spent for achieving a given throughput (synonym “energy-per-throughput ratio”).

$$ET = \frac{E}{\Theta} = \frac{P}{\Theta^2}, \text{ e.g. in } \frac{\mu\text{J}}{\text{datablock/s}} \text{ or } \frac{\text{mWs}}{\text{videoframe/s}}$$



Figures of Merit (FoM) (3/3)

⇒ An Example:



⇒ Approximations:

- ⇒ Interconnect delays neglected (overly optimistic)
- ⇒ Delays of arithmetic operations summed up (sometimes pessimistic)
- ⇒ Glitching ignored (optimistic)

$$A = 3A_{reg} + 4A_{*} + 3A_{+}, \quad \Gamma = 1, \quad t_{lp} = t_{reg} + t_{*} + 3t_{+}$$

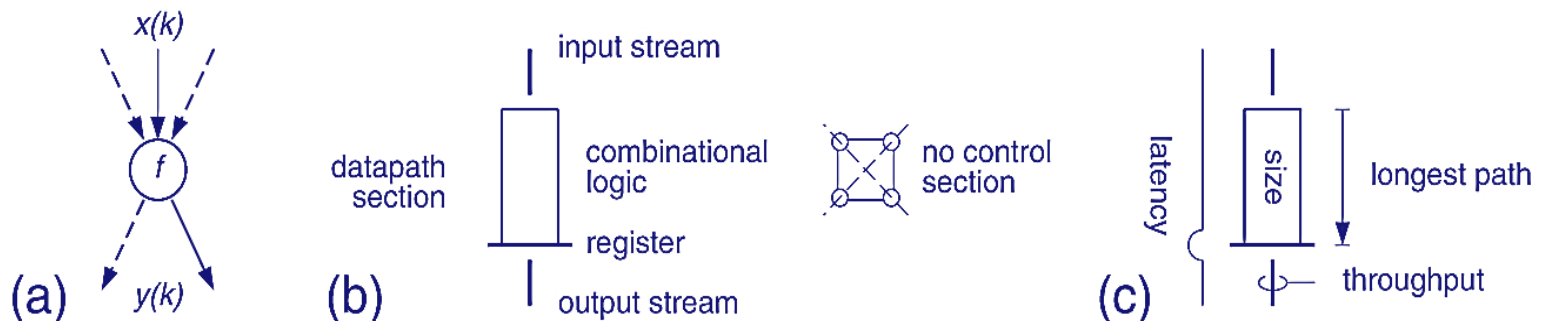
$$AT = (3A_{reg} + 4A_{*} + 3A_{+})(t_{reg} + t_{*} + 3t_{+}), \quad L = 0$$

$$E = 3E_{reg} + 4E_{*} + 3E_{+}$$



Symbolic HW Representation

- ➡ Reference hardware = isomorphic architecture + output register(s)
 - ➡ (a) DDG for combinational function, (b) a symbolic representation of the reference hardware configuration, and (c) the highlighted key characteristics



Figures of Merit - Summary

➡ Computation cycle versus clock period

- ➡ A computation period T_{cp} is the time span that separates two consecutive computation cycles.
- ➡ During each computation cycle, fresh data emanate from a register, propagate through combinational circuitry before the result gets stored in the next analogous register.
- ➡ It is the combinational circuitry that performs all arithmetic, logic, and data routing operations.
- ➡ Computation rate $f_{cp} = \frac{1}{T_{cp}}$ denotes the inverse.
- ➡ For all circuits that adhere to **single-edge-triggered one-phase clocking**, computation cycle and clock period are the same. (only one half for dual-edge triggering)

$$f_{cp} = f_{clk} \quad \Leftrightarrow \quad T_{cp} = T_{clk}$$



Equivalent Transforms – Overview (1/3)

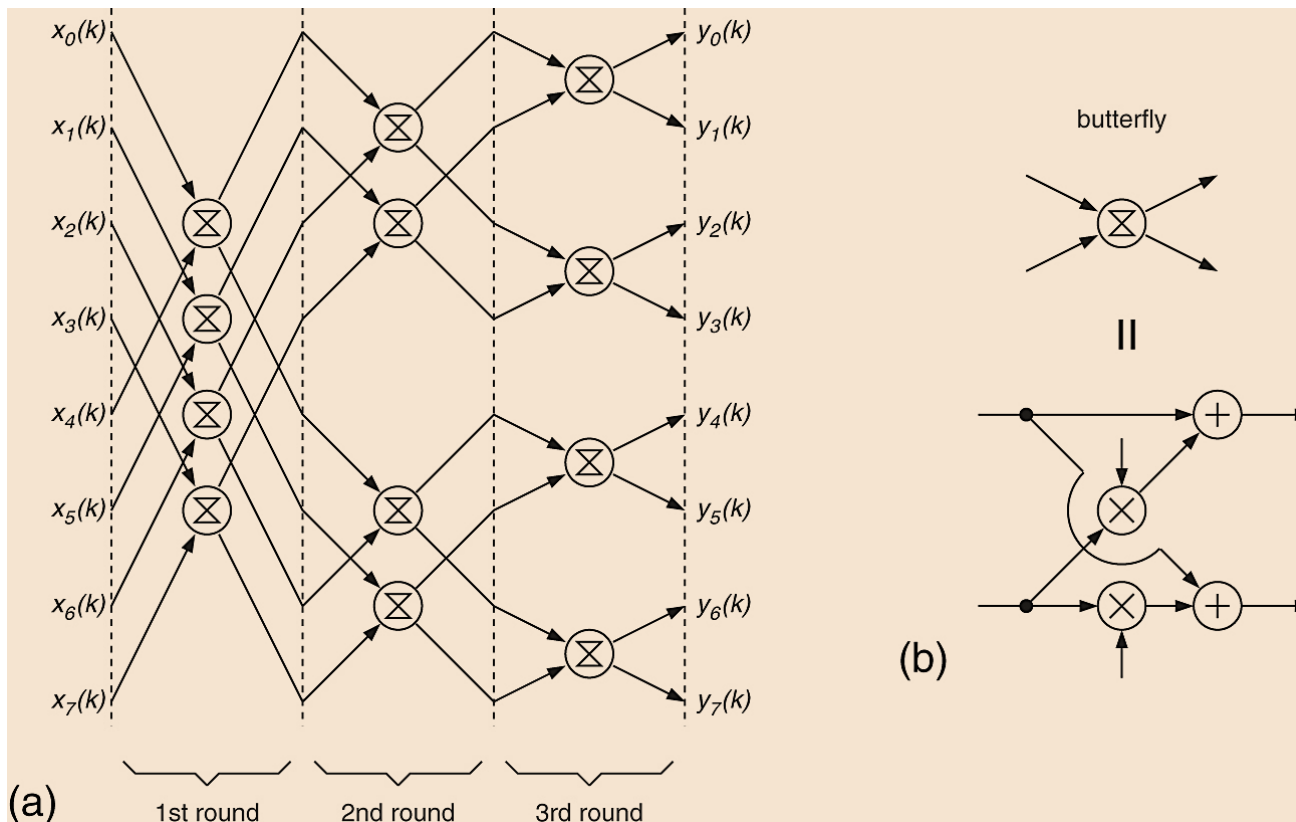
- ➡ We will learn some of the techniques for equivalent transforms of combinational computations.
- ➡ What is the combinational computation?
 - ➡ Result depends on the present arguments exclusively.
 - ➡ All edges in the DDG have weight zero.
 - ➡ DDG is free of circular paths.
- ➡ Generally, architecture designers primarily focus on those operations that heavily impact chip size, performance, power dissipation, etc.
 - ➡ What is the combinational computation of interest?
 - ➡ a two-bit adder or for instance a fast Fourier transform (FFT) operation



Equivalent Transforms – Overview (2/3)

➡ An 8-point FFT Example

➡ What do you suggest for choosing the “basic” operations?



Equivalent Transforms – Overview (3/3)

- ⇒ Three common options for reorganizing and improving the given design at the architectural level.
 - ⇒ **Decomposing** function f into a sequence of subfunctions that get executed one after the other on the same hardware.
 - ⇒ **Pipelining** of the functional unit for f to improve computation rate by cutting down the combinational depth.
 - ⇒ **Replicating** the hardware for f and having all units work concurrently.
- ⇒ **Open questions:**
 - ⇒ Does it make sense to combine pipelining with iterative decomposition?
 - ⇒ How do replication and pipelining compare?
 - ⇒ Are there situations where one should be preferred over the other?



Reference Architecture/Design

➔ Reference hardware = isomorphic architecture + output register(s)

➔ Comparison metrics:

$$A(0) = A_f + A_{reg}, \Gamma(0) = 1, t_{lp}(0) = t_f + t_{reg}$$

$$AT(0) = (A_f + A_{reg})(t_f + t_{reg}), L(0) = 1, E(0) = E_f + E_{reg}$$

➔ Some common assumptions:

- ➔ The word width w is assumed to be fixed.
- ➔ For a fairly substantial operation f , $A_{reg} \times t_{reg} \ll A_f \times t_f$
- ➔ Extra circuitry A_{ctl} for controlling datapath operation and for routing data items is included if necessary.
- ➔ Dynamic contribution for energy is assumed.

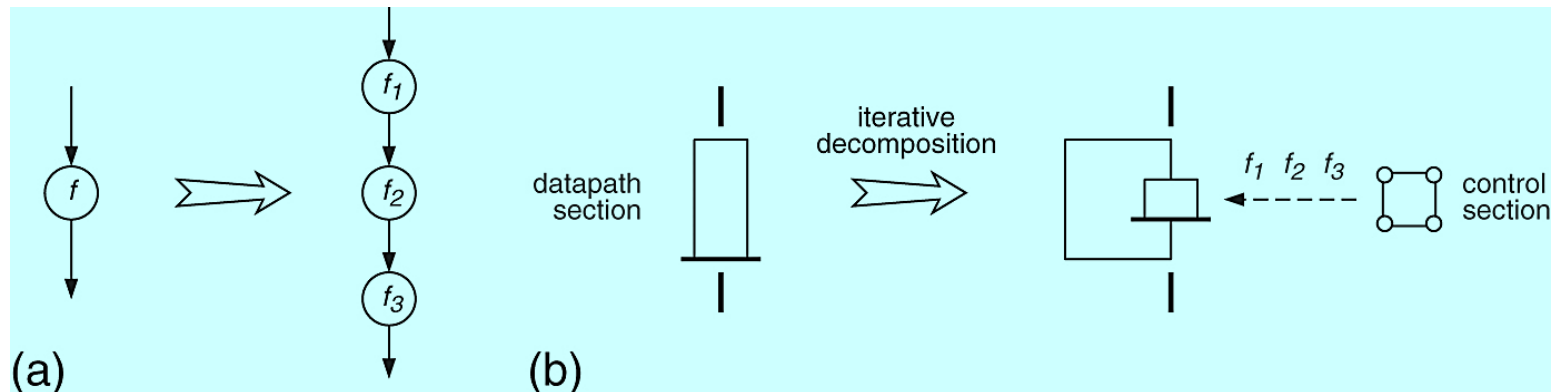


Iterative Decomposition (1/4)

➡ Basic mindset: resource sharing through step-by-step execution.

- ➡ The computation of function f is broken up into a sequence of d subtasks which are carried out one after the other.
- ➡ From a dataflow point of view, intermediate results are recycled until the final result becomes available at the output d computation cycles later.

➡ An example for $d = 3$.



Iterative Decomposition (2/4)

⇒ Performance and cost analysis

- ➡ Assumptions: (i) The total size required to implement the subfunction is between A_f/d and A_f . (ii) The decomposition is lossless and balanced, i.e., breaking up f into d subfunctions, each consuming t_f/d time unit.

⇒ A first-order approximation:

$$A_f/d + A_{reg} + A_{ctl} \leq A(d) \leq A_f + A_{reg} + A_{ctl}$$

$$\Gamma(d) = d, \quad t_{lp}(d) \approx t_f/d + t_{reg}$$

$$d(A_{reg} + A_{ctl})t_{reg} + (A_{reg} + A_{ctl})t_f + A_f t_{reg} + A_f t_f/d \leq AT(d) \leq$$

$$d(A_f + A_{reg} + A_{ctl})t_{reg} + (A_f + A_{reg} + A_{ctl})t_f$$

$$L(d) = d, \quad E(d) (\leq \text{ or } \geq) E_f + E_{reg}$$



Iterative Decomposition (3/4)

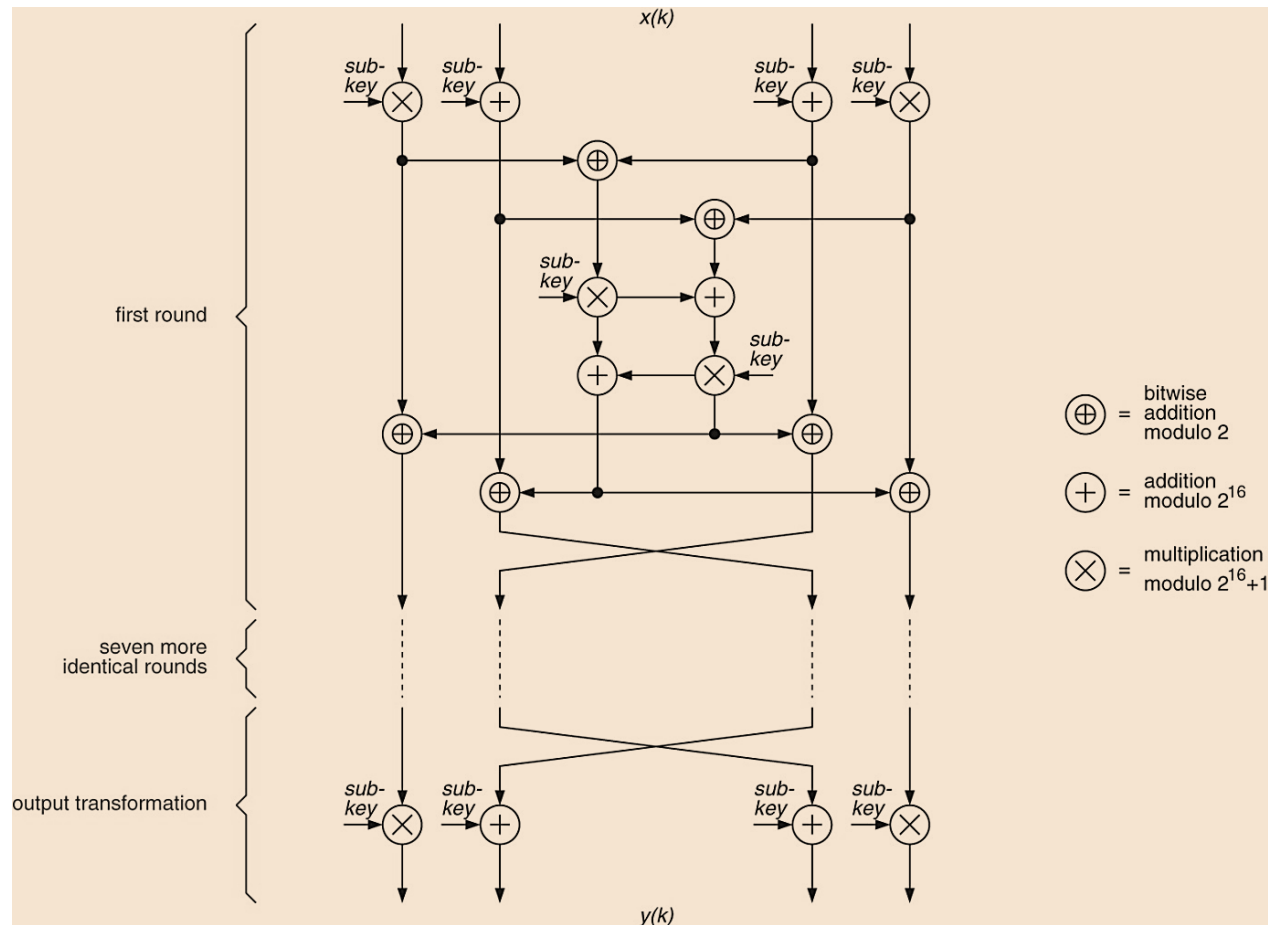
⇒ Lesson learned

- ➡ It is attractive when a computation makes repetitive use of a single subfunction, such as multiplication, because a lot of area can then be saved.
- ➡ It is unattractive when subfunctions are very disparate such as the square root operation and, therefore, cannot be made to share much hardware resources.
- ➡ Does not impact throughput much as long as $t_{reg} \ll t_{lp}$.
- ➡ May or may not improve energy efficiency
 - ⇒ Yes, if cutting overly long signal propagation paths mitigates excessive glitching and the associated energy losses.
 - ⇒ No, if the extra activity of data registers, control logic, and data recycling circuitry dominates.



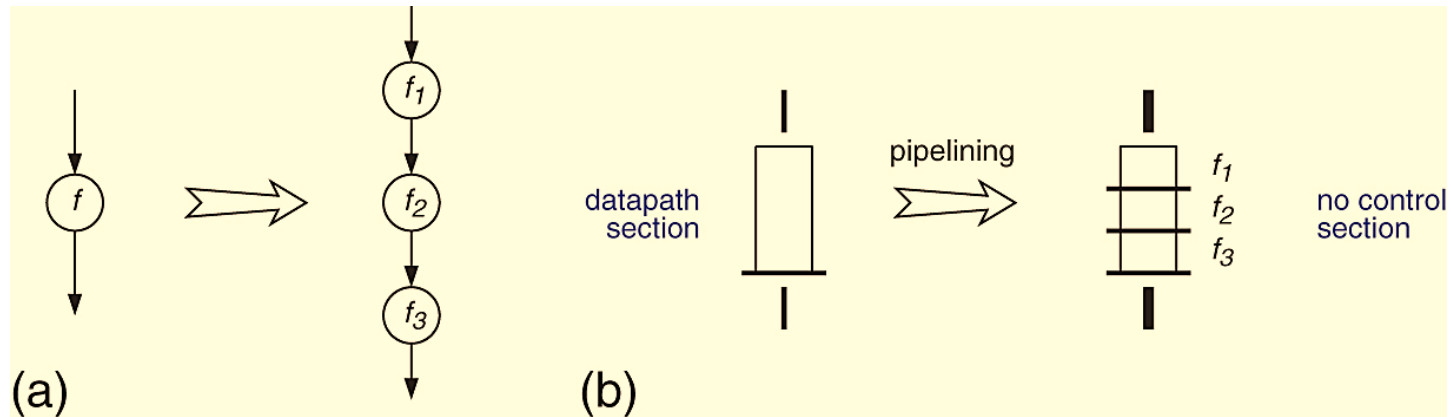
Iterative Decomposition (4/4)

➔ An example: the block cipher IDEA



Pipelining (1/6)

- ➔ Basic mindset: an assembly line operated by specialist workers
 - ➔ Increasing throughput by cutting combinational depth into several separate stages of approximately uniform computational delays by inserting registers in between.
- ➔ An example for $p = 3$ pipeline stages (with $p - 1$ extra registers)



Pipelining (2/6)

⇒ Performance and cost analysis

- ⇒ Assumptions: (i) The combinational logic for implementing f is not affected by the number of pipeline stages introduced (i.e., A_f remains constant). (ii) The pipeline is lossless and balanced, i.e., partitioning the logic into p stages, each having identical delay t_f/p . (iii) Penalties come from A_{reg} and t_{reg} .

⇒ A first-order approximation:

$$\begin{aligned}
 A(p) &= A_f + pA_{reg} & AT(p) &\approx pA_{reg}t_{reg} + (A_{reg}t_f + A_ft_{reg}) + \frac{1}{p}A_ft_f \\
 \Gamma(p) &= 1 & L(p) &= p \\
 t_{lp}(p) &\approx \frac{t_f}{p} + t_{reg} & E(p) &\underset{\text{coarse grain}}{\overset{\text{fine grain}}{\geq}} E_f + E_{reg}
 \end{aligned}$$

- ⇒ Both performance and size grows monotonically with pipeline depth. AT complexity is lower for coarse grain pipelining (modest number of pipeline stage).



Pipelining (3/6)

- ➡ **Coarse grain pipelining:** few registers evenly inserted into a deep combinational network
 - ➡ Little extra area for much better throughput.
 - ➡ AT -product lowered dramatically.
 - ➡ Long re-convergent fanout paths cut $\Rightarrow$ reduced glitching
- ➡ **Fine grain pipelining:** combinational delay in each stage approaches register delay
 - ➡ Diminishing speedup for more and more overhead.
 - ➡ AT -product augments significantly.
 - ➡ Significant register activity added $\Rightarrow$ waste of energy
- ➡ **Theoretical low bound:** each pipeline stage contains at least one 2-input NAND or NOR gate
 - ➡ $T_{cp} \geq \min(t_{lp}) = \min(t_{gate}) + t_{reg} = \min(t_{nand}, t_{nor}) + t_{su\ ff} + t_{pd\ ff}$
 - ➡ $T_{cp} \geq t_{NOR2D1} + t_{DFFPB1} = 18ps + 249ps \approx 267ps$ for 130nm CMOS



Pipelining (4/6)

⇒ A glance at microprocessors

CPU	year	clock [MHz]	FO4 inverter delays per pipeline stage
Intel 80386	1989	33	≈ 80
Intel Pentium 4	2003	3200	12...16
Core 2 Duo	2007	2167	≈ 40
Core i7 980X	2011	3333...3600	42...46
IBM POWER5	2004	1650...1900	22
IBM POWER6	2007	3500...5000	13
IBM Cell Processor	2006	3200	11

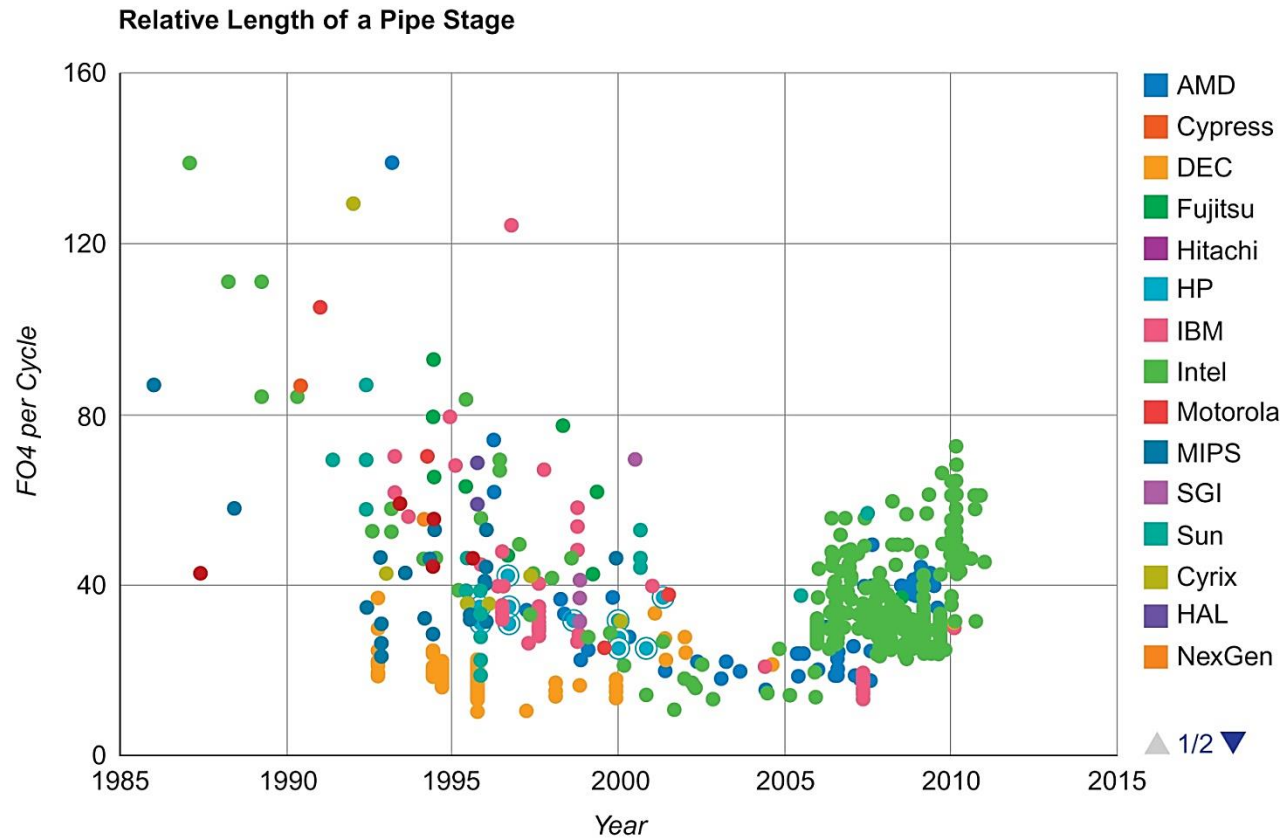
⇒ Observations

- ⇒ Pipelining has been instrumental in pushing processor clock frequencies.
- ⇒ 12 or so FO4 (fanout of 4) inverter delays per stage is close to practical limit.
- ⇒ Trend towards ever deeper pipelines reversed in the Intel Core family to reclaim energy efficiency.



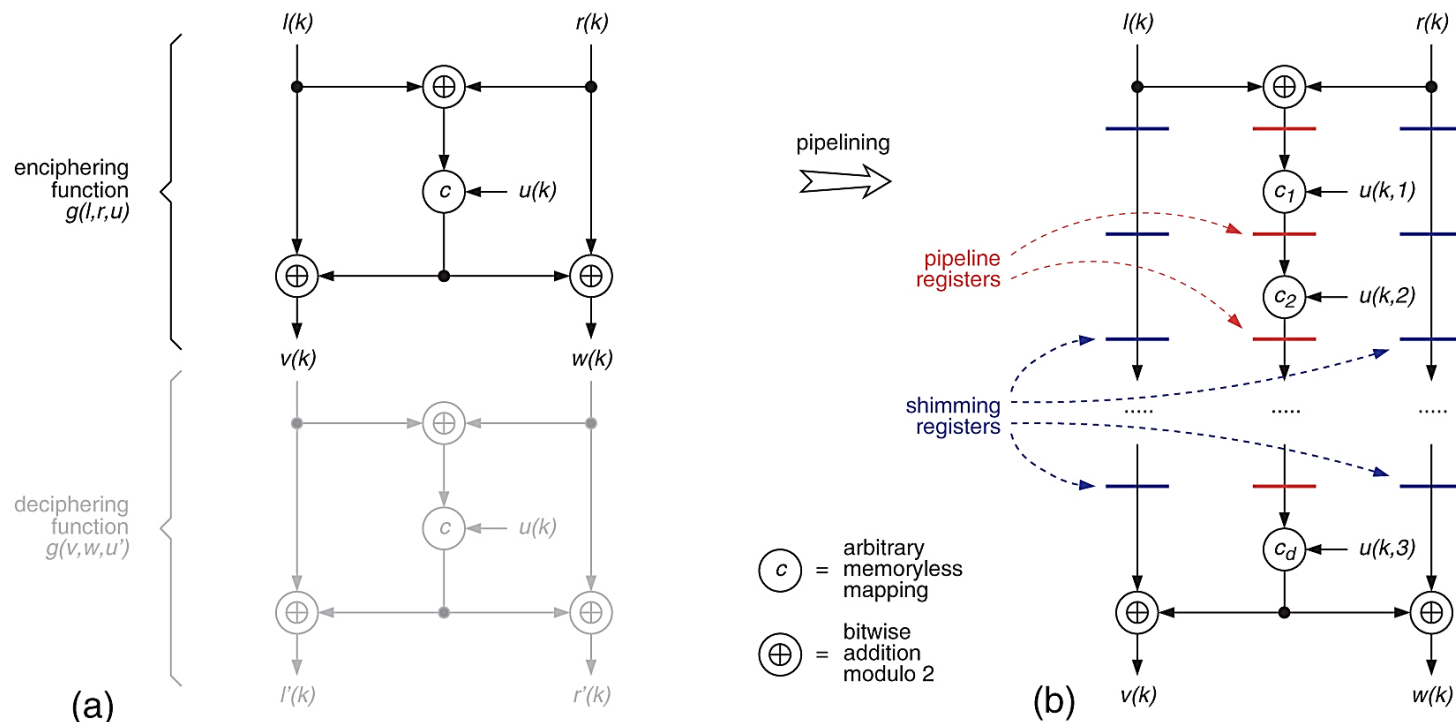
Pipelining (5/6)

➔ Evolution of pipeline depth over the years



Pipelining (6/6)

➔ An Example: an involutory cipher algorithm with multiple feedforward paths



➔ How many stages yield optimum pipeline efficiency?

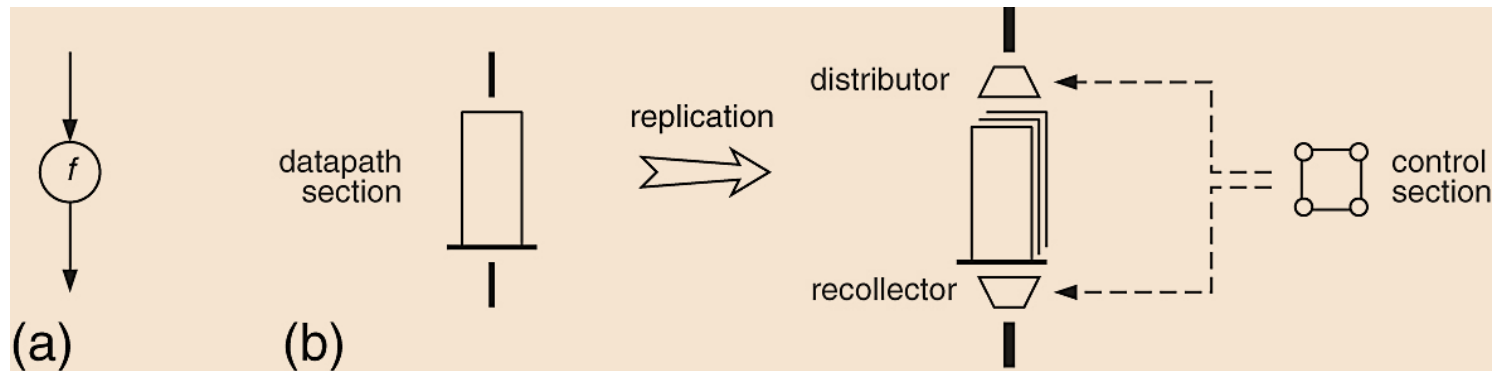


Replication (1/4)

➡ Basic mindset: parallel processing on the data

- ➡ Providing q instances of identical functional units for f , each processing one out of q data items in a cyclic manner.
- ➡ Two synchronous q -way switches distribute and recollect data at the chunk's input and output.

➡ An example for $q = 3$.



Replication (2/4)

⇒ Performance and cost analysis

- ➡ Assumptions: Any size and energy penalties associated with distributing data to replicated functional units and with recollecting them are neglected.

⇒ A first-order approximation:

$$A(q) = q(A_f + A_{reg}) + A_{ctl}$$

$$\Gamma(q) = \frac{1}{q}$$

$$t_{lp}(q) \approx t_f + t_{reg}$$

$$AT(q) \approx (A_f + A_{reg} + \frac{1}{q}A_{ctl})(t_f + t_{reg}) \approx (A_f + A_{reg})(t_f + t_{reg})$$

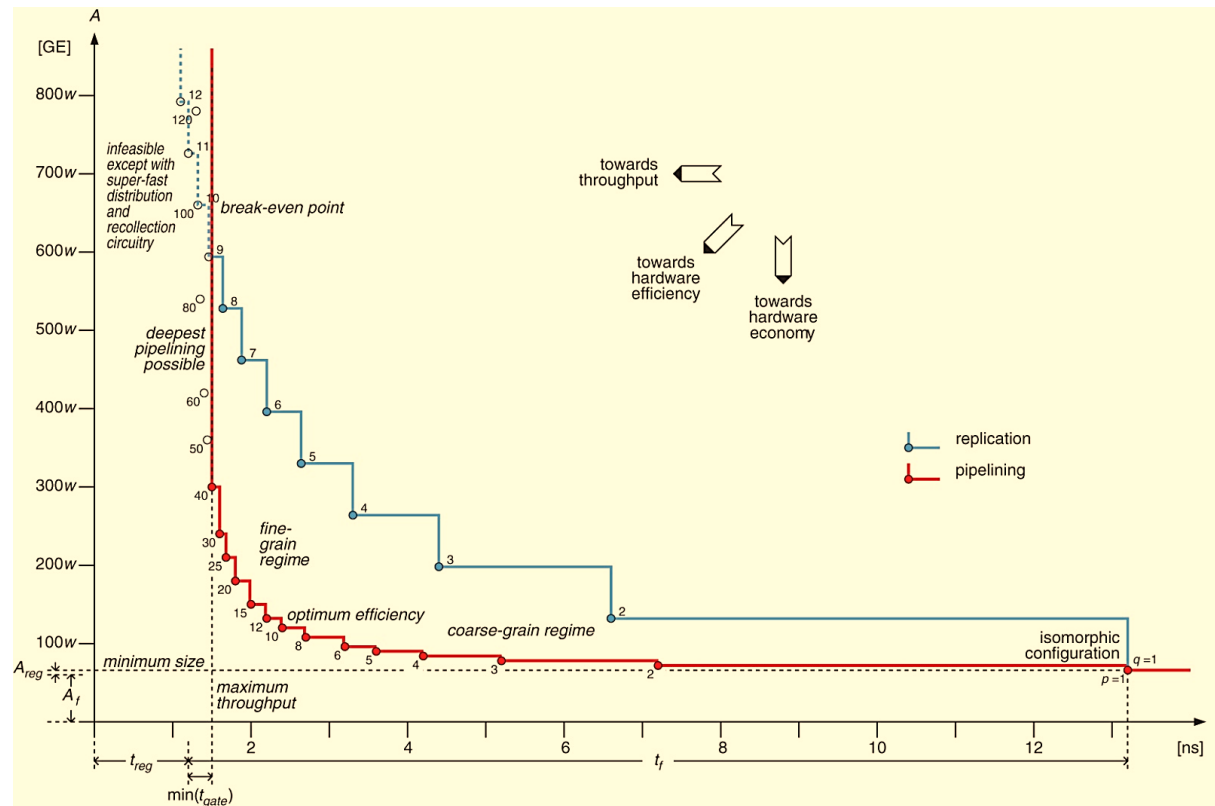
$$L(q) = 1$$

$$E(q) \approx E_f + E_{reg} + E_{ctl}$$



Replication (3/4)

➔ Replication versus pipelining



Note: Replication is limited to high-performance circuits and always combined with pipelining.



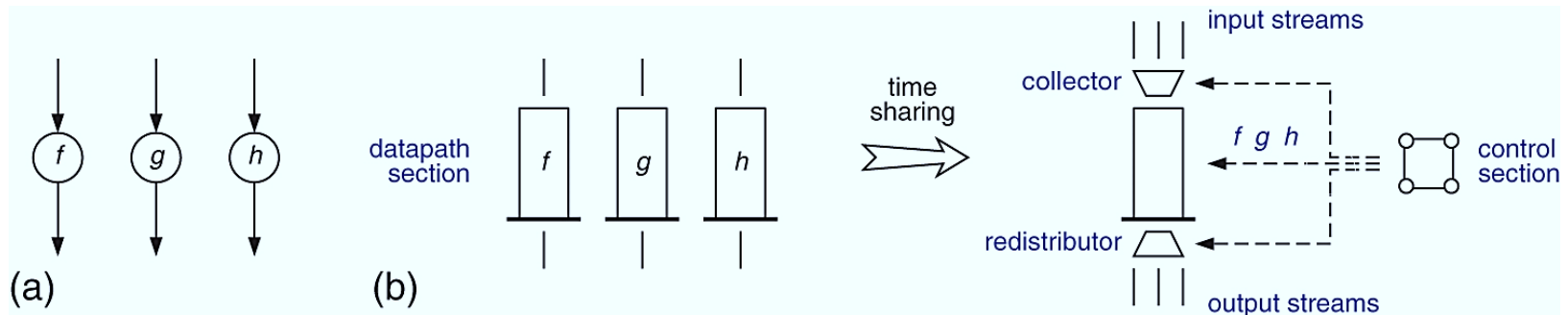
Replication (4/4)

- ⇒ Example: Microprocessor architecture - computer industry has been pushed towards replication because
- ⇒ CMOS offered more room for increasing circuit complexity than for pushing clock frequencies higher.
 - ⇒ The faster the clock, the smaller the region on a semiconductor die that can be reached within a single clock period.
 - ⇒ Fine grain pipelines dissipate a lot of energy for relatively little computation.
 - ⇒ Reusing a well-tried subsystem benefits design productivity and lowers risks.
 - ⇒ A multicore processor can still be of commercial value even if one of its CPUs is found to be defective.



Time Sharing (1/4)

- ➡ Basic mindset: reducing cost for simultaneous processing of multiple parallel data streams
 - ➡ It is also known as **time multiplexing** or **resource sharing**.
 - ➡ Two synchronous s-way switches collect and redistribute data at the chunk's input and output.
 - ➡ It requires that the circuitries for computing the various functions involved all be combined into a single datapath.
- ➡ An example for $s = 3$.



Time Sharing (2/4)

➡ Performance and cost analysis

- ➡ Assumptions: (i) The size of a circuit capable of implementing f , g , and h is between $\max(A_f, A_g, A_h)$ and $(A_f + A_g + A_h)$. (ii) The time for the combined computational unit to evaluate any of the function is $\max(t_f, t_g, t_h)$. (iii) Penalties for collecting and redistributing the data are neglected. (iv) The energy consumed is closer to $s \max(E_f, E_g, E_h)$ than $(E_f + E_g + E_h)$.

➡ A first-order approximation:

$$\max_{f,g,h}(A) + A_{reg} + A_{ctl} \leq A(s) \leq \sum_{f,g,h} A + A_{reg} + A_{ctl}$$

$$\Gamma(s) = s$$

$$t_{lp}(s) \approx \max_{f,g,h}(t) + t_{reg}$$

$$s(\max_{f,g,h}(A) + A_{reg} + A_{ctl})(\max_{f,g,h}(t) + t_{reg}) \leq AT(s) \leq$$

$$s(\sum_{f,g,h} A + A_{reg} + A_{ctl})(\max_{f,g,h}(t) + t_{reg})$$

$$L(s) = s$$

$$E(s) \approx s \max_{f,g,h}(E) + E_{reg} + E_{ctl}$$



Time Sharing (3/4)

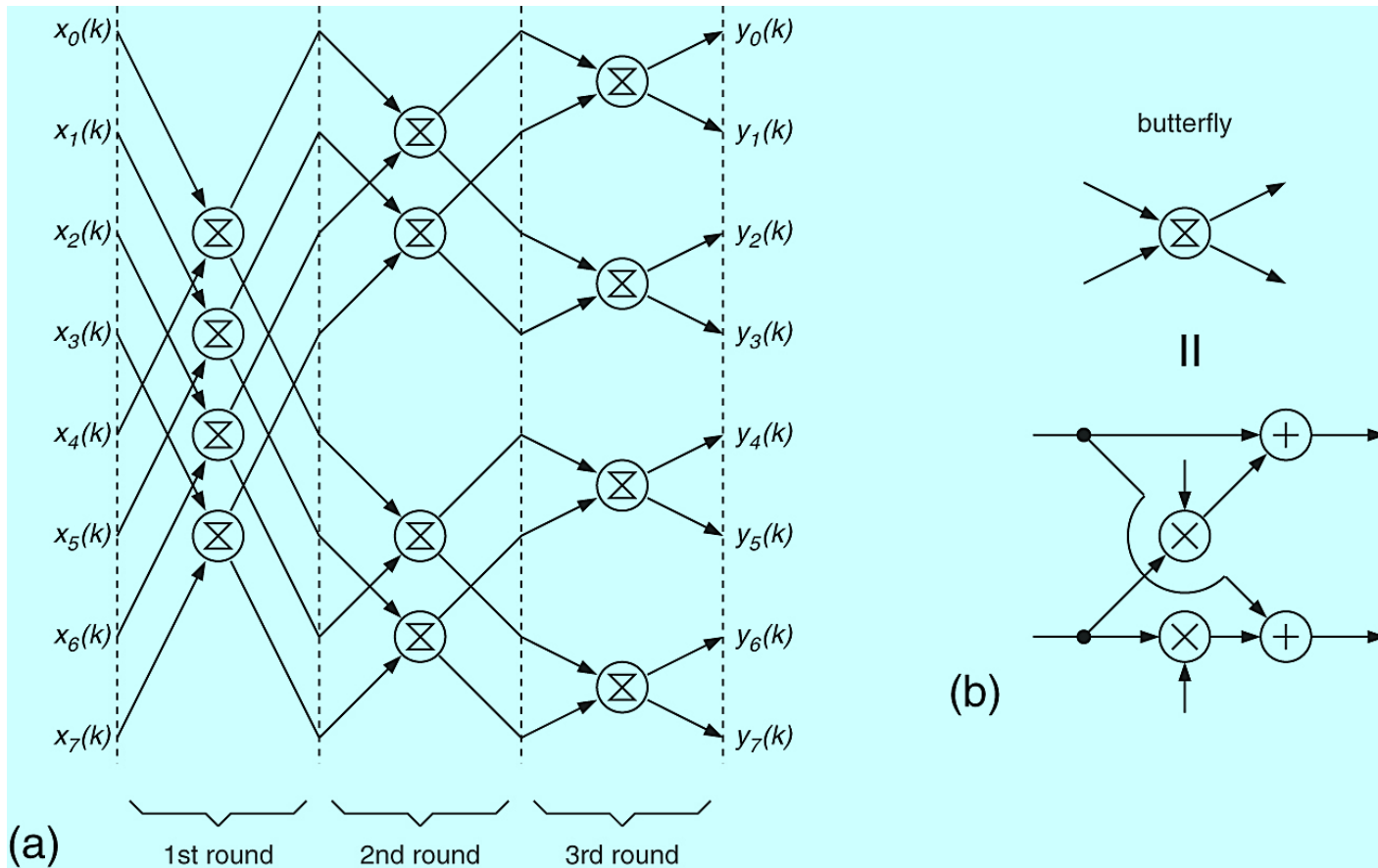
⇒ Time sharing

- ⇒ is most favorable when one **monofunctional datapath** proves sufficient because all streams are to be processed in exactly the same way.
- ⇒ is unattractive when subfunctions are very disparate because no savings can be obtained from concentrating their processing into one multifunctional datapath.
- ⇒ refrains from taking advantage of the parallelism inherent in the original problem.
- ⇒ may be viewed as antithetic to replication.



Time Sharing (4/4)

➔ Example: an 8-point FFT processor

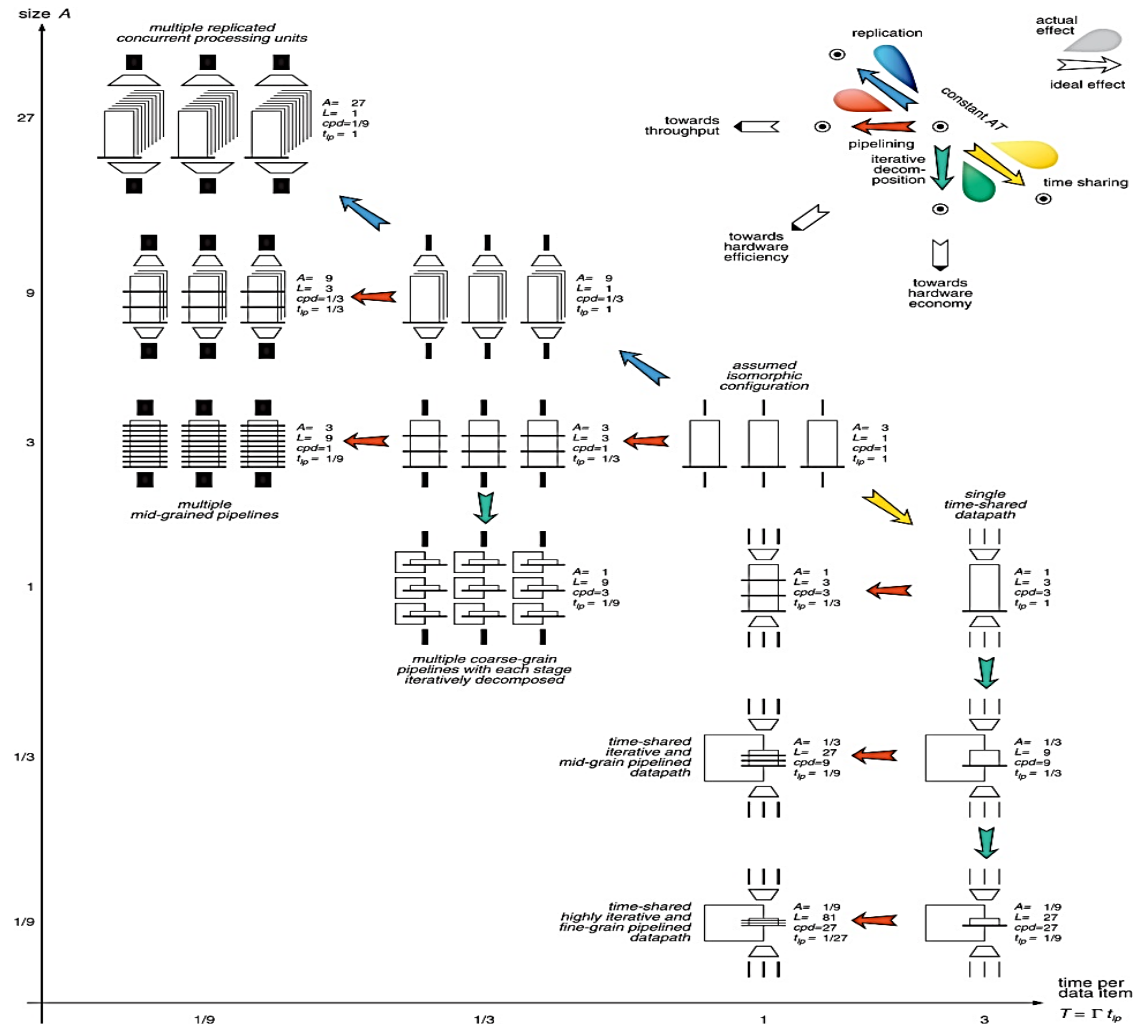


A Concluding Remark

➡ A roadmap illustrating the four universal transforms for tailing combinational hardware

➡ Assume that $A_{reg} = 0$, $t_{reg} = 0$, and no controlling overhead.

➡ $T = cpd \times t_{lp}$
(cpd: cycle per datum)



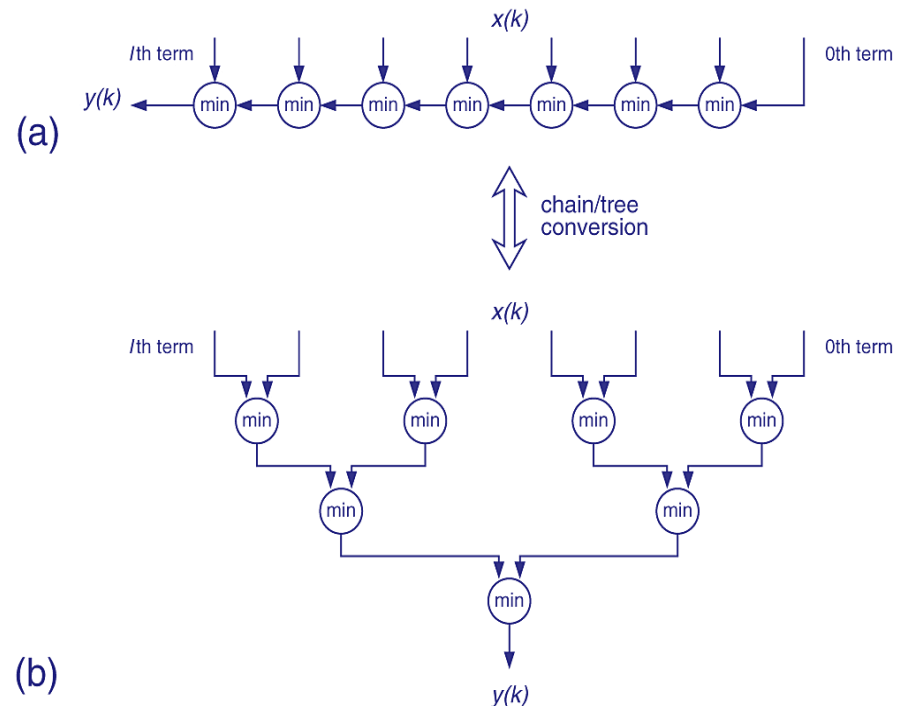
Universal vs. Algebraic Transform

- ➡ **Universal transforms:** Whether and how to apply them can be decided from the DDG alone, no matter what operations the vertices stand for.
 - ➡ Any two computations of isomorphic DDGs can be solved by the same architecture just with the vertices interpreted differently.
 - ➡ Examples: iterative decomposition, pipelining, replication, time sharing, etc.
- ➡ **Algebraic transforms:** Take advantage of specific algebraic properties of the operations involved.
 - ➡ Associativity transform, commutativity transform, Horner's scheme (for evaluating polynomials), Karatsuba multiplication (for wide data words), superposition, DeMorgan's theorem, ...



Associativity Transform

- ➡ The most valuable algebraic property from an architectural point of view
 - ➡ Convert a chain structure into a tree or vice versa. (The longest path is shortened from $(I - 1)$ to $\lceil \log_2 I \rceil$.)
 - ➡ Reorder operations such as to accommodate input data that arrive later than others do.
 - ➡ Reverse the order if execution in a chain.
 - ➡ Relocate operations from within a recursive loop to outside the loop.
 - ➡ More ...



Equivalence Transform - Summary

- ➔ Equivalence transforms that help optimize combinational computations
 - ➔ Iterative decomposition, pipelining, replication, and time sharing are based on DDG, which make no assumptions on the nature of computations carried out in its vertices.
 - ➔ Iterative decomposition and time sharing are most effective when a computational unit can be reused several times.
 - ➔ Pipelining is generally superior to replication. Coarse grain pipelining improves throughput dramatically, but benefits decline as more and more stages are included.
 - ➔ Pipelining and iterative decomposition are complementary, they both can contribute to lowering the AT product.
 - ➔ Lowering AT always implies cutting down the longest path t_{lp} .
 - ➔ Important power saving can be obtained from operating CMOS logic with a supply voltage below its nominal value.



Temporary Storage of Data

➡ Why and when do we need to store data?

- ➡ The data processing algorithm is of sequential nature and, therefore, asks for **functional** memory.
- ➡ **Nonfunctional** storage got introduced into the circuit as a consequence from architectural transformations.

➡ Architectural options for temporary storage of data

- ➡ **On-chip registers** built from individual flip-flops or latches.
- ➡ **On-chip memory**: SRAM or possibly embedded DRAM.
- ➡ **Off-chip memory**: SRAM or DRAM catalog part.

➡ Differences that impact high-level design decisions

- ➡ One-at-a-time versus all-at-a-time data access patterns, available memory configurations and area occupation, storage capacities, wiring and the costs of going off-chip, energy efficiency, latency and timing, ...



Data Access Patterns

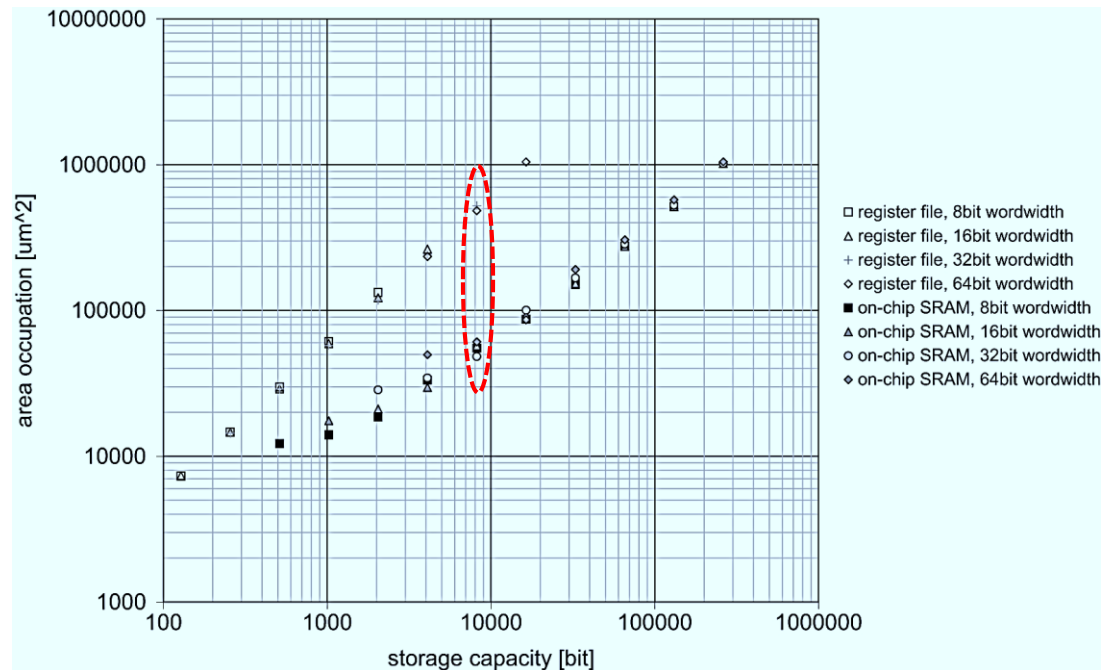
- ➡ RAMs impose access one data word after the other
 - ➡ Fine in architectures obtained from **iterative decomposition** and **time sharing**.
 - ➡ Perfect match for microprocessors (**operating sequence of fetch, load, execute, and store**)
- ➡ Registers allow for simultaneous access to all stored data words
 - ➡ Mandatory in high-throughput architectures obtained from **pipelining**, **retiming**, and **loop unrolling** where data are kept moving in every computation cycle.



How to Choose Memory? (1/3)

➔ Available memory configurations

- ➔ In a typical commodity DRAM, roughly 60% of the die is occupied by storage cells.
- ➔ Area occupation of registers and on-chip RAMs for a 130 nm CMOS.
- ➔ Register file: flip-flops.
- ➔ DRAM: 1-T memory cell plus 1C
- ➔ SRAM: 6-T memory cell (typically).
- ➔ Latency and timing



How to Choose Memory? (2/3)

⇒ Wiring and the costs of going off-chip

- ⇒ Off-chip memories add to pin count, package count, and board space.
 - ⇒ Extra parasitic capacitances
 - ⇒ Extra delay and energy dissipation
- ⇒ Commodity RAMs impose bidirectional pads with special attention required.
 - ⇒ Stationary and transient drive conflicts must be avoided.
 - ⇒ ATE must be made to alternate between read and write modes with no physical access to any control signal within the chip.
 - ⇒ Test patterns must address bidirectional operation and high-impedance states.
 - ⇒ Electrical and timing measurements become more complicated.



How to Choose Memory? (3/3)

➔ Comparisons for temporary data storage

architectural option	o n - c h i p				off-chip commodity DRAM
	bistables flip-flop	latch	embedded SRAM	DRAM	
fabrication process	compatible with logic				optimized
devices in each cell	20...30T	12...16T	6T	1T1C	1T1C
cell area per bit [F^2]	1700...2800	1100...1800	135...170	18...30*	6...8
extra circuit overhead	none		1.3 ≤ factor ≤ 2		off-chip
memory refresh cycles	n o n e				y e s
extra package pins	none		none		addr. & data bus
nature of wiring	multitude of local lines		on-chip busses		package & board
bidirectional busses	none		optional		mandatory
access to data words	all at a time		one at a time		
available configurations	any		restricted		
energy efficiency	good		fair	poor	very poor
latency and paging	none		no fixed rules		yes
impact on clock period	minor		substantial		severe

* As low as 6...8 for processes that accomodate 3D capacitors (4 to 6 extra masks)



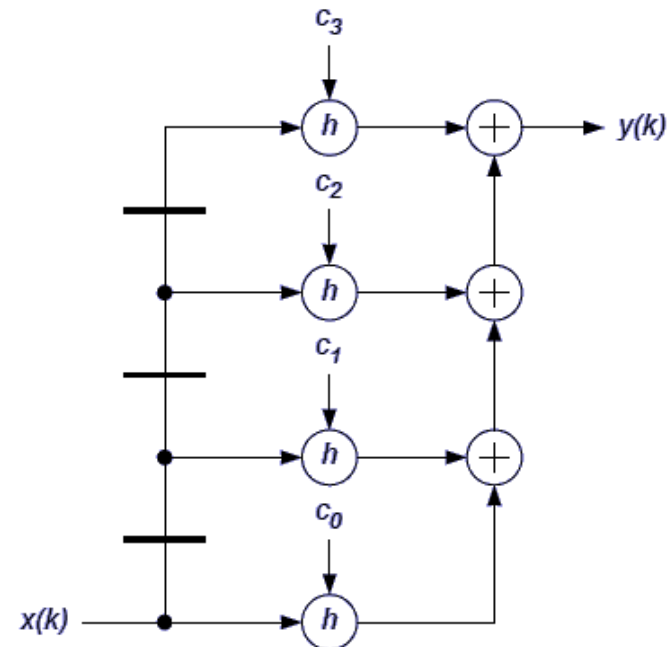
Temporary Data Storage - Summary

- ➔ There is no such thing as an optimal solution for temporary storage of data, what is best strongly depends on the situation and requirements.
 - ➔ Only registers allow for simultaneous access to all data, but occupy a lot of die area per bit.
 - ➔ SRAMs can hold more significant quantities of data than registers but are slower than registers, yet faster than DRAMs.
 - ➔ DRAMs require periodical refresh, consuming power even when idle.
 - ➔ DRAM and flash memories are cost-efficient for large data quantities.
 - ➔ Flash is used for permanent storage, but is much slower than RAM.
 - ➔ Commodity memories offer virtually unlimited capacities at low costs, but are associated with speed, energy and other penalties.



Transforms for NR Computations

- ➔ A computation is termed (sequential and) non-recursive if
 - ➔ Result is dependent on past arguments, not just present.
 - ➔ Edges with weights greater than zero are present in the DDG.
 - ➔ DDG is free of circular paths.
 - ➔ Example: A nonlinear time-invariant third-order correlator
 - ➔ Can you do better in terms of speed and area?
 - ➔ Pipelining helps boost throughput but is rather inefficient in this case.

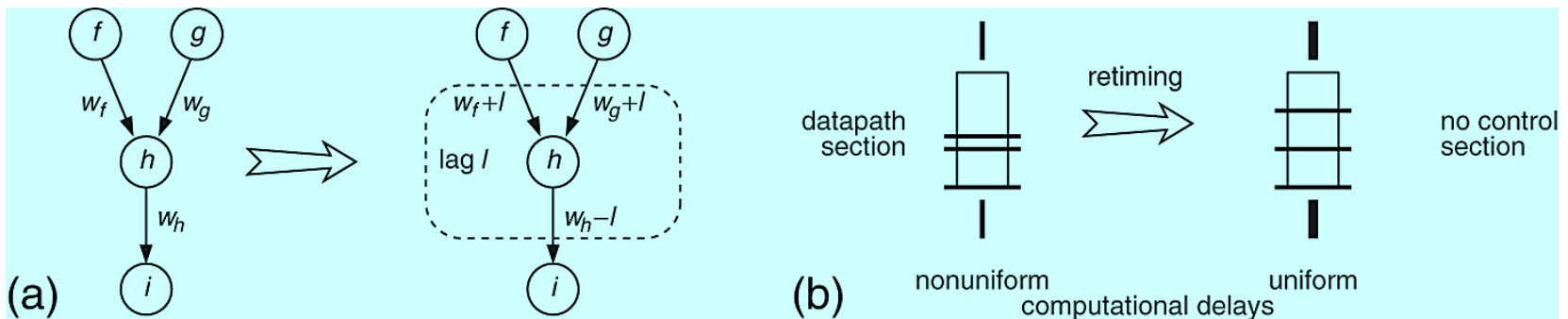


Retiming (1/2)

➡ Basic mindset: register relocation for a higher computation rate.

- ➡ The goal is to equalize computation delays between two registers (**register balancing**) without affecting functionality.
- ➡ Retiming does not affect latency, but may alter the number of registers in the circuit depending on the structure of DDG. (also switching activities, fanouts, and node capacitance)
- ➡ Think about the equivalent circuit in circuit theory.

➡ An example for $l = 1$.



Retiming (2/2)

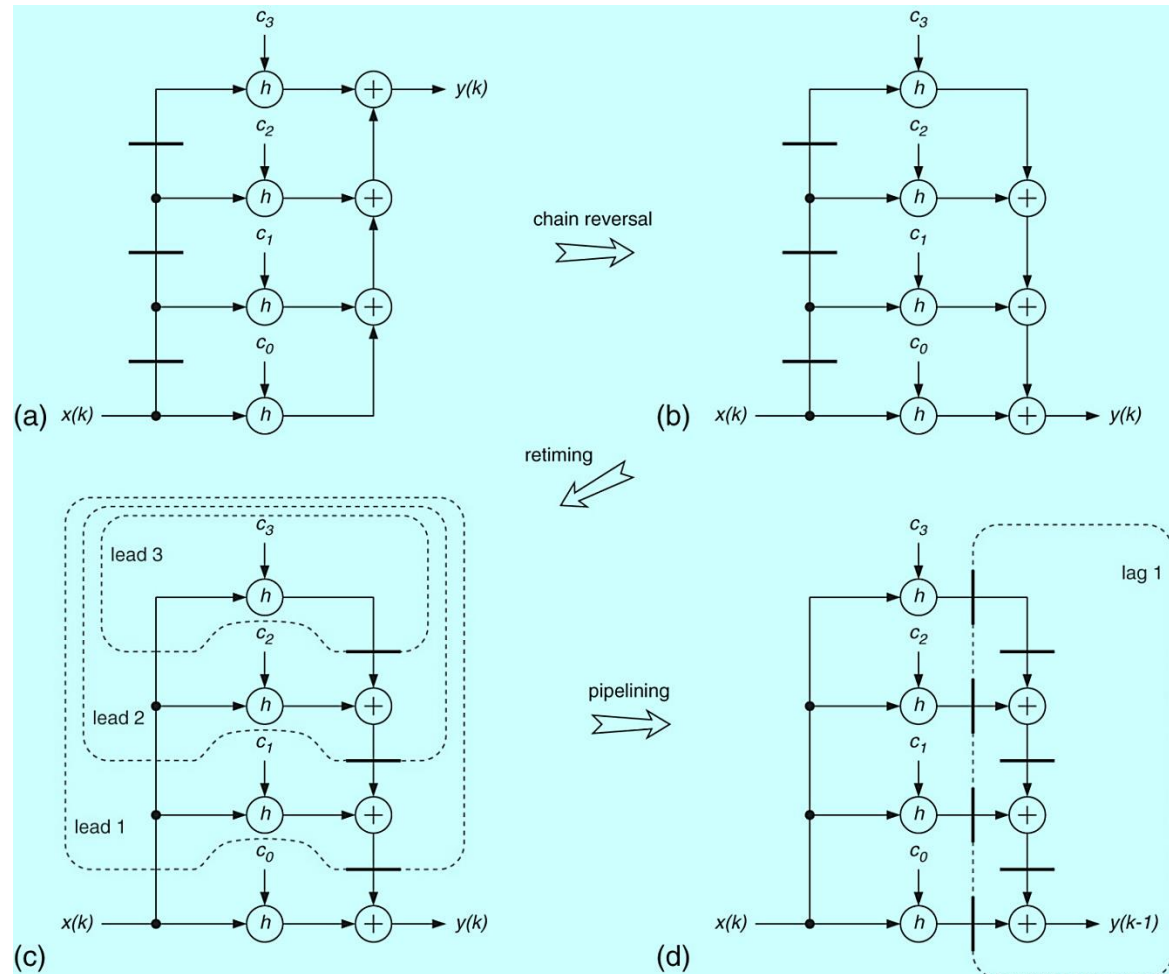
⇒ Formal rules: To be legal, any retiming must observe the rules below.

- ⇒ Neither outputs nor sources of time-varying inputs may be part of a **supervertex** that is to be retimed. Sources of constant data do not change in any way subjecting to retiming.
- ⇒ When a supervertex is assigned a lag (lead) by l computation cycles, the weights of all its incoming edges are incremented (decremented) by l and the weights of all its outgoing edges are decremented (incremented) by l .
- ⇒ No edge weight may be changed to assume a negative value (should be a **feasible solution**).
- ⇒ Any circular path must always include at least one edge of strictly positive weight (**roundtrip weights will never change**).



Retiming – An Example (1/2)

➔ An example:
a nonlinear
time-
invariant
third-order
correlator



Retiming – An Example (2/2)

⇒ An example: a nonlinear time-invariant third-order correlator (*cont'd*)

Key characteristics	Architectural variant			
	original (a)	reversed (b)	+ retimed (c)	+ pipelined (d)
arithmetic units	$(N + 1)A_h + NA_+$	<i>idem</i>	<i>idem</i>	<i>idem</i>
functional registers	NA_{reg}	<i>idem</i>	<i>idem</i>	<i>idem</i>
nonfunctional registers	0	<i>idem</i>	<i>idem</i>	$(N + 1)A_{reg}$
cycles per data item Γ	1	<i>idem</i>	<i>idem</i>	<i>idem</i>
longest path delay t_{lp}	$t_{reg} + t_h + N t_+$	<i>idem</i>	$t_{reg} + t_h + t_+$	$t_{reg} + \max(t_h, t_+)$
for $N = 3$ [ns]	9.5	<i>idem</i>	5.5	3.5
for $N = 30$ [ns]	63.5	<i>idem</i>	5.5	3.5
latency L	0	<i>idem</i>	<i>idem</i>	1

⇒ New benefits:

- ⇒ Long path delay is greatly reduced at little hardware costs.
- ⇒ Maximum operating speed is no longer a function of the correlation order N .



Pipelining & Retiming

- ⇒ Pipelining uses the same rules as for retiming except
 - ⇒ Any supervertex to be assigned a lag (lead) must include all outputs (all time-varying inputs).
- ⇒ Both transforms aim at shortening the longest path.
- ⇒ Pipelining increases latency as registers get added.
- ⇒ Retiming leaves latency unchanged as registers get relocated.
- ⇒ Note that a DDG is termed **systolic** if the edge weight between any two vertices is one or more.



Iterative Decomposition & Time Sharing

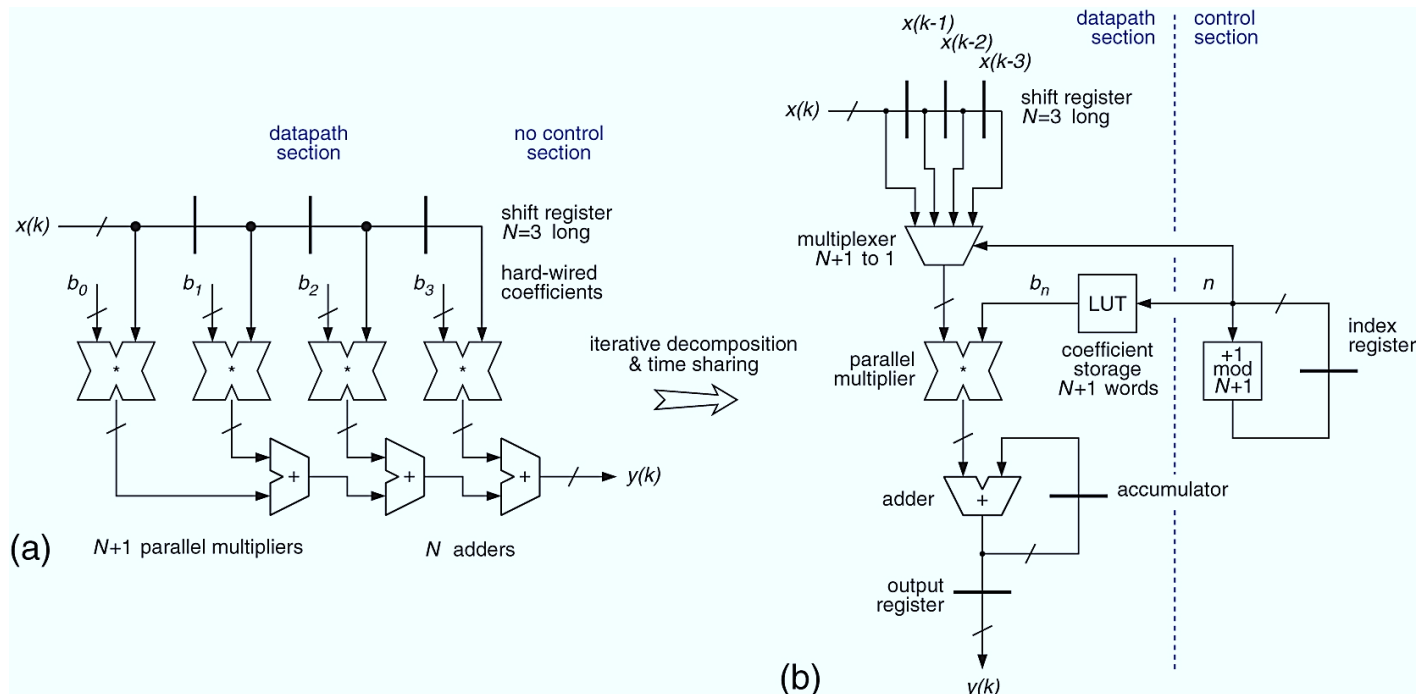
- ➡ Decomposing and time sharing sequential computations is straightforward and can significantly reduce datapath hardware.
 - ➡ Only computation elements are multiplexed - **functional memory requirements** remain the same as in the isomorphic architecture (**memory bound**).
 - ➡ More uniform combinational depth reduces glitching activity.
 - ➡ Extra multiplexers necessary to route, recycle, collect and/or redistribute data.
 - ➡ Extra counter or finite state machine required to control the datapath.



Time Sharing – An Example

➔ An example: a 3-order transversal filter and its time-shared implementation

➔ an adder to sum up the partial products, an accumulator to store intermediate sum, LUT to keep coefficients, ...



Transforms for Recursive Computations

- ➡ A computation is termed (sequential and) recursive if
 - ➡ the result is dependent on earlier outcomes of the computation itself,
 - ➡ edges with weights greater than zero are present in the DDG, and
 - ➡ circular paths (of non-zero weight) exist in the DDG.

➡ An example:

- ➡ A first-order recursive equation

$$y(k) = ay(k-1) + x(k)$$

- ➡ The transfer function in z domain

$$H(z) = \frac{Y(z)}{X(z)} = \frac{1}{1 - az^{-1}}$$

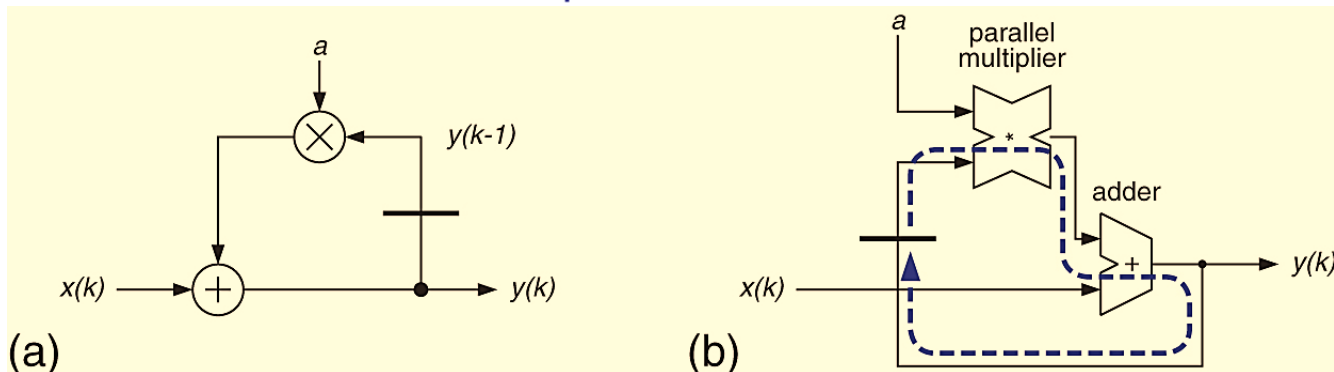


LT-invariant 1st-order Feedback Loop (1/6)

➡ Expression: $y(k) = ay(k-1) + x(k)$

- ➡ Recursion demands that the result from the actual computation cycle be available no later than the next input sample.
 - ➡ It is not obvious to use replication or to insert pipeline registers without altering the overall transfer function and I/O mapping.

➡ The iteration bound: $\sum_{loop} t = t_{reg} + t_* + t_+ = t_{lp} \leq T_{cp}$



LT-invariant 1st-order Feedback Loop (2/6)

➡ How to relax the timing constraint by inserting additional latency registers into the feedback loop?

➡ Algorithmic transformation

$$\text{Given } H(z) = \frac{Y(z)}{X(z)} = \frac{1}{1 - az^{-1}}$$

$$\text{Assume } H(z) = \frac{Y(z)}{X(z)} = \frac{N(z)}{1 - a^p z^{-p}}$$

where $N(z)$ is here to compensate for the changes due to the new denominator.

Therefore, $N(z)$ can be obtained by recalling the sum of geometric series as

$$N(z) = \frac{1 - a^p z^{-p}}{1 - az^{-1}} = \sum_{n=0}^{p-1} a^n z^{-n}$$



LT-invariant 1st-order Feedback Loop (3/6)

➡ The new transfer function becomes

$$H(z) = \frac{\sum_{n=0}^{p-1} a^n z^{-n}}{1 - a^p z^{-p}}$$

which corresponds to the new (lookahead) recursion in the time domain as

$$y(k) = a^p y(k - p) + \sum_{n=0}^{p-1} a^n x(k - n)$$

➡ An example for $p = 4$:

$$y(k) = a^4 y(k - 4) + a^3 x(k - 3) + a^2 x(k - 2) + a x(k - 1) + x(k)$$

$$H(z) = \frac{1 + az^{-1} + a^2 z^{-2} + a^3 z^{-3}}{1 - a^4 z^{-4}} \quad \text{in lieu of} \quad \frac{1}{1 - az^{-1}}$$



LT-invariant 1st-order Feedback Loop (4/6)

➡ Some thoughts about the new result

$$y(k) = a^4 y(k-4) + a^3 x(k-3) + a^2 x(k-2) + ax(k-1) + x(k)$$

- ➡ Denominator has been widened to include p unit delays rather than one.
- ➡ Numerator stands for a feedforward circuit that is amenable to pipelining.
- ➡ Particularly elegant and efficient solutions exist when p is an integer power of 2 because of the lemma

$$\sum_{n=0}^{p-1} a^n z^{-n} = \prod_{m=0}^{\log_2 p - 1} (a^{2^m} z^{-2^m} + 1) \quad p = 2, 4, 8, 16, \dots$$

- ➡ For $p = 4$,

$$H(z) = \frac{(1 + az^{-1})(1 + a^2 z^{-2})}{1 - a^4 z^{-4}}$$

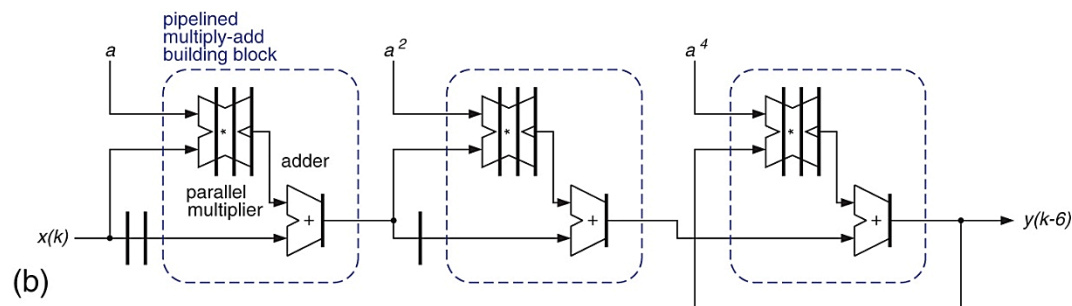
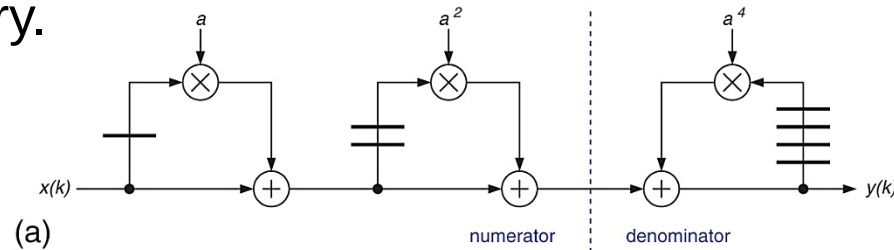


LT-invariant 1st-order Feedback Loop (5/6)

➡ Implement the transfer (lookahead) function for $p = 4$

$$H(z) = \frac{(1 + az^{-1})(1 + a^2z^{-2})}{1 - a^4z^{-4}}$$

➡ The DDG unfolded for $p = 4$ and the high-performance architecture derived by pipelining, retiming, and shimming if necessary.



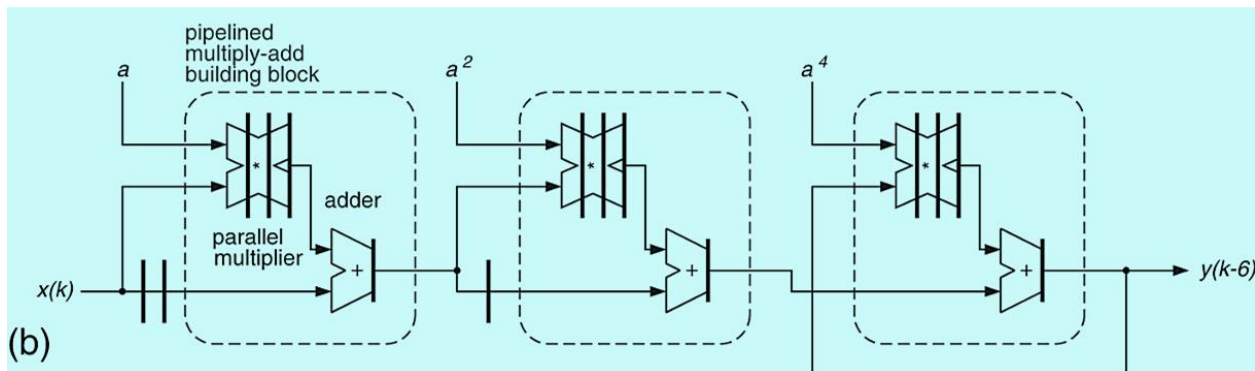
LT-invariant 1st-order Feedback Loop (6/6)

➡ Performance and cost analysis

- ➡ Low bound for the circuit size

$$A(p) \geq (\log_2 p + 1)A_f + p(\log_2 p + 1)A_{reg}$$

- ➡ The speedup of loop unfolding tends to diminish when it is difficult to balance the delay in combinational logic for $p \gg 1$.
- ➡ A hidden cost factor is due to finite precision arithmetics. (The larger number of roundoff operations with respect to the initial configuration leads to more quantization errors.)
- ➡ Is it energy efficient?



Transforms for High-order Loops

- ➡ Do not attempt to unfold loops of arbitrary order directly. Use a common technique from digital filter design.
- ➡ Any higher-order transfer function can be factored into a product of second- and first-order terms.
 - ➡ The DDG takes the form of cascaded 2nd- and 1st-order sections.
 - ➡ Cascaded structures are known to be less sensitive to quantization of coefficients and signals than direct forms.



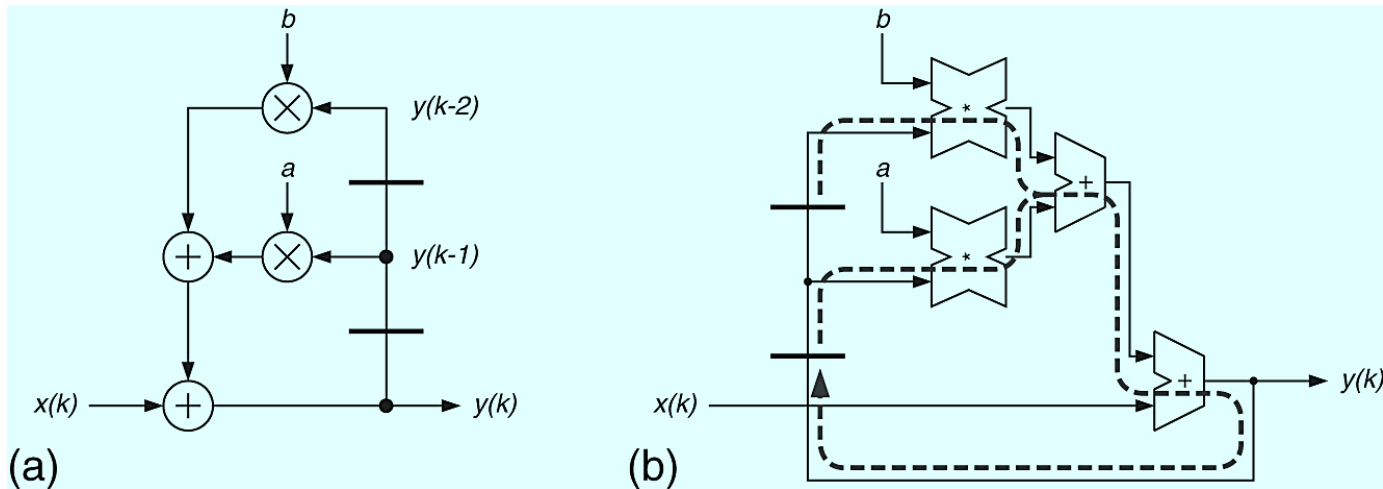
LT-invariant 2nd-order Feedback Loop (1/3)

➡ Recursive expression:

$$y(k) = ay(k-1) + by(k-2) + x(k)$$

➡ The z-domain representation:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{1}{1 - az^{-1} - bz^{-2}}$$



LT-invariant 2nd-order Feedback Loop (2/3)

➡ Given

$$H(z) = \frac{Y(z)}{X(z)} = \frac{1}{1 - az^{-1} - bz^{-2}}$$

unfolding is obtained from multiplying numerator and denominator by an adequate factor.

➡ For $p = 2$,

$$H(z) = \frac{1 + az^{-1} - bz^{-2}}{1 - (a^2 + 2b)z^{-2} + b^2z^{-4}}$$

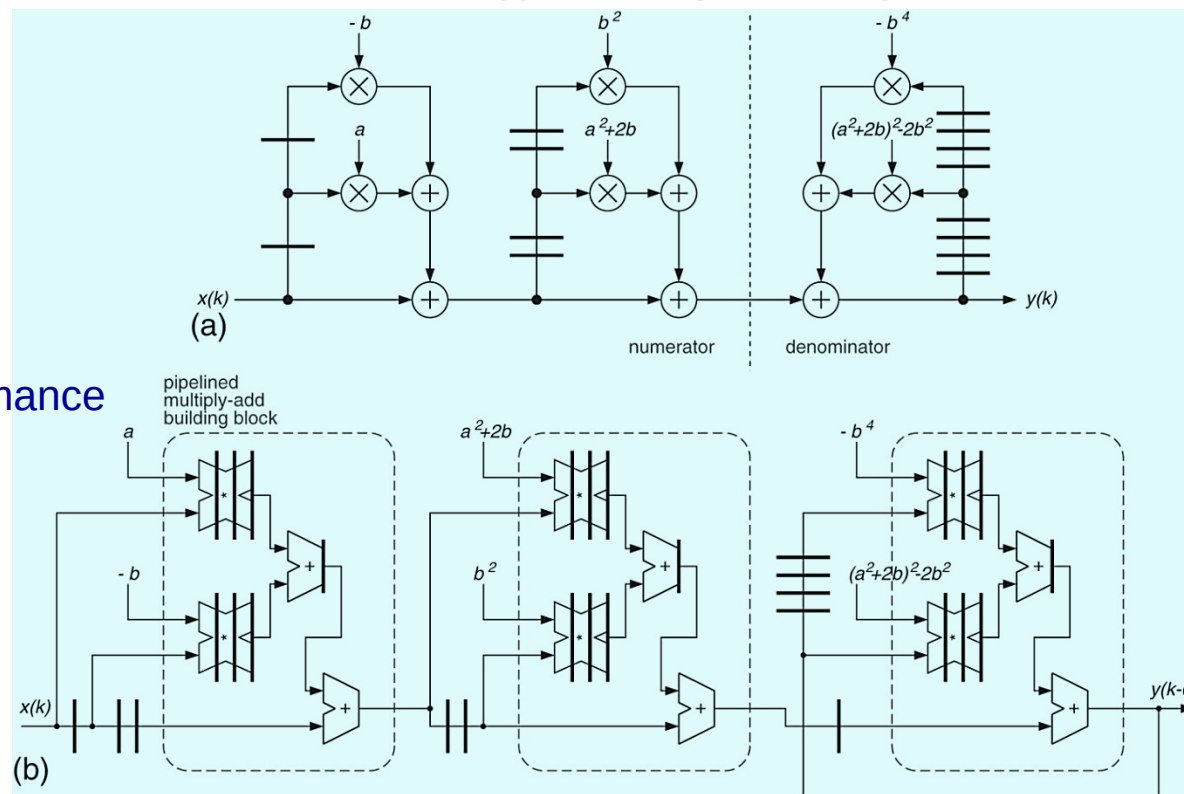
➡ For $p = 4$,

$$H(z) = \frac{(1 + az^{-1} - bz^{-2})(1 + (a^2 + 2b)z^{-2} + b^2z^{-4})}{1 - ((a^2 + 2b)^2 - 2b^2)z^{-4} + b^4z^{-8}}$$



LT-invariant 2nd-order Feedback Loop (3/3)

➔ Given $H(z) = \frac{(1 + az^{-1} - bz^{-2})(1 + (a^2 + 2b)z^{-2} + b^2z^{-4})}{1 - ((a^2 + 2b)^2 - 2b^2)z^{-4} + b^4z^{-8}}$



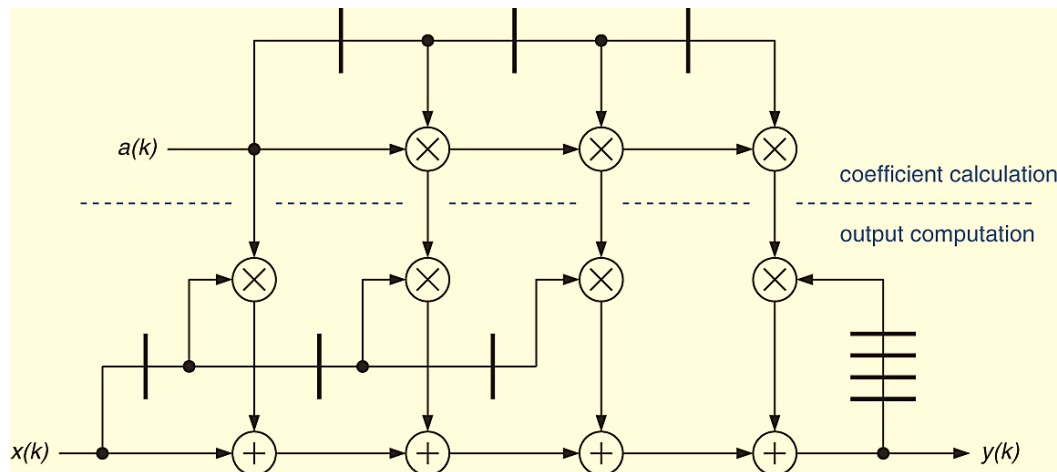
$$A(p) \geq (\log_2 p + 1)A_f + (2p(\log_2 p + 1))A_{reg}$$



LT-variant 1st-order Feedback Loop

- ➡ Expression: $y(k) = a(k)y(k-1) + x(k)$
 - ➡ Coefficient terms must be calculated on-line requiring extra hardware.
- ➡ Unfolding for $p = 2$, we have

$$y(k) = a(k)a(k-1)y(k-2) + a(k)x(k-1) + x(k)$$
- ➡ The DDG for $p = 4$



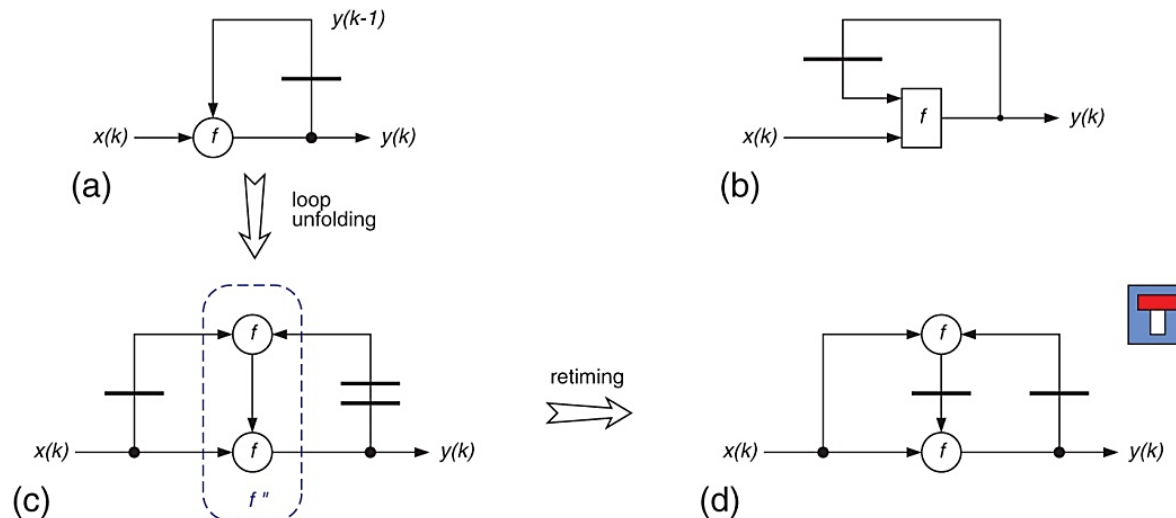
Nonlinear/General Loops (1/2)

- ➡ The principle of superposition does not hold.
- ➡ General expression of a first-order recursion:

$$y(k) = f(y(k-1), x(k))$$

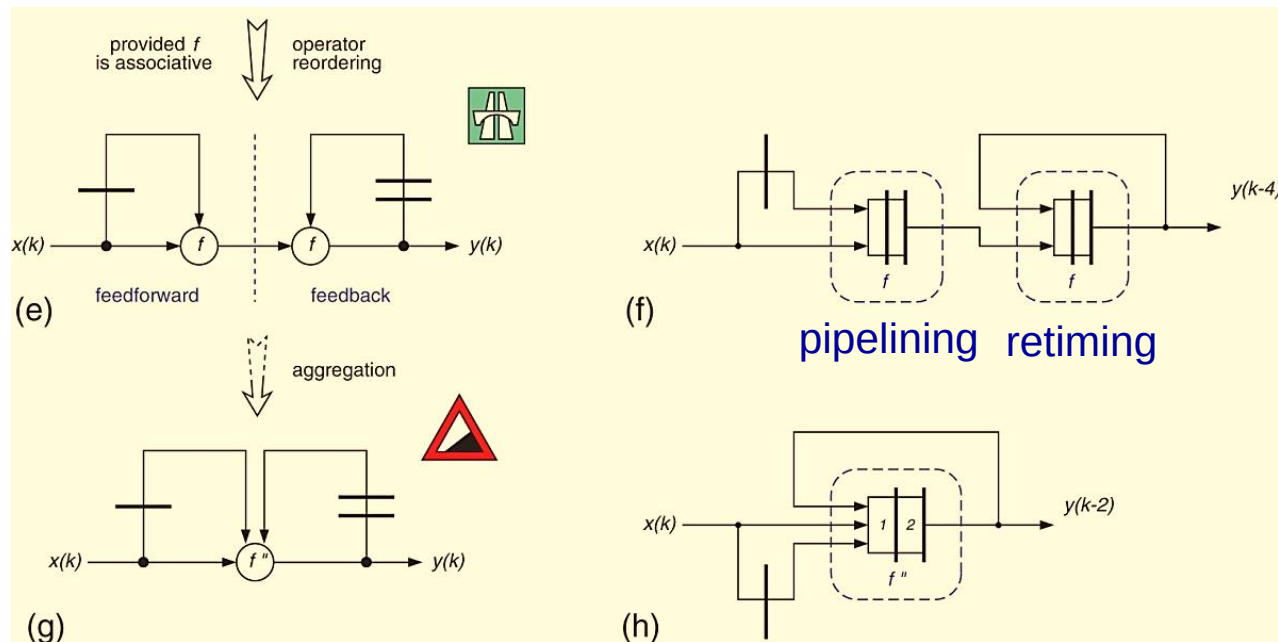
Unfolding for $p = 2$, we have

$$y(k) = f(f(y(k-2), x(k-1)), x(k))$$



Nonlinear/General Loops (2/2)

➡ Given $y(k) = f(f(y(k-2), x(k-1)), x(k))$

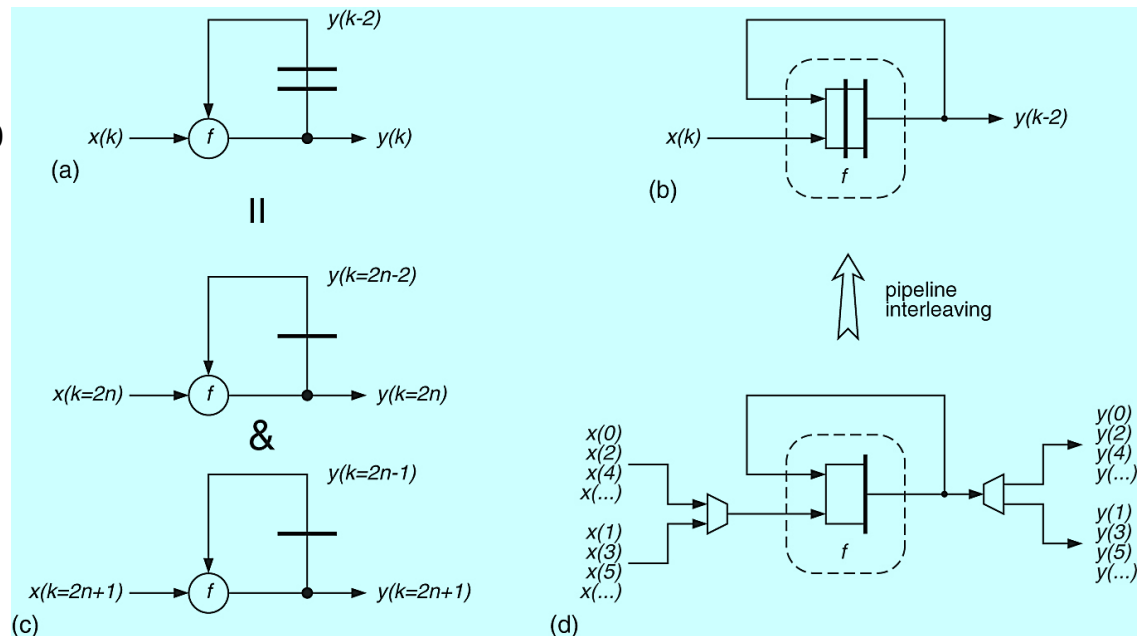


⇒ All successful architectural transforms for recursive computations take advantage of algorithmic properties such as linearity, fixed coefficients, associativity, limited word width or a very limited set of register states.



Pipeline Interleaving

- ➡ Any attempt to insert an extra register into a feedback loop destroys the equivalence between original and pipelined computations unless its effect is compensated.
- ➡ What happens if we do just that for a first-order recursion? **That is, $y(k) = f(y(k-2), x(k))$**
 - ➡ Can restate the ensuing I/O mapping as two separate recursions with no interaction between even and odd items.
 - ➡ Cut feedback loop for interleaving.



* Pipeline interleaving does not obviously qualify as an equivalence transform.

Loop Unfolding - Summary

- ➡ Loop unfolding can significantly improve the throughput of linear time-invariant feedback calculations.
 - ➡ The rapid growth of overall circuit size tends to limit economically practical unfolding degrees to fairly low values, say $p = 2 \dots 8$. (Increased latency is also an issue.)
 - ➡ The large number of roundoff operations resulting from loop unfolding must be compensated for by using longer word widths. (increase circuit area and computation time)
 - ➡ Nonlinear feedback loops are, in general, not amenable to throughput multiplication by applying unfolding techniques.
 - ➡ A notable exception exists when the loop function is associative.
 - ➡ Pipeline interleaving is not an equivalence transform but nevertheless helpful where multiple data streams undergo the same processing independently from each other.



Generalization to Word/Bit Levels

➡ What if we try to apply equivalence transforms at levels of abstraction other than the word level?

Level of abstraction	Granularity	Relevant items	
		Operations	Data
Architecture	○	subtasks, processes	time series, pictures
Word	○	arithmetic/logic ops	words, samples, pixels
Bit	·	gate-level ops	individual bits

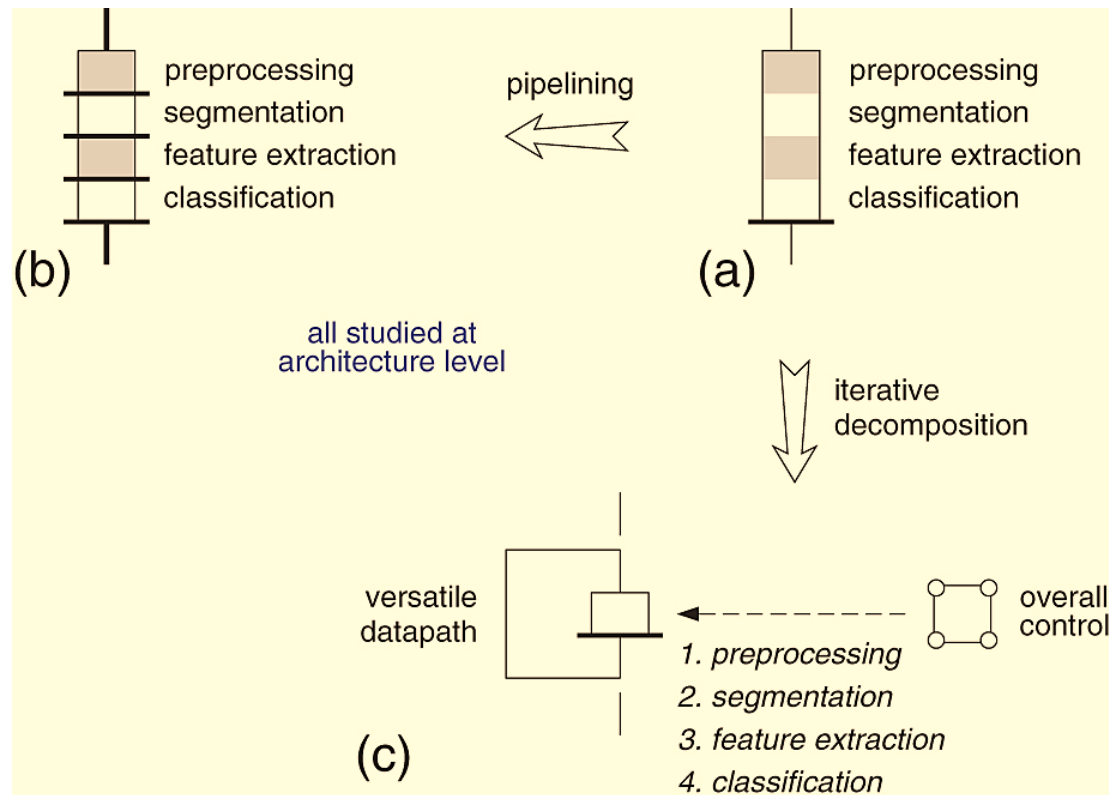
➡ Note that DDGs are not concerned with the granularity of operations and data.

➡ **Observation:** Everything we have learned for equivalent architecture transforms is applicable at multiple levels of abstraction.



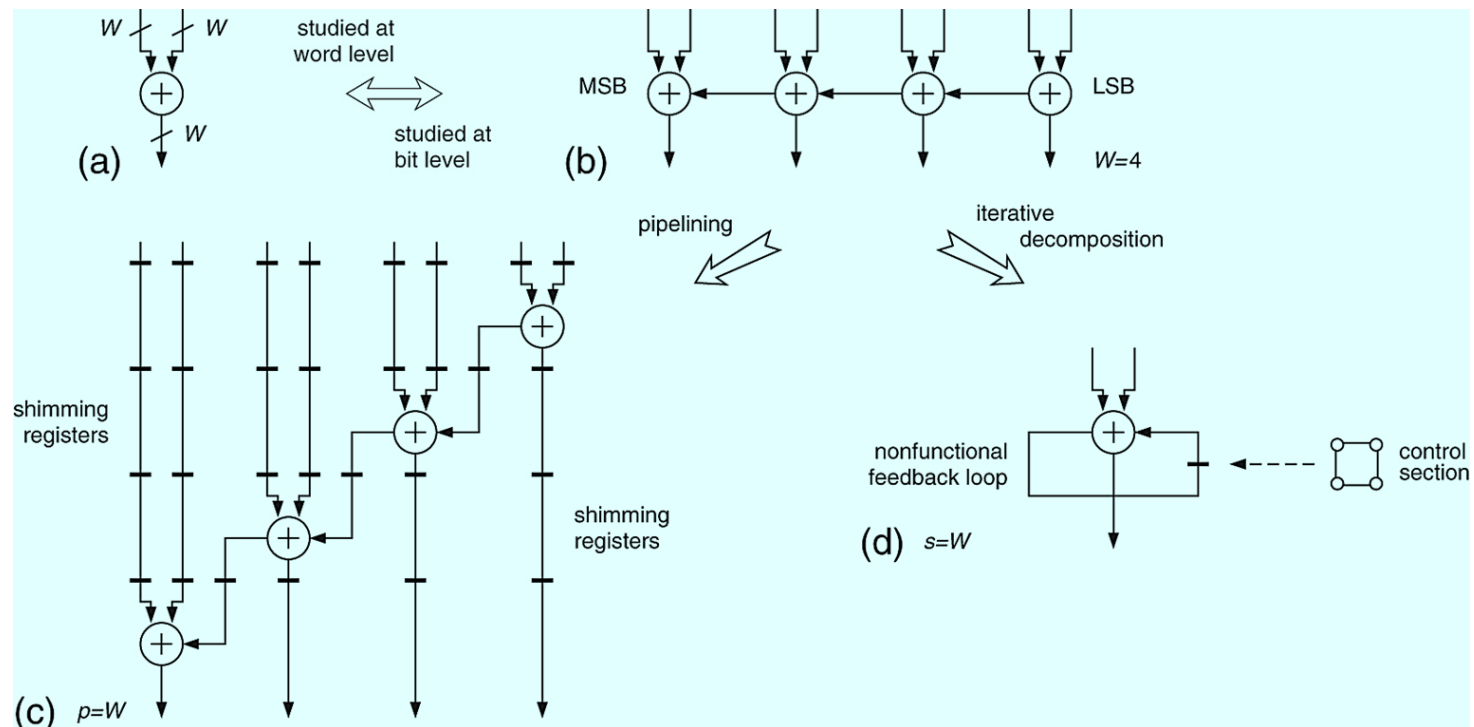
Transforms at Architecture Level

- ➡ A typical pattern recognition system transformed at the architecture level



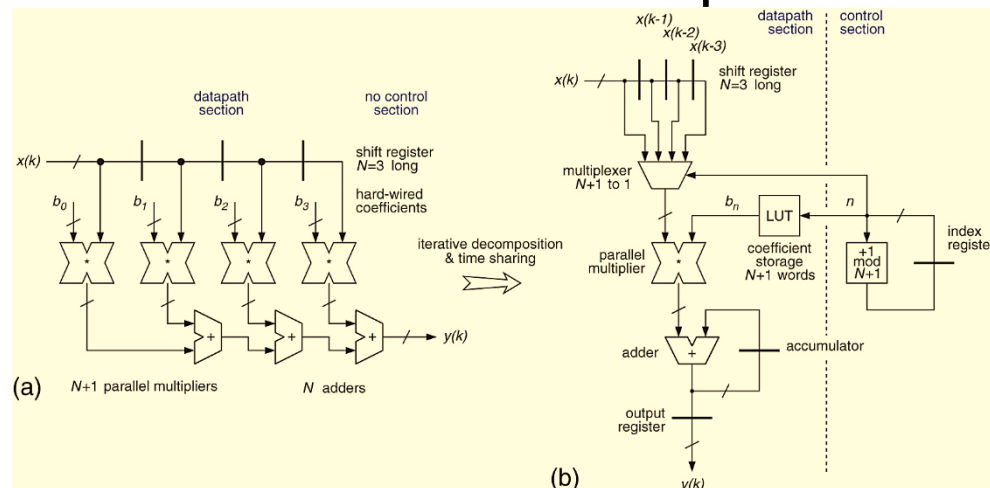
Transforms at Bit Level

- ➡ A 4-bit ripple carry adder transformed at the bit level
 - ➡ A gate-level pipelined version makes it possible to operate the circuit at very high computation rate.



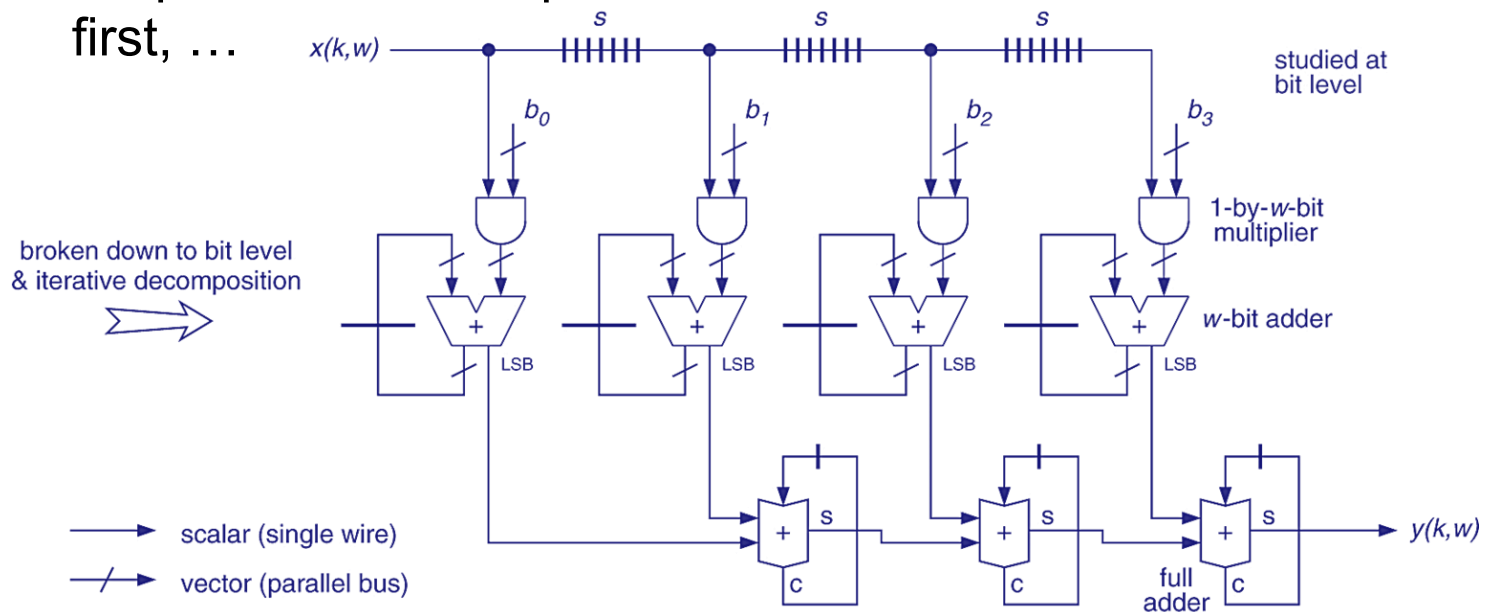
Bit-Serial Architectures (1/2)

- ➔ An uncommon architectural concept where one bit from each data word is being operated upon at a time until all bits have been processed.
 - ➔ Word-level operations broken into bit-level operations.
 - ➔ Iterative decomposition (and possibly pipelining too).
 - ➔ Good for addition and multiplication by a constant
- ➔ The 3rd-order transversal filter operated at word level



Bit-Serial Architectures (2/2)

- ➡ The 3rd-order transversal filter operated at bit level
- ➡ **Pros:** simple control, shallow computation path, serial non-local data communication, local data circulation, **good for FPGA design**, ...
 - ➡ **Cons:** not flexible, idle computation for operations without full precision, incompatible with word-oriented RAMs, LSB first, ...



Distributed Arithmetic (1/3)

- ➡ Basic concept: breaking the word-level operations into bit level using algebraic transforms to get rid of multiplication
- ➡ Consider the inner product operation

$$y = \sum_{k=0}^{K-1} c_k x_k$$

with c_k a fixed coefficient, x_k scaled such that $|x_k| < 1$ and coded with W bits in 2's-complement format.

Therefore,

$$x_k = -x_{k,0} + \sum_{w=1}^{W-1} x_{k,w} 2^{-w} \quad y = \sum_{k=0}^{K-1} c_k \left(-x_{k,0} + \sum_{w=1}^{W-1} x_{k,w} 2^{-w} \right)$$



Distributed Arithmetic (2/3)

- ⇒ With distributive law, commutative law, and reversed order of summation, we have

$$y = \sum_{k=0}^{K-1} c_k(-x_{k,0}) + \sum_{w=1}^{W-1} \left(\sum_{k=0}^{K-1} c_k x_{k,w} \right) 2^{-w}$$

- ⇒ The pivotal equation
$$\sum_{k=0}^{K-1} c_k x_{k,w} = p(w)$$

- ⇒ Implies that for any given bit position w , calculating the sum of products takes one bit from each of the K data words x_k , so $p(w)$ can take on no more than 2^K distinct values. With the coefficients c_k constant, all those values can be kept in a lookup table (LUT). (p156)

- ⇒ Therefore,
$$y = -p(0) + \sum_{w=1}^{W-1} p(w) 2^{-w}$$

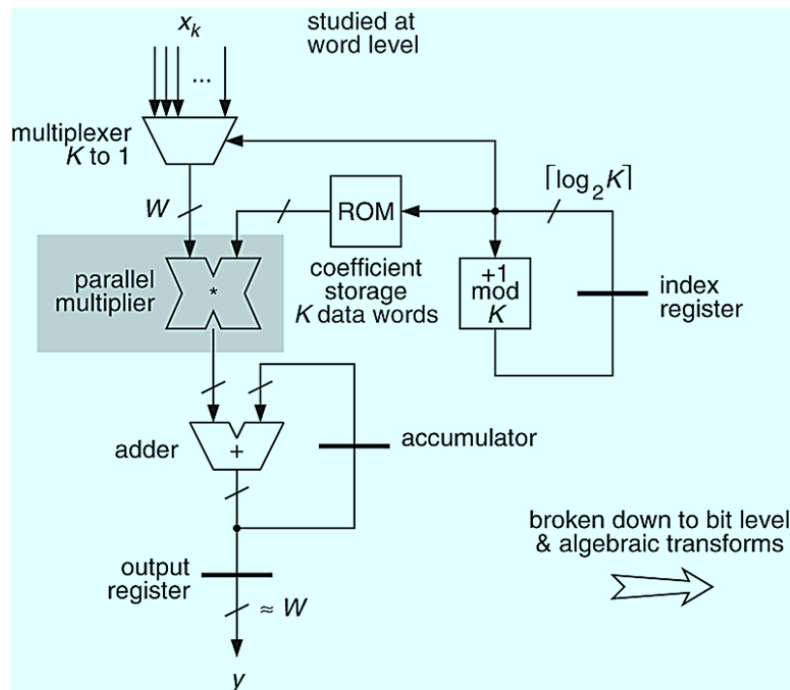


Distributed Arithmetic (3/3)

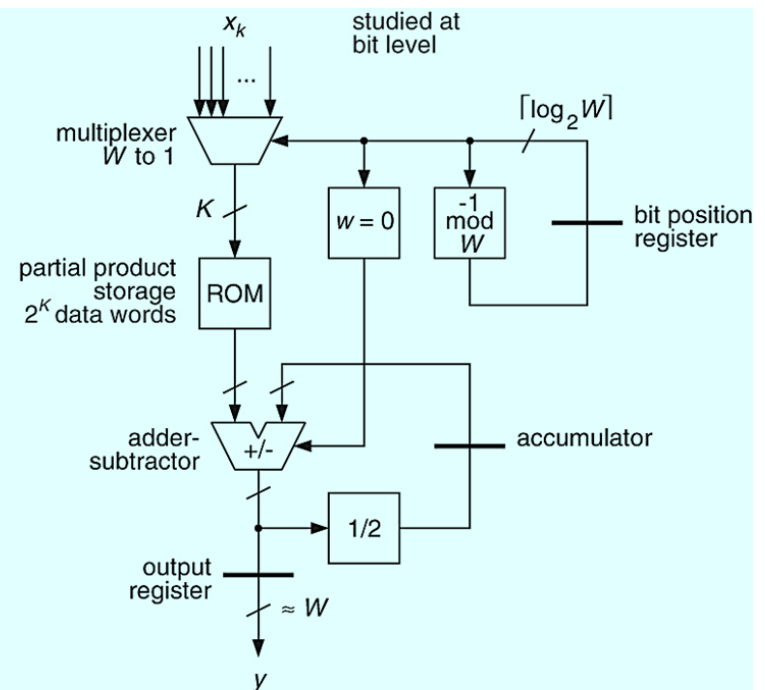
➔ DA implementation:

$$y = \sum_{k=0}^{K-1} c_k x_k$$

$$y = -p(0) + \sum_{w=1}^{W-1} p(w) 2^{-w}$$



(a) motto: "all bits from one word at a time"



(b) motto: "one bit from each word at a time"

Distributed Arithmetic - Summary

- ➡ Distributed arithmetic should be considered when
 - ➡ coefficients are fixed,
 - ➡ number (K) of distinct coefficient values is small (2^K), and
 - ➡ hardware multipliers are expensive compared to lookup tables.
- ➡ Pros and cons of distributed arithmetic
 - ➡ No need for costly multipliers as these get merged with coefficient tables.
 - ➡ Memory size grows exponentially with the order of the inner product to be computed. (Applying memory partition can relax the constraint.)
 - ➡ Mitigation techniques exist but depend heavily on coefficient values.
 - ➡ Example: DSP applications with table-based FPGAs



Extension to Other Algebraic Structures

- ⇒ **Observation:** Everything we have learned for architecture transforms is applicable to any algebraic field.
 - ⇒ All algebraic fields share a common set of axioms, so any algebraic transform that is valid in one field must necessarily hold for any other field. **Universal transforms remain valid anyway.**
- ⇒ **Infinite fields:** $(\mathbb{R}, +, \cdot)$ and $(\mathbb{C}, +, \cdot)$ are commonplace in digital signal processing.
- ⇒ **Finite fields:** $\text{GF}(2)$, $\text{GF}(p)$, $\text{GF}(p^n)$ have numerous applications in
 - ⇒ data compression (**source coding**), error correction (**channel coding**), and information security (**ciphering**).

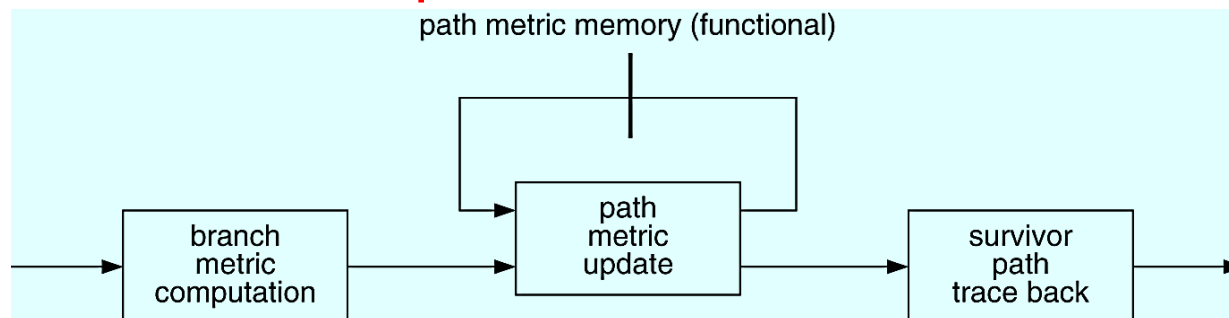


An Example – Viterbi Algorithm (1/2)

➡ The Viterbi algorithm:

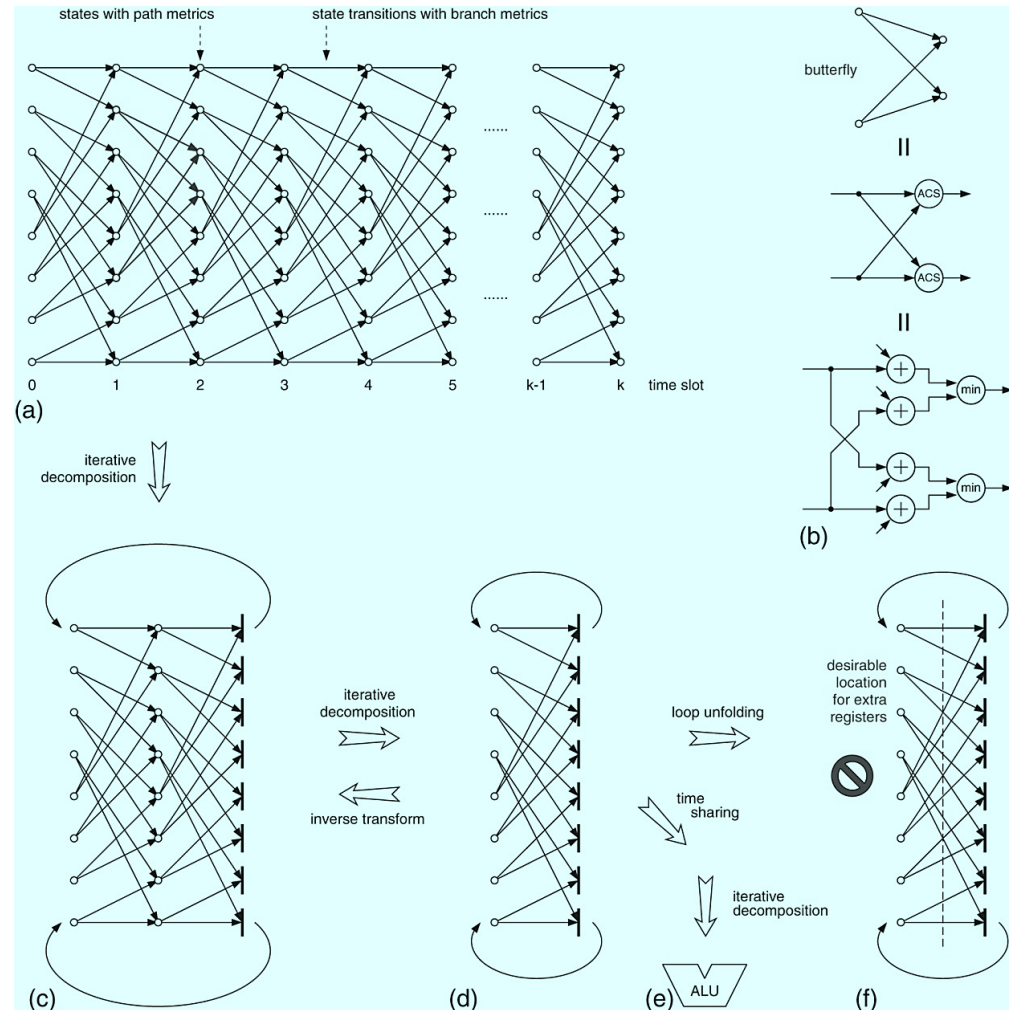
- ➡ Convolutional decoding is a multi-stage decision problem where Richard Bellman's principle of optimality applies.
- ➡ The globally optimum solution includes no suboptimal local decision.
- ➡ Bellman has developed a technique called “Dynamic Programming”, the Viterbi algorithm is a particular case thereof.

➡ Three main steps of VA



An Example – Viterbi Algorithm (2/2)

➔ Abstracted trellis-type DDG for path metric computation with details for one butterfly.



An Example – Viterbi Decoder (1/5)

➡ **Natural choice:** A datapath that computes one set of path metrics from the previous set in a single clock cycle (**architecture (d) in previous slice**)

➡ Combine iterative decomposition and time sharing, ultimately leads to a processor-type datapath built around an ALU.

➡ **Butterfly (add-compare-select) operation:**

$$y_1(k) = \min(a_{11}(k) + y_1(k-1), a_{12}(k) + y_2(k-1))$$

$$y_2(k) = \min(a_{21}(k) + y_1(k-1), a_{22}(k) + y_2(k-1))$$

➡ This is a **nonlinear** first-order recursion, and none of our architectural transforms applies.



An Example – Viterbi Decoder (2/5)

⇒ Boosting throughput of a Viterbi decoder

⇒ Consider a **semiring** where

- Set of elements: $S = \mathbb{R} \cup \{\infty\}$,
- Algebraic addition: $\boxplus = \min$, and
- Algebraic multiplication: $\boxdot = +$.

⇒ The reformulated add-compare-select operation

$$y_1(k) = a_{11}(k) \boxdot y_1(k-1) \boxplus a_{12}(k) \boxdot y_2(k-1)$$

$$y_2(k) = a_{21}(k) \boxdot y_1(k-1) \boxplus a_{22}(k) \boxdot y_2(k-1)$$

which, making use of vector and matrix notation, can be rewritten as

$$\vec{y}(k) = A(k) \boxdot \vec{y}(k-1)$$

⇒ This is a **linear** first-order recursion!



An Example – Viterbi Decoder (3/5)

⇒ Boosting throughput of a Viterbi decoder (*cont'd*)

- ⇒ By replacing $\bar{y}(k-1)$, one gets the unfolded recursion for $p = 2$

$$\bar{y}(k) = A(k) \boxdot A(k-1) \boxdot \bar{y}(k-2)$$

- ⇒ To take advantage of this unfolded form, the product

$$B(k) = A(k) \boxdot A(k-1)$$

must be computed outside the loop.

- ⇒ Resubstituting the original operators and variables we obtain the recursion

$$\begin{aligned} y_1(k) &= \min(b_{11}(k) + y_1(k-2), b_{12}(k) + y_2(k-2)) \\ y_2(k) &= \min(b_{21}(k) + y_1(k-2), b_{22}(k) + y_2(k-2)) \end{aligned}$$

which includes the same number and types of operations as the original formulation in **twice as much time**.

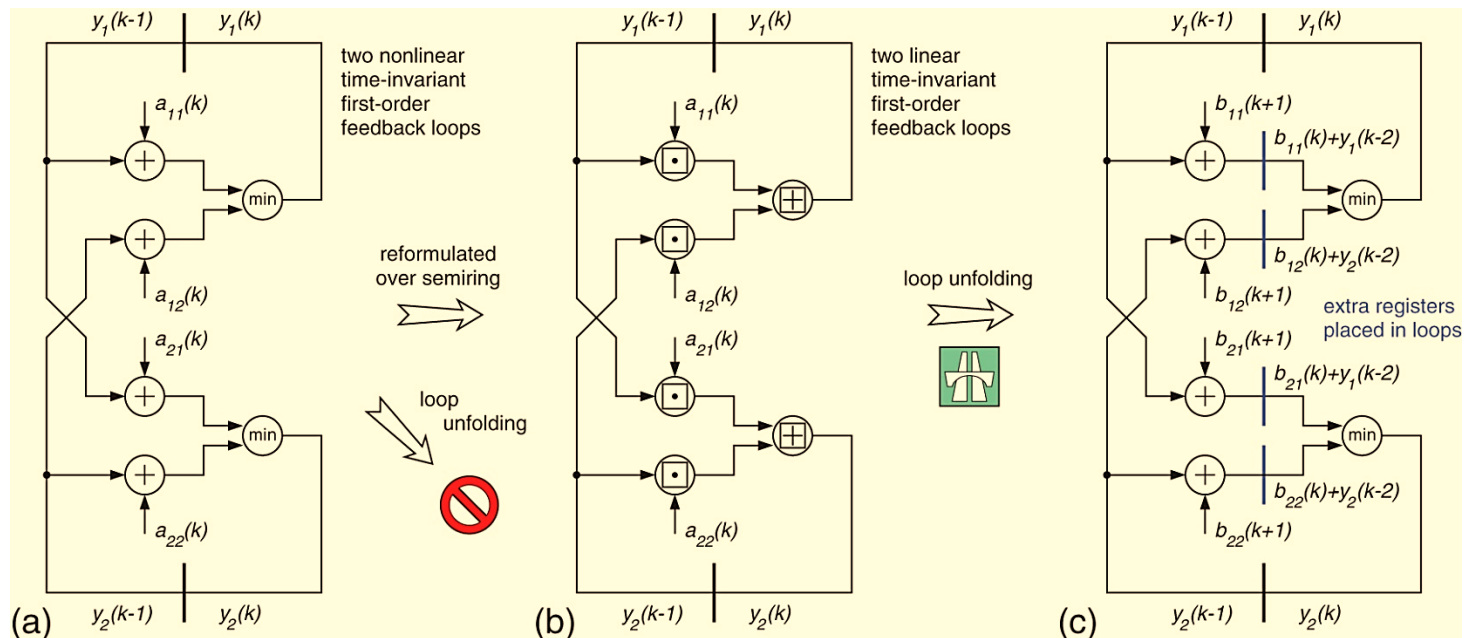


An Example – Viterbi Decoder (4/5)

➔ Boosting throughput of a Viterbi decoder (*cont'd*)

$$y_1(k) = \min(b_{11}(k) + y_1(k-2), b_{12}(k) + y_2(k-2))$$

$$y_2(k) = \min(b_{21}(k) + y_1(k-2), b_{22}(k) + y_2(k-2))$$



An Example – Viterbi Decoder (5/5)

⇒ Boosting throughput of a Viterbi decoder (*cont'd*)

- ➡ The price to pay is the extra hardware required to perform the **non-recursive computations outside the loop**

$$b_{11}(k) = \min(a_{11}(k) + a_{11}(k-1), a_{12}(k) + a_{21}(k-1))$$

$$b_{12}(k) = \min(a_{11}(k) + a_{12}(k-1), a_{12}(k) + a_{22}(k-1))$$

$$b_{21}(k) = \min(a_{21}(k) + a_{11}(k-1), a_{22}(k) + a_{21}(k-1))$$

$$b_{22}(k) = \min(a_{21}(k) + a_{12}(k-1), a_{22}(k) + a_{22}(k-1))$$

in a **heavily pipelined** way.

- ➡ Taking advantage of specific properties of an algorithm and of algebraic transforms has more potential to offer than universal transforms alone.

⇒ Compare the two formulations of the same problem:

- ➡ Nonlinear recursion over field → not amenable to loop unfolding
- ➡ Linear recursion over semiring → amenable to loop unfolding



Summary and Conclusions (1/6)

➡ Options available for reorganizing datapath architectures

		combinational (memoryless)	Type of computation sequential (memorizing)	
			non-recursive	recursive
	Data flow	feedforward	feedforward	feedback
	Memory	no	yes	yes
	Data dependency graph	DAG with all edge weights zero	DAG with some or all edge weights non-zero	Directed cyclic graph with no circular path of weight zero
	Response length	$M = 1$	$1 < M < \infty$	$M = \infty$
Nature of system	linear time-invariant	D,P,Q,S,a	D,P,q,S,a,R	D,S,a,R,i,U
	linear time-variant	D,P,Q,S,a	D,P,S,a,R	D,S,a,R,i,U
	nonlinear	D,P,Q,S,a	D,P,S,a,R	D,S,a,R,i,u

D: Iterative decomposition

P: Pipelining

Q: Replication

q: Multipath filtering as a special case of rep.

a: Associativity transform provided operations are identical and associative

R: Retiming

i: Pipeline interleaving

U: Loop unfolding

S: Time sharing

u: Loop unfolding provided computation is linear over a semiring



Summary and Conclusions (2/6)

➡ Some important architectural transforms and their characteristics

Architectural transform	Decomposition	Pipelining	Replication	Time sharing	Associativity	Retiming	Loop unfolding
Kind	universal	universal	universal	universal	algebraic	universal	algebraic
Applicable to	combinational computations					sequential computations	
Impact on						nonrecurs.	recursive
A	$-... =$	$= ... +$	$+$	$-... =$	$=$	$=$	$+$
Γ	$+$	$=$	$-$	$+$	$=$	$=$	$=$
t_{lp}	$-$	$-$	$=, \text{mux } -$	$=$	$-... +$	$-$	$-$
$T = \Gamma \cdot t_{lp}$	$=$	$-$	$-$	$+$	$-... +$	$-$	$-$
AT	$-... =$	$-... =$	$=$	$= ... +$	$-... +$	$-$	$+$
L	$+$	$+$	$=, \text{mux } +$	$+$	$=$	$=$	$+$
E	$-... +$	$-... +$	$=$	$= ... +$	$-... +$	$=$	$+$
Extra hardware overhead	recy. and cntl.	none	distrib., recoll., and cntl.	collect., redist., and cntl.	none	none	extra word width
Helpful for indirect energy saving	no	coarse grain yes	possibly yes	no	yes	yes	possibly yes
Compatible storage type	any	register	register	any	any	register	register



Summary and Conclusions (3/6)

- ➡ **Power and energy considerations:** What is meant by “Helpful for indirect energy saving”?
 - ➡ In CMOS, the most effective way to cut the energy spent per operation is to **lower the supply voltage**.
 - ➡ The long paths through a circuit are likely to become unacceptably slow and need to be **trimmed to recover clock rate** and throughput.
 - ➡ Architectural transforms that help do so with no circuit overhead.
 - ➡ Retiming
 - ➡ Chain/tree conversion (**and other algebraic transforms**)
 - ➡ Coarse grain pipelining (**small overhead only**)
 - ➡ Effectiveness must be examined in detail on a per case basis.



Summary and Conclusions (4/6)

➡ Architecture design in an energy-constrained world -

A key challenge of architecture design is to

- ➡ minimize redundant switching activities,
- ➡ provide just as much flexibility as required,
- ➡ keep the effort for design and verification within reasonable bounds,
- ➡ Finding clever combinations between **hardwired units** and **program-controlled processors**.

➡ The impressive throughputs of **general-purpose processors** have been bought by operating them under conditions such as

- ➡ fine-grain pipelining, extremely fast clock, comparatively high supply voltage, low MOSFET threshold and hence significant leakage.



Summary and Conclusions (5/6)

- ⇒ A guide to evaluating architectural alternatives
 - ⇒ Begin by analyzing the algorithm. Give quantitative indications for the data rates between all major building blocks, the word widths, the memory bounds and access schemes for all building blocks, and the computation rates for all major arithmetic operations.
 - ⇒ Look for simplifications and optimizations in the algorithmic domain.
 - ⇒ Examine the control flow. Find out where to go for a hard-wired dedicated architecture, where for a program-controlled processor, and where to look for a compromise.
 - ⇒ Let your intuition come up with preliminary architectural concepts. Establish a rough block diagram for each. Have boundaries between major subfunctions coincide with registers.



Summary and Conclusions (6/6)

- ➡ A guide to evaluating architectural alternatives (*cont'd*)
 - ➡ Prepare a spreadsheet that opposes all architectures considered.
 - ➡ Estimate the overall circuit size, computation period, latency, and dissipated energy for each architecture. **Synthesize, place and route time-critical portions** as propagation delays often depend on lower-level details.
 - ➡ Identify bottlenecks and unacceptably burdensome subfunctions. **Improve architecture with the aid of equivalence transforms.**
 - ➡ Compare, then narrow down your choice.

Architecture design is more art than science.



Exercises

- ➡ **Problem 2:** Discuss the idea of combining replication with pipelining.
- ➡ **Problem 3:** Reconsider the third-order correlator of Fig. 3.32.
- ➡ **Problem 5:** Show different ways of implementing the arithmetic mean and standard deviation.
- ➡ **Problem 8:** Calculate the convolution of a two-dimensional array with a fixed two-dimensional operator.



Appendix

Memory Design



Types of Memories (1)

⇒ Volatile

⇒ Random Access Memory (RAM)

- ⇒ SRAM – Static RAM
- ⇒ DRAM – Dynamic RAM

⇒ Non-volatile

⇒ Read Only Memory (ROM)

- ⇒ Mask ROM: mask programmable
- ⇒ EPROM: electrically programmable
- ⇒ EEPROM: electrically erasable and programmable
- ⇒ FLASH memory - similar to EEPROM

⇒ Single-port vs. multi-port



Types of Memories (2)

⇒ Static

- ⇒ Information is stored in latches
- ⇒ Remains valid as long as power is applied
- ⇒ Short read/write cycle

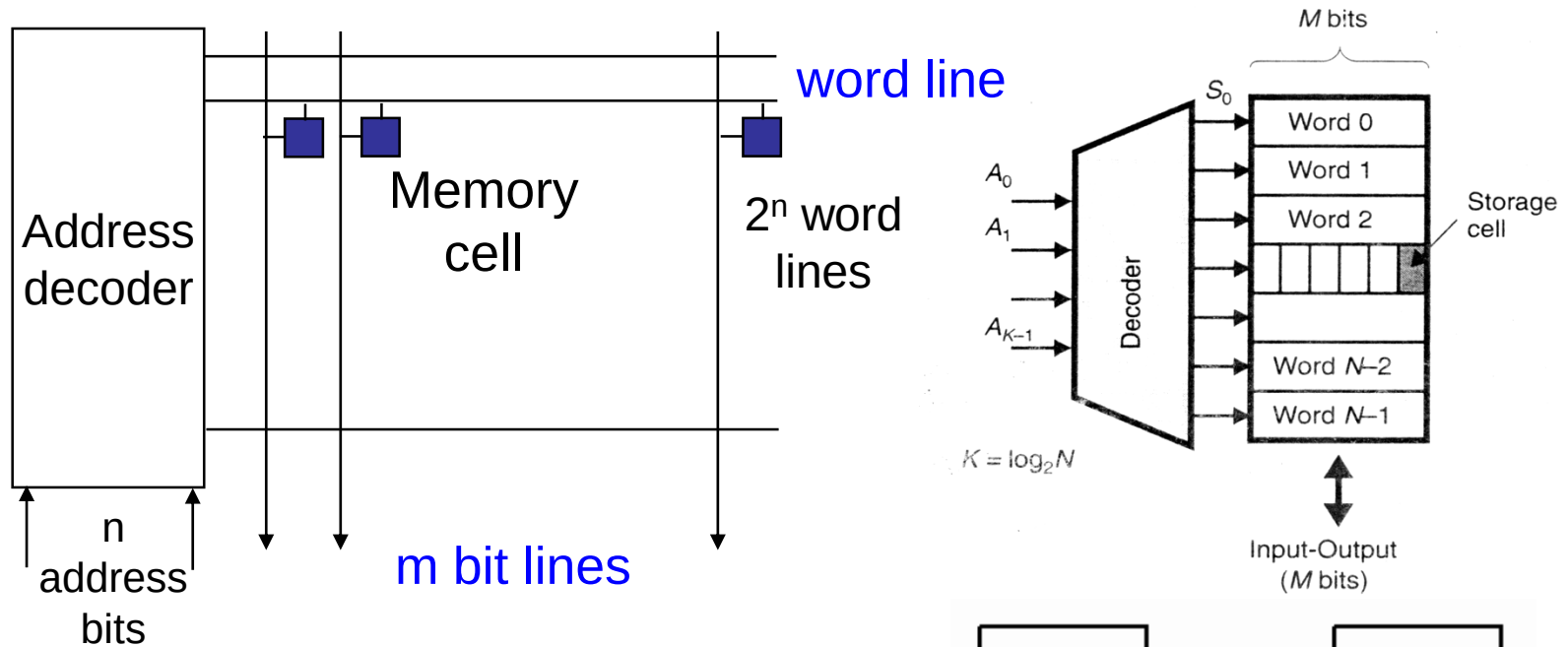
⇒ Dynamic

- ⇒ Information is stored in the form of charges on capacitors
- ⇒ Stored charges tend to discharge with time
- ⇒ Need to be refreshed (read and write back)
- ⇒ Reduced power consumption
- ⇒ Larger memory capacity

The type of memory unit that is preferable for a given application is a function of the required memory size, the time it takes to access the stored data, the access pattern, and the system requirement.



Definitions and Notations

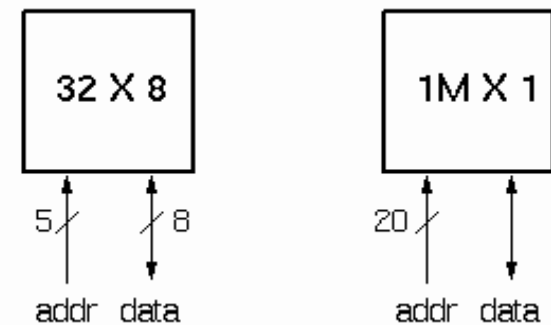


RAM/ROM naming convention

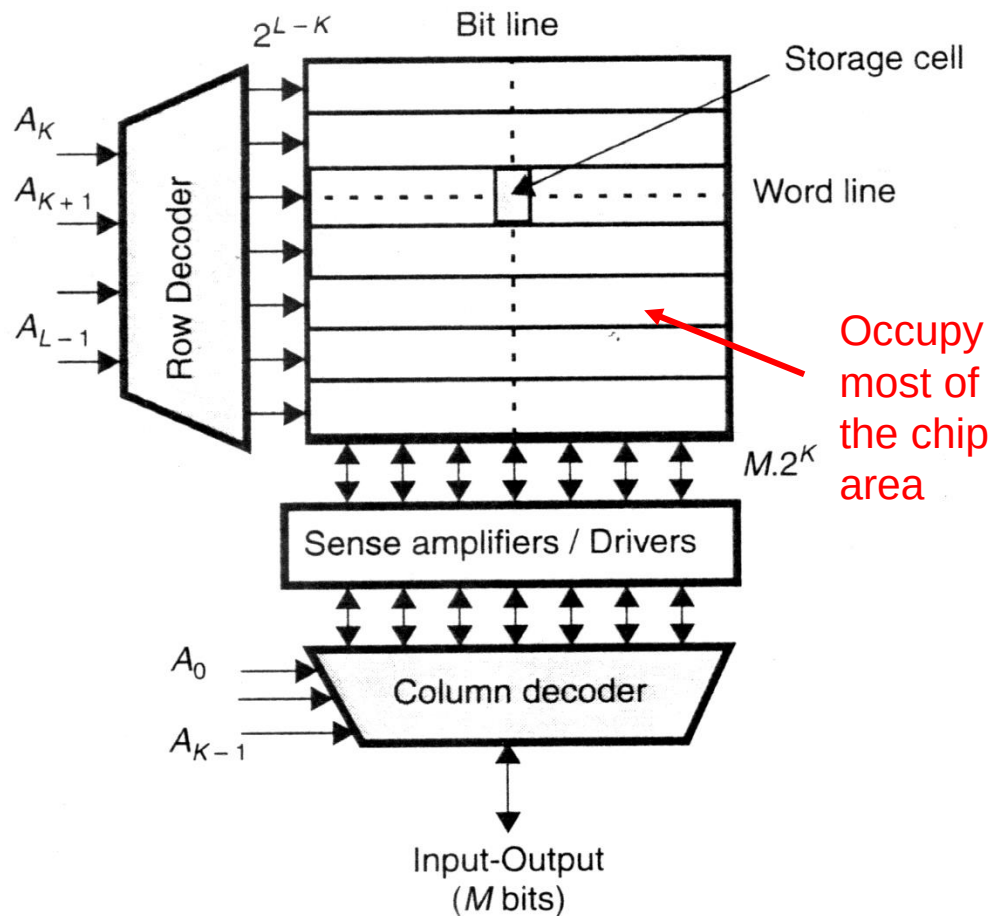
32×8 , "32 by 8" $\rightarrow$ 32 8-bit words

$1M \times 1$, "1 MEG by 1" $\rightarrow$ 1M 1-bit words

What happens if n and/or m is very large?



Memory Organization



Column/row addresses

- The memory arrays are organized so that the vertical and horizontal dimensions are of the same order of magnitude (aspect ratio ~ 1)
- word lines and bit lines

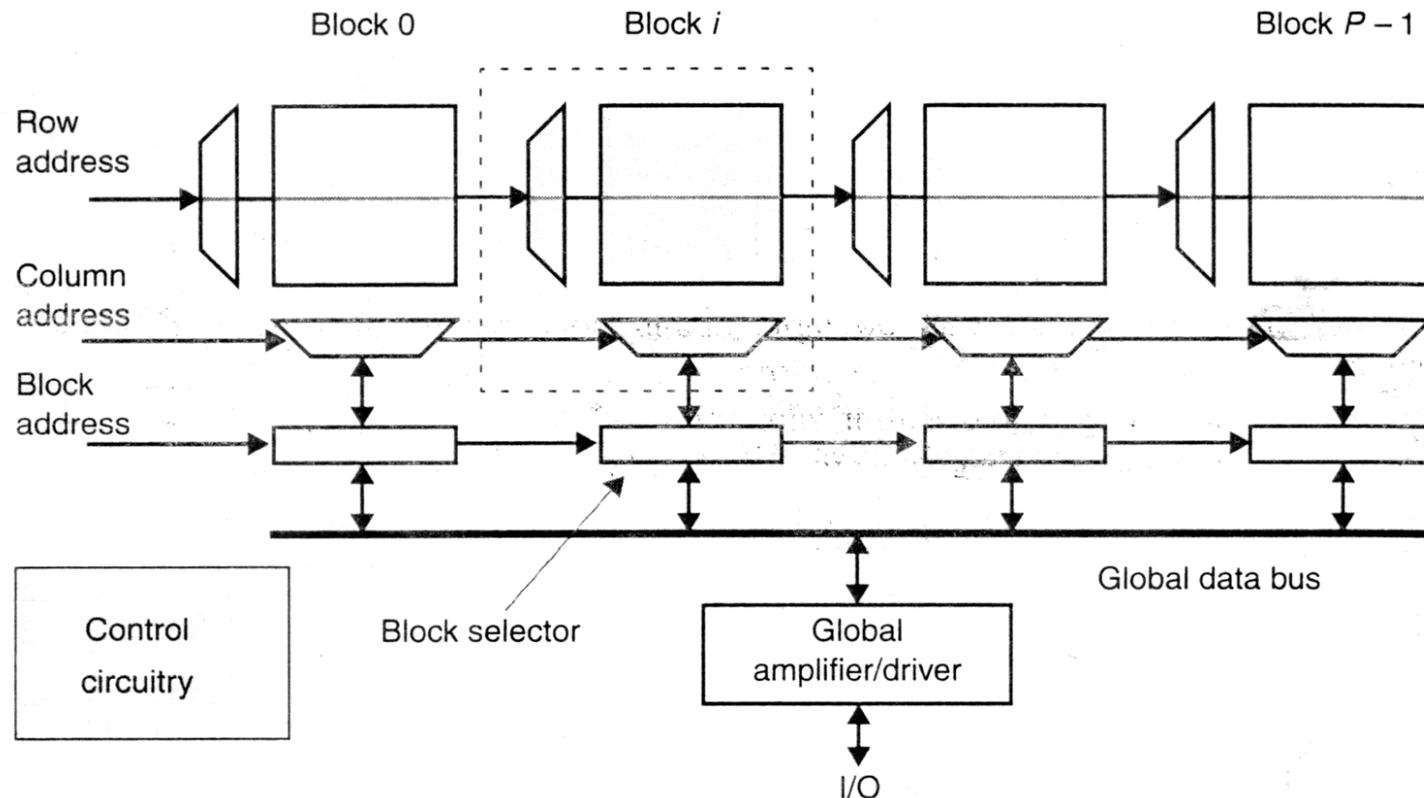
Sense amplifiers

- It is used to reduce the voltage swing on the bit line to a value substantially below the supply voltage, thus reducing both the propagation delay and the power consumption



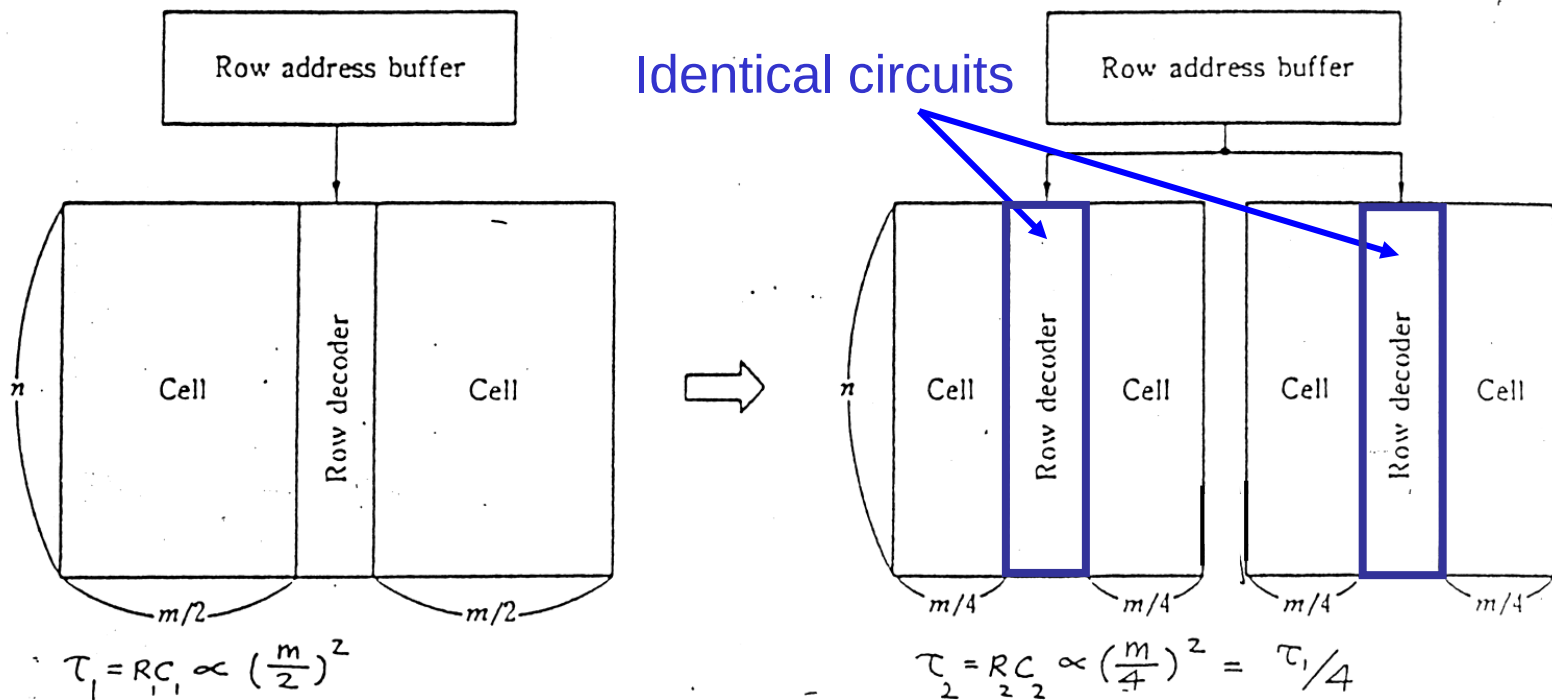
Hierarchical Memory Arch. (1/2)

- ➔ The block selector enables a single memory block at a time



Hierarchical Memory Arch. (2/2)

- ➔ Memory array organization to improve the word line delay



SRAM



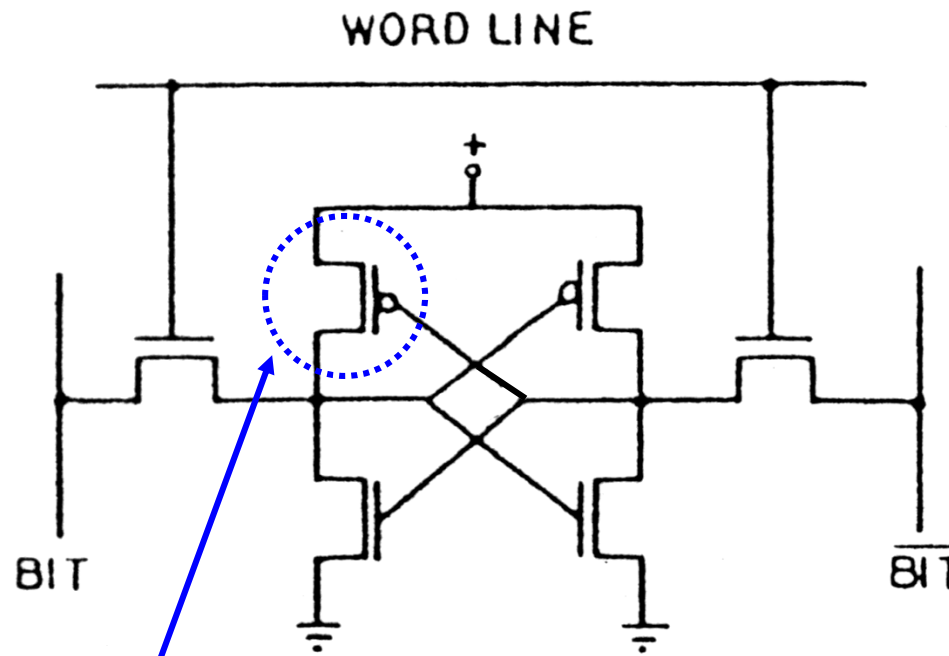
Static RAM (SRAM)

- ➡ Bits stored as on/off switches
- ➡ Cache memory
- ➡ Advantages (c.p. DRAM)
 - ➡ faster
 - ➡ no charges to leak (more reliable)
 - ➡ no refreshing needed when powered
- ➡ Disadvantages (c.p. DRAM)
 - ➡ larger per bit
 - ➡ more expensive
 - ➡ more power dissipation



SRAM Cell

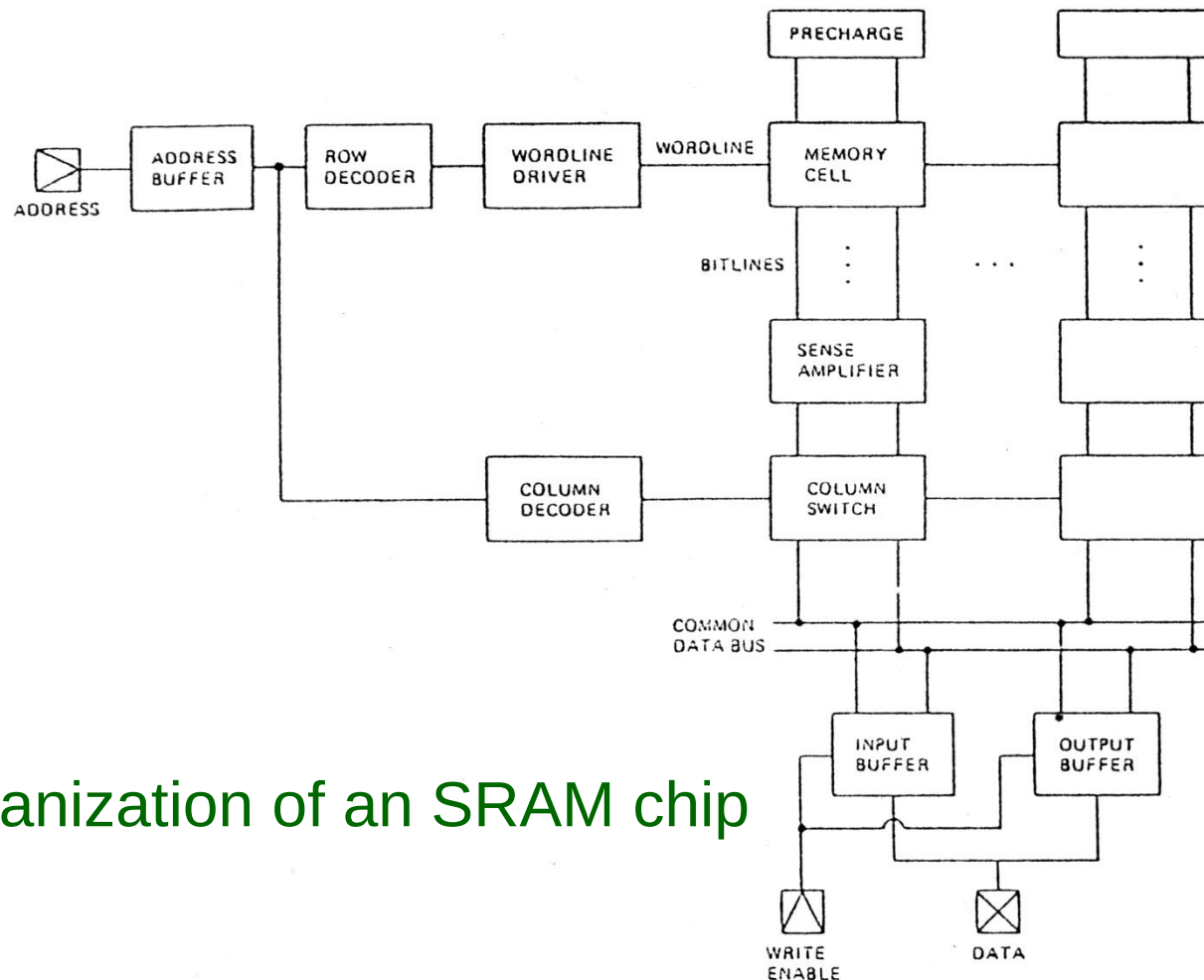
- ➡ SRAMs use a static latch (cross coupled bistable element) as a memory cell.



- * The pull-up device (load) could be PMOS's, enhanced NMOS's, and polysilicon resistors.



SRAM Organization (1)



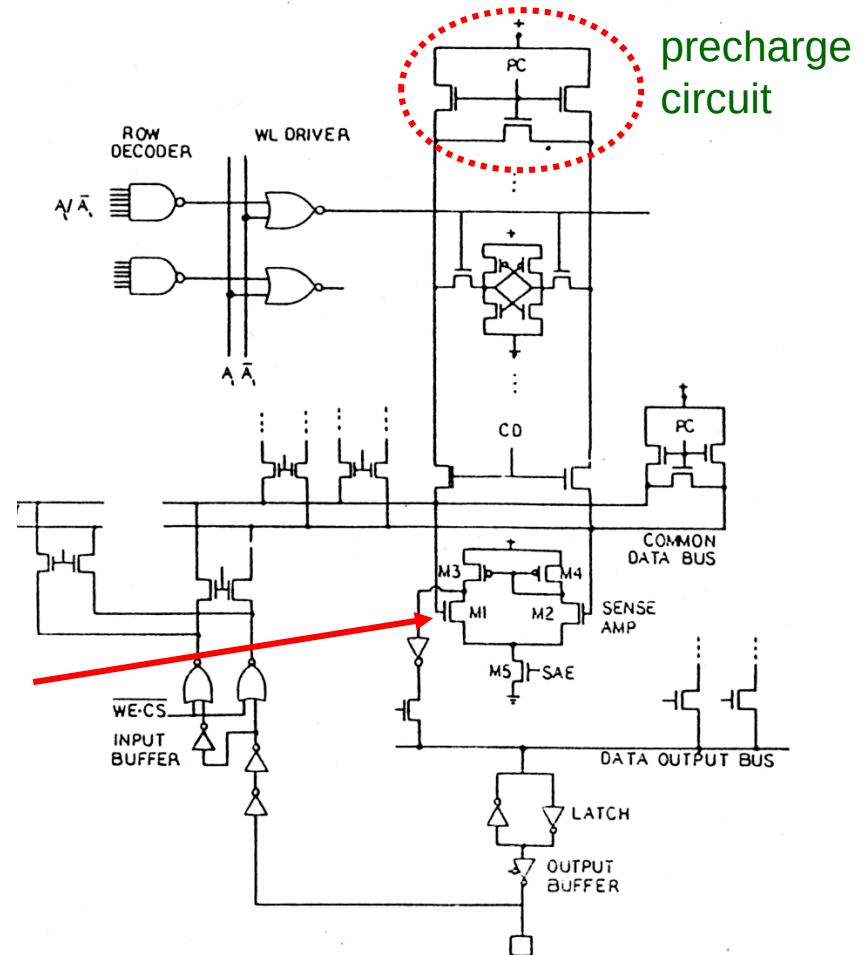
* Organization of an SRAM chip



SRAM Organization (2)

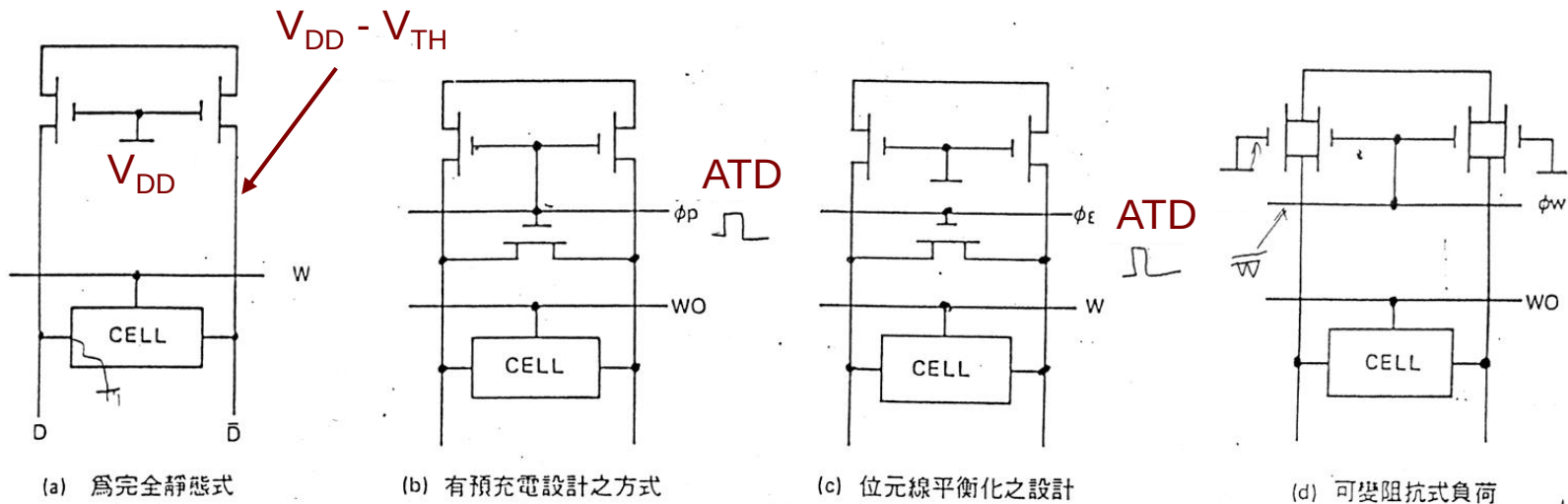
- ➔ Read/write path of an SRAM chip

* A differential pair of nMOS transistors couples to bit lines via their gate capacitances to sense a small potential difference between bit lines.



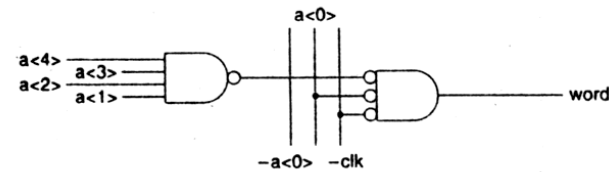
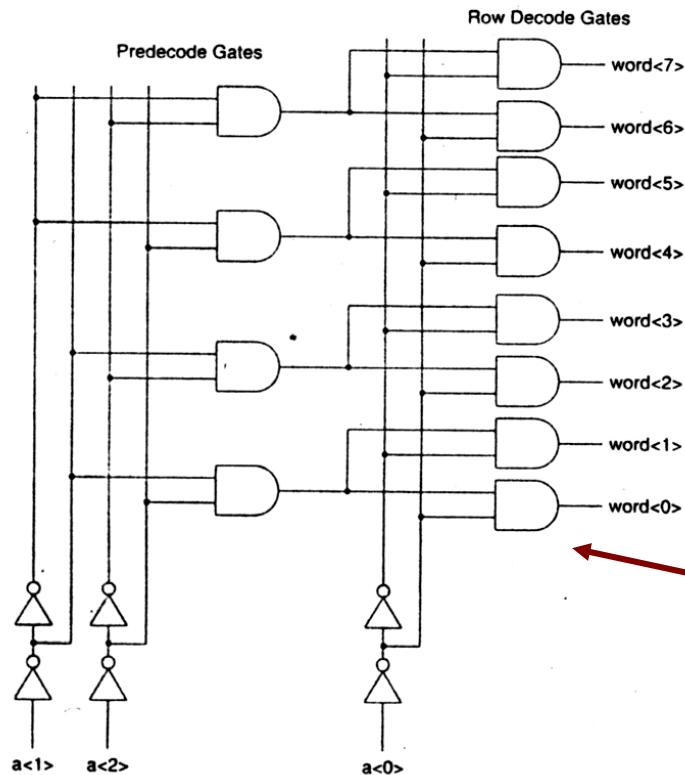
Precharge Schemes

- Sometimes, bit lines are precharged to a high potential and equalized.
- ➡ The chip then waits for a transition in any one of its address, write enable (WE) and chip select (CS) pins

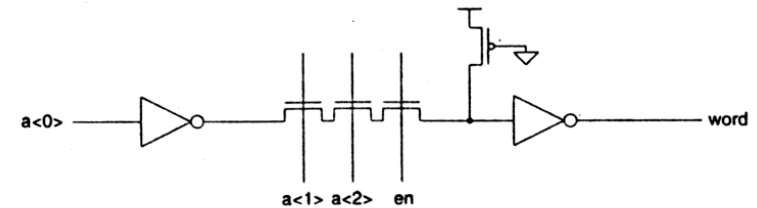


Decoders

- Column decoders and row decoders
- Predecode concept



(b)



(c)

- (a) Basic approach;
- (b) actual implementation;
- (c) pseudo-nMOS example



Sense Amplifier (1/2)

- ➔ To minimize the delays at the high-capacitance bit lines

➔ An example

Detect a potential swing equal to a small fraction of the full power-supply range.

* Pitch: the repeated distance

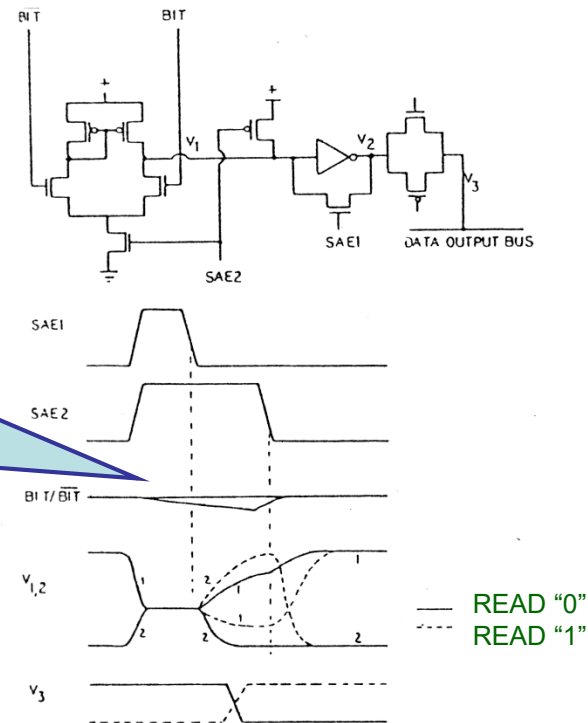


FIGURE 4.11 Differential amplifier with a second stage precharged to $V_{DD}/2$. The feedback loop around the inverter precharges the circuit to its high-gain region when enabled by SAE1. The dashed line represents a "1" being read, and the solid line a "0" being read.



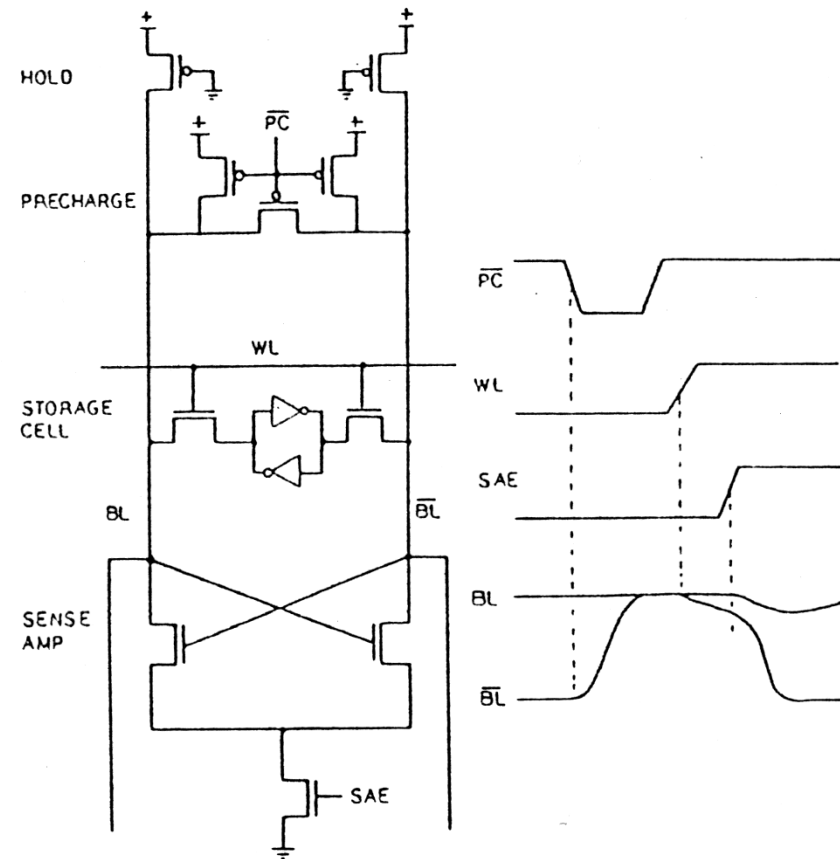
Sense Amplifier (2/2)

➡ Column selectors placed after the SA

- ➡ More amplifiers are needed.
- ➡ Increase power and area.
- ➡ Match the pitch of the SA and bit line.

➡ Column selectors placed before the SA

- ➡ Inputs to amplifiers become slower.
- ➡ Increase access time.
- ➡ Better SA can be used.



Sense amplifier for a differential data bus



DRAM



Dynamic RAM (DRAM) (1/2)

➡ Pros and Cons

- ➡ high density, low power, cheap, and slow in comparison with the SRAM
- ➡ needs to be refreshed periodically
- ➡ more prone to soft errors caused by α particles

➡ Column and Row Addresses

- ➡ Addresses divided into 2 halves (memory as a 2D matrix) to reduce pin counts for packaging
 - ➡ *RAS* or *Row Access Strobe*
 - ➡ *CAS* or *Column Access Strobe*



Dynamic RAM (DRAM) (2/2)

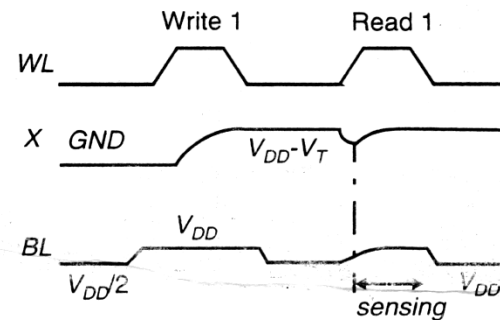
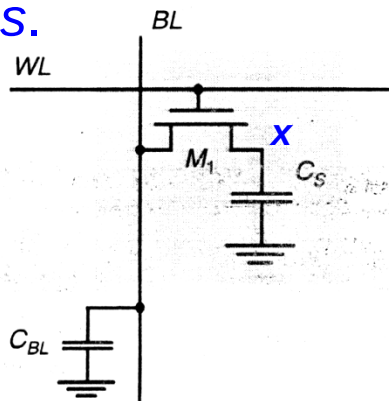
- ⇒ The storage capacity of SRAMs is typically **four** times smaller than that of DRAMs at the same time period.
- ⇒ Charge sharing between the capacitance for storing data and the bit line capacitance
 - ⇒ Destructive read $\Rightarrow$ restore data after read $\Rightarrow$ refresh cycle $\sim ms$
- ⇒ Basic cell classification
 - ⇒ The dynamic memory cell consists of 4-transistor, 3-transistor, or 1-transistor design.



DRAM Cell (1/2)

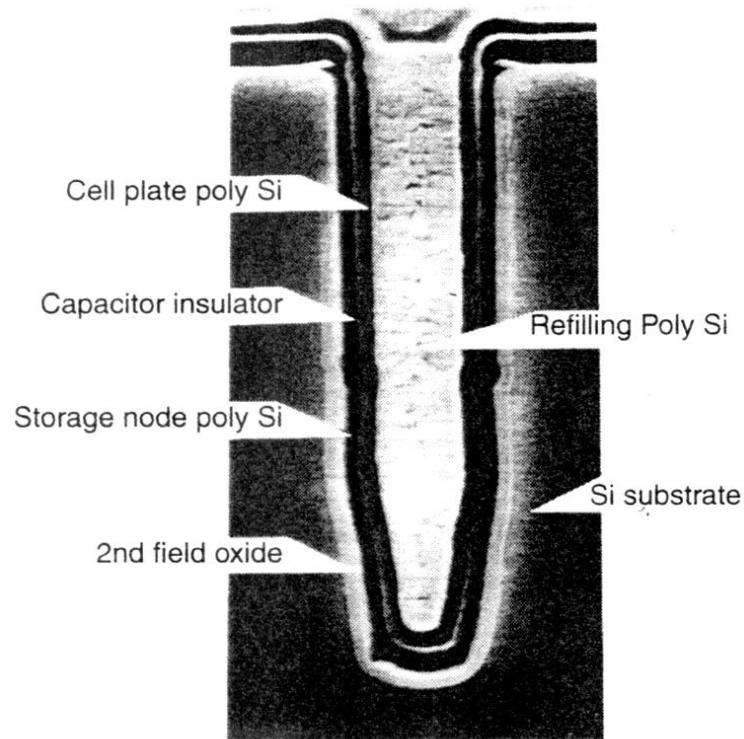
➔ One-transistor dynamic RAM cells

- ➔ It consists of one transistor and one capacitor (typical value 30 ~ 100 fF).
- ➔ The cell has one read-write control line (word line) and one I/O line (bit line).
- ➔ Charge sharing between the large C on the bit line and small C for data storage $\Rightarrow$ destructive read
- ➔ Due to leakage current, the data might only be hold for up to 1 ms.

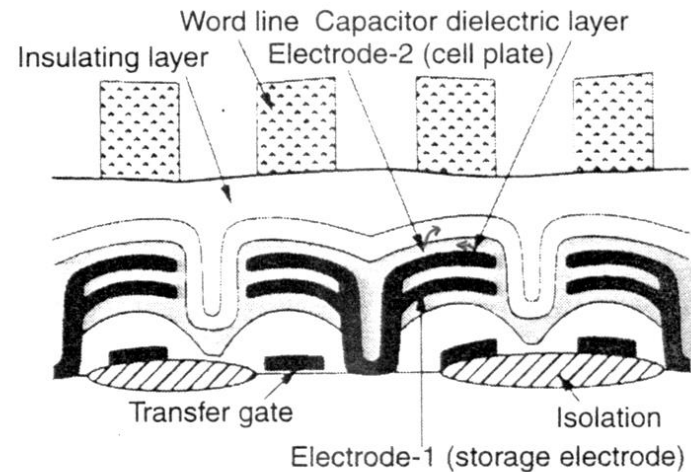


DRAM Cell (2/2)

Advanced 1T1R DRAM memory cells



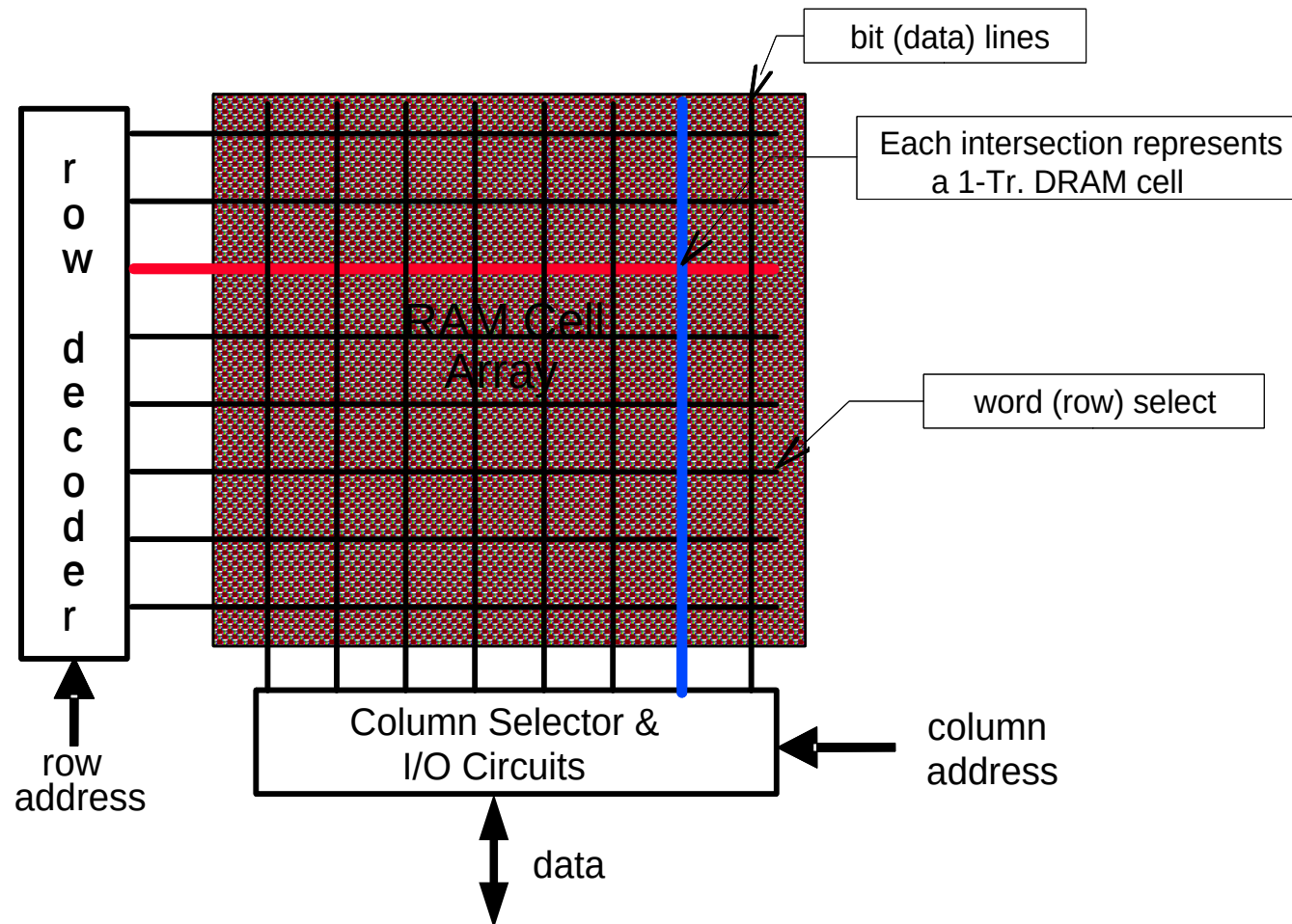
(a) Trench-capacitor cell (from [Mano87])



(b) Stacked-capacitor cell (from [Taguchi91])



Classical DRAM Organization (1)



Classical DRAM Organization (2)

⇒ Precharge level and dummy cell (reference cell)

- ⇒ Depending on the bit line precharge level, dummy cells may not be necessary, e.g., the bit lines are charged to $V_{DD}/2$.
- ⇒ **Dummy cells** are implemented to minimize noise by keeping the circuit symmetry, e.g. the noise coupling between the bit lines from voltage variation.
- ⇒ DRAM SA has to fit the bit line pitch because a SA is required for each column in DRAMs.
 - ⇒ Every cell selected by the word line needs its data to be refreshed by a SA at the end of a read cycle.
- ⇒ Due to leakage current, data might be hold for up to 1 *ms*.



R/W 1-T Memory Cell

⇒ Write

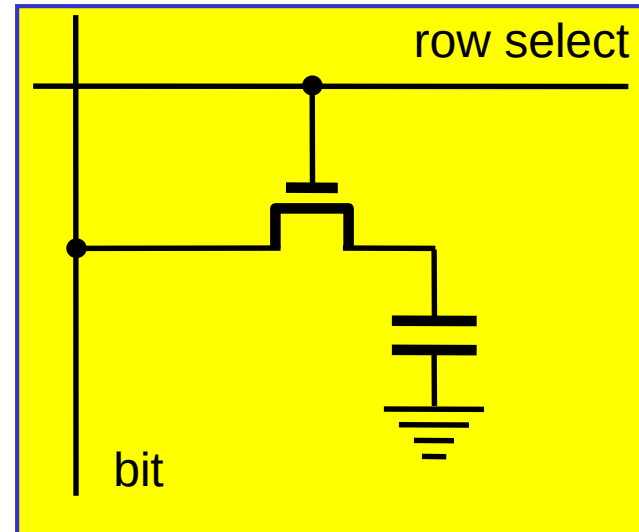
1. Drive bit line
2. Select row

⇒ Read

1. Precharge bit line to $V_{DD}/2$
2. Select row
3. Cell and bit line share charges
 - ⇒ Very small voltage changes on the bit line
4. Sense the voltage variation (sense amp)
 - ⇒ Can detect changes of $\sim 10^6$ electrons
5. Restore (write) the value

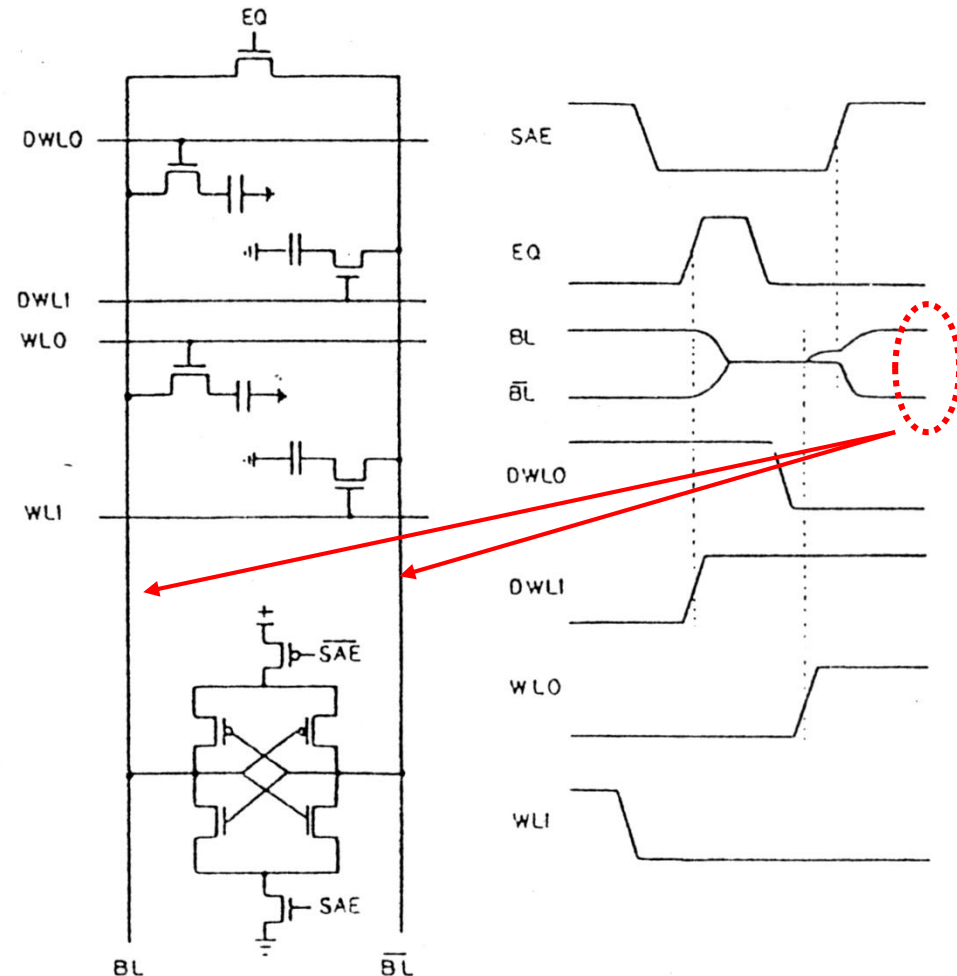
⇒ Refresh

- ⇒ Just do a dummy read to every cell



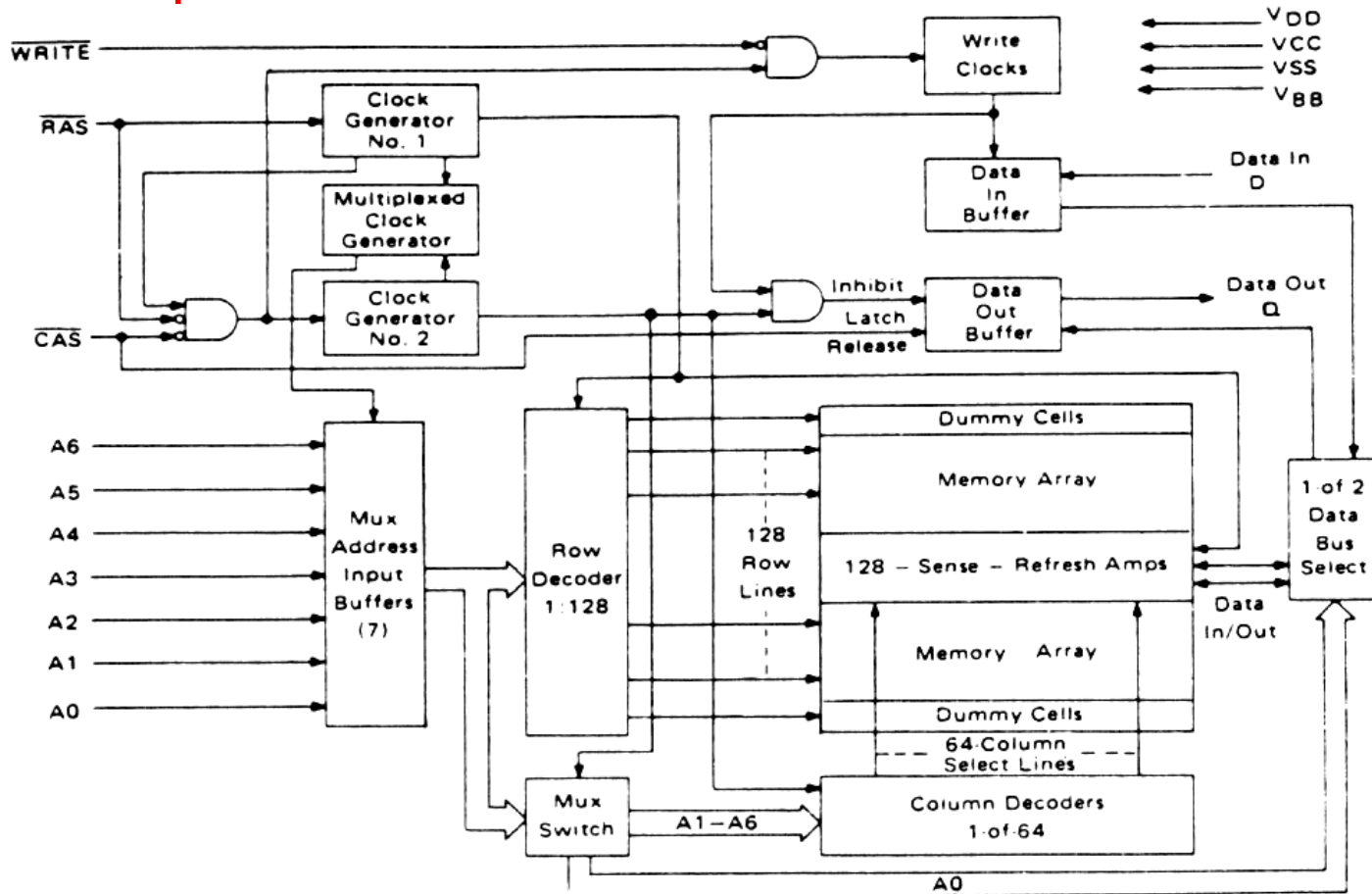
Midpoint Sensing

- DRAM with $V_{DD}/2$ sensing scheme
 - A full-size dummy cell is precharged to $V_{DD}/2$ and compared with a V_{DD} or V_{SS} stored in a full-size memory cell.
 - It is faster because building sense amplifiers with a large gain at the midpoint between the power level is faster.



DRAM Architecture - Example

➡ An example: 16K DRAM



ROM



Introduction

➡ Use memory devices that will retain information even when

- ➡ the power is temporarily interrupted;
- ➡ the device is left without applied power.

➡ **Examples:**

- ➡ Magnetic, optical media
- ➡ Nonvolatile memories (NVMs)
 - ➡ Low cost per bit, high density, fast random access,
 - ➡ Low power consumption,
 - ➡ Operation over a wide temperature range, and
 - ➡ A single low-voltage power supply operation.



ROM Families (1/2)

⇒ Masked ROMs

- ⇒ Data are permanently written during the manufacturing.
- ⇒ Cannot be altered by users.
- ⇒ Can be used to store a microprocessor operating-system program, look-up tables for mathematical calculation or character generate, etc.

⇒ PROMs

- ⇒ Manufactured in bipolar or MOS technology.
- ⇒ Can be programmed by end users with a special apparatus.
 - ⇒ one-time programmable (OTP) or write-once memory (WOM).
- ⇒ A data path exists between every word and bit line through a “fuse”.
- ⇒ Storage cells are selectively altered by electrically blowing the “fuse”.



ROM Families (2/2)

⇒ EPROMs

- ➔ Can be electrically programmed in a system by using a special 12V programming pin.
- ➔ Charge is stored on the floating polysilicon gate and can be erased from the system by exposing to the ultraviolet light.

⇒ EEPROMs

- ➔ Can be electrically erased.
- ➔ Data are written or erased one byte at a time, whereas flash memory allows data to be written or erased in block.

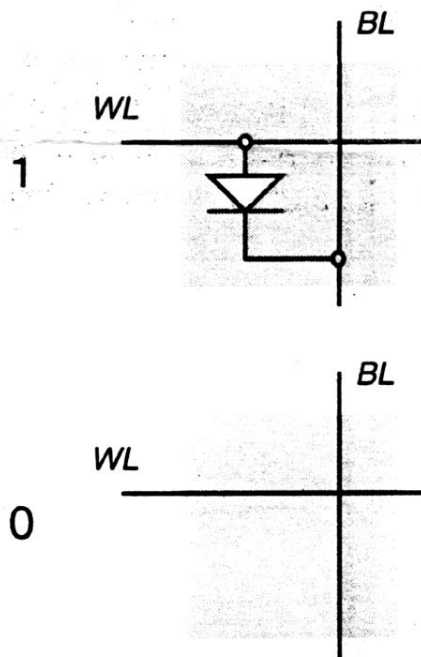
⇒ Flash Memories

- ➔ Contents of all memory array cells can be erased simultaneously through the use of an electrical erase signal.
- ➔ Based on either the EPROM or EEPROM technology.

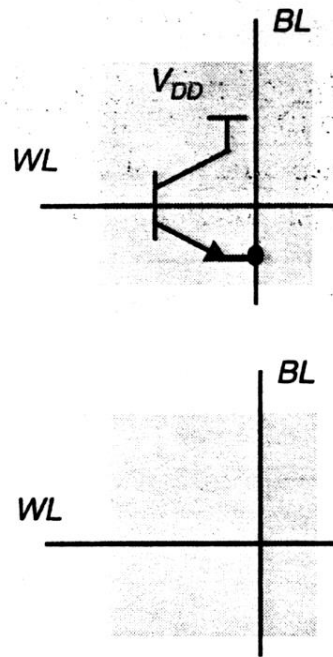


ROM Cells

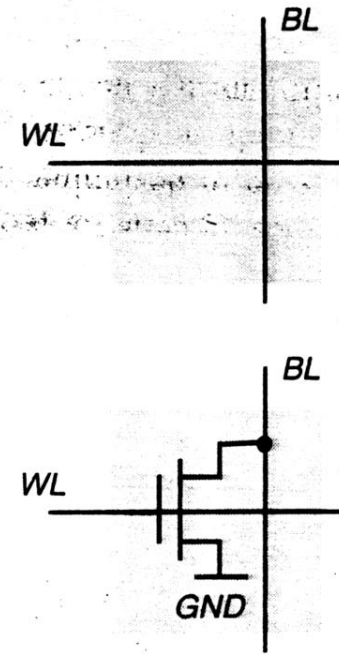
➔ Basic ROM cells in different technologies



(a) Diode ROM



(b) Bipolar ROM

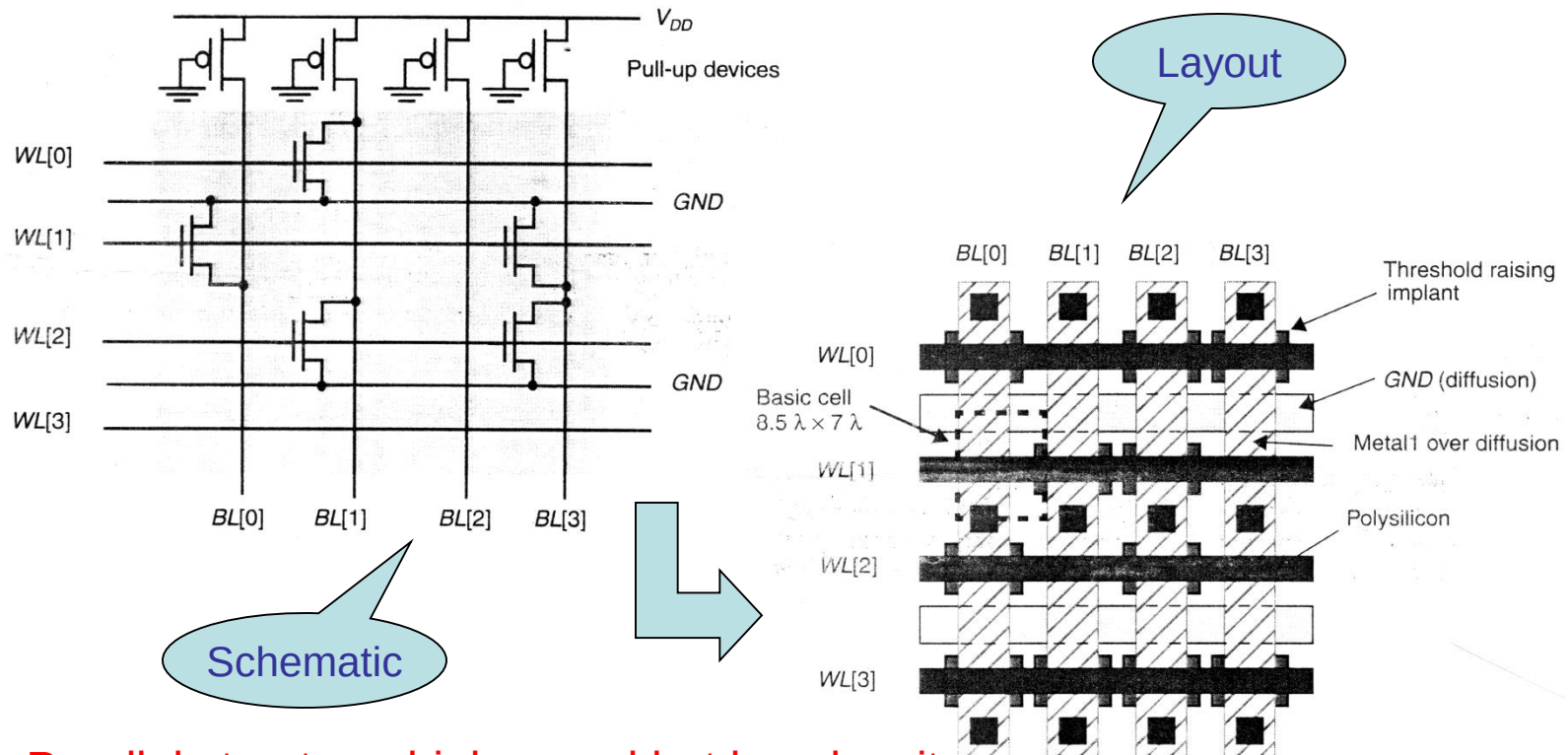


(c) MOS ROM



NOR ROM Structures (1/2)

- ➔ A 4×4 MOS NOR ROM using threshold-raising implants to disable transistors

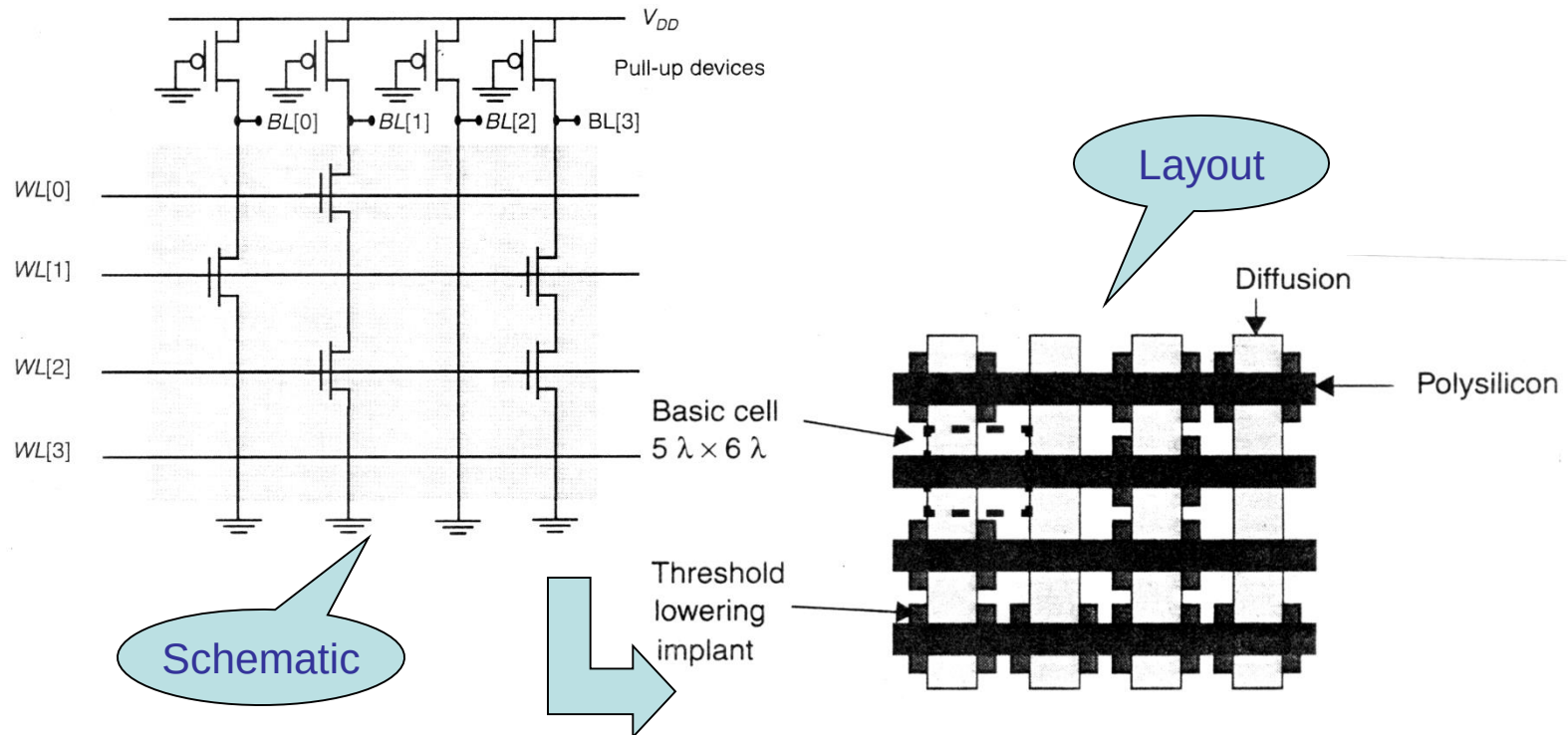


Parallel structure: high speed but low density



NAND ROM Structures (2/2)

- A 4×4 MOS NAND ROM using threshold-lowering implants for programming

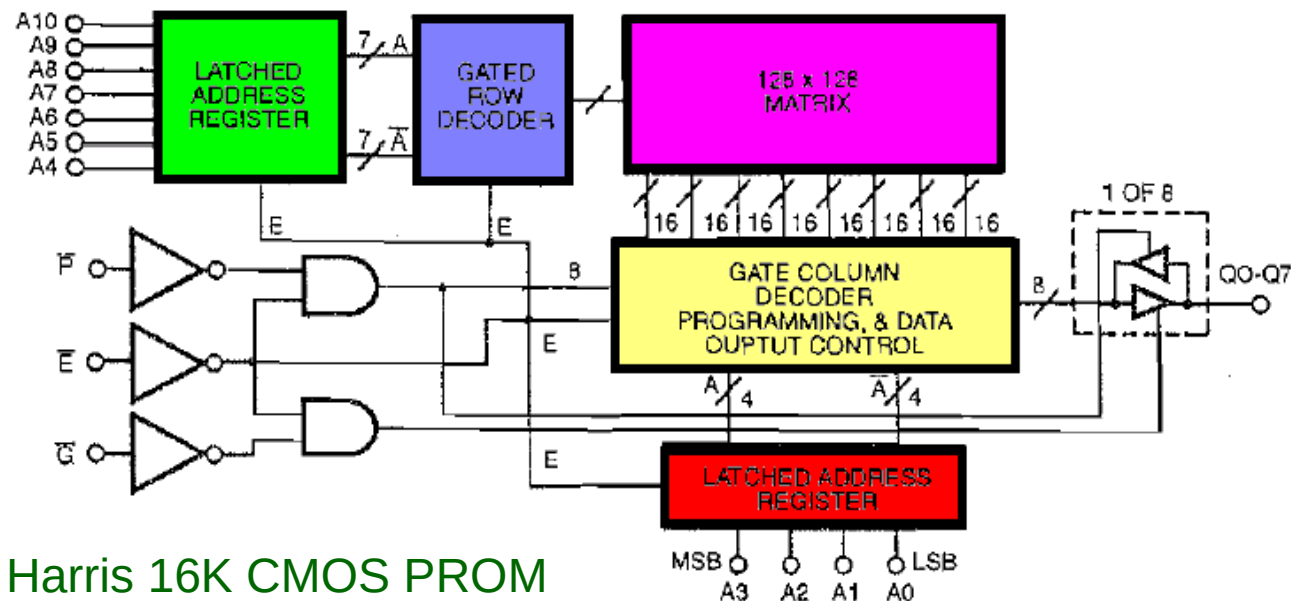


Serial structure: low speed but high density



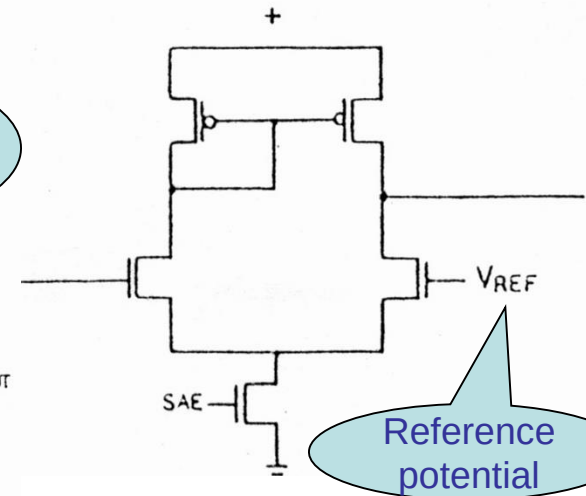
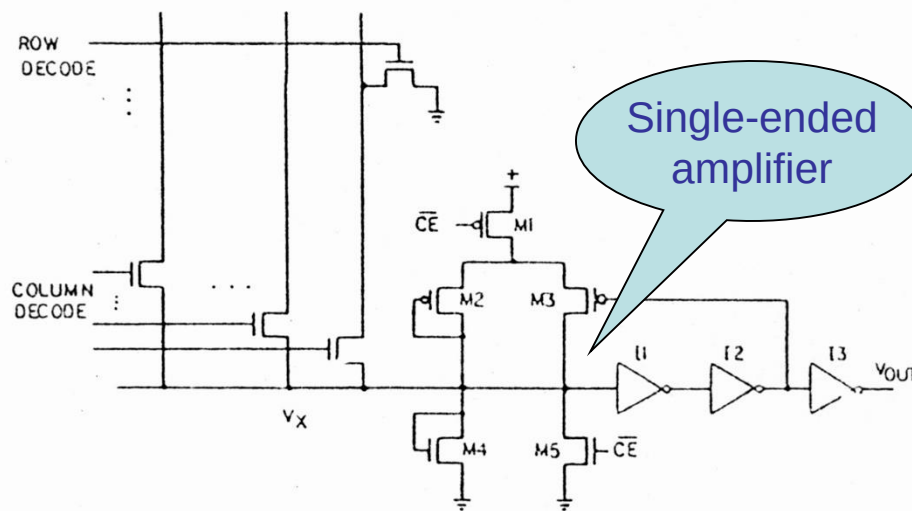
ROM Circuit Structures

- ➔ A $2^n \times m$ ROM consists of an addressable array of semiconductor memory cells organized as 2^n words of m bits each.
 - ➔ n input called “address line”.
 - ➔ m outputs “bit line”.



Sense Amplifiers

- ROMs usually do not have double-rail signals available from their memory cell.
 - Single-ended amplifier is employed.
 - Differential amplifier is applied when the other input is tied to a reference potential.



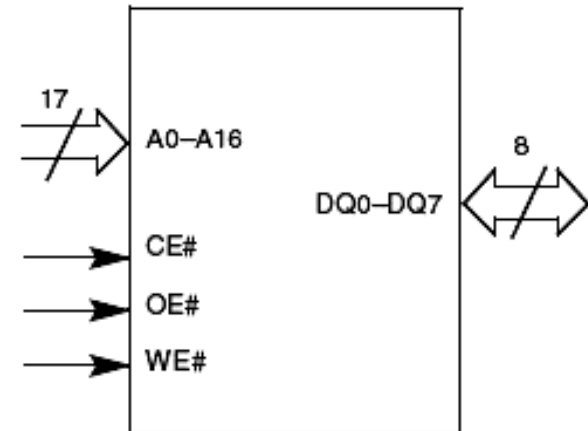
AMD 5V Flash Memory (1/2)

* pin configuration

PIN CONFIGURATION

A0–A16	=	17 Addresses
DQ0–DQ7	=	8 Data Inputs/Outputs
CE#	=	Chip Enable
OE#	=	Output Enable
WE#	=	Write Enable
V _{CC}	=	+5.0 Volt Single Power Supply (See Product Selector Guide for speed options and voltage supply tolerances)
V _{SS}	=	Device Ground
NC	=	Pin Not Connected Internally

LOGIC SYMBOL

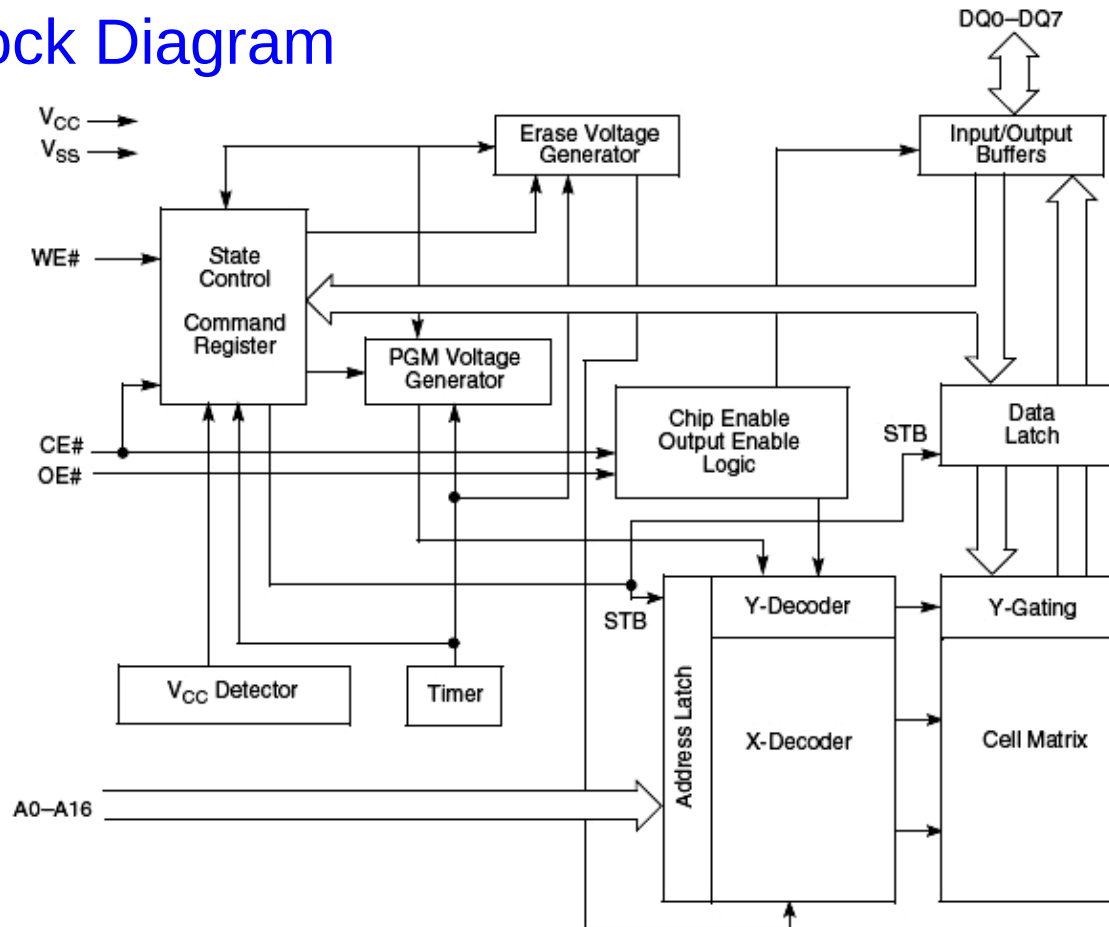


Source <http://www.spansion.com/amd/>



AMD 5V Flash Memory (2/2)

* Block Diagram

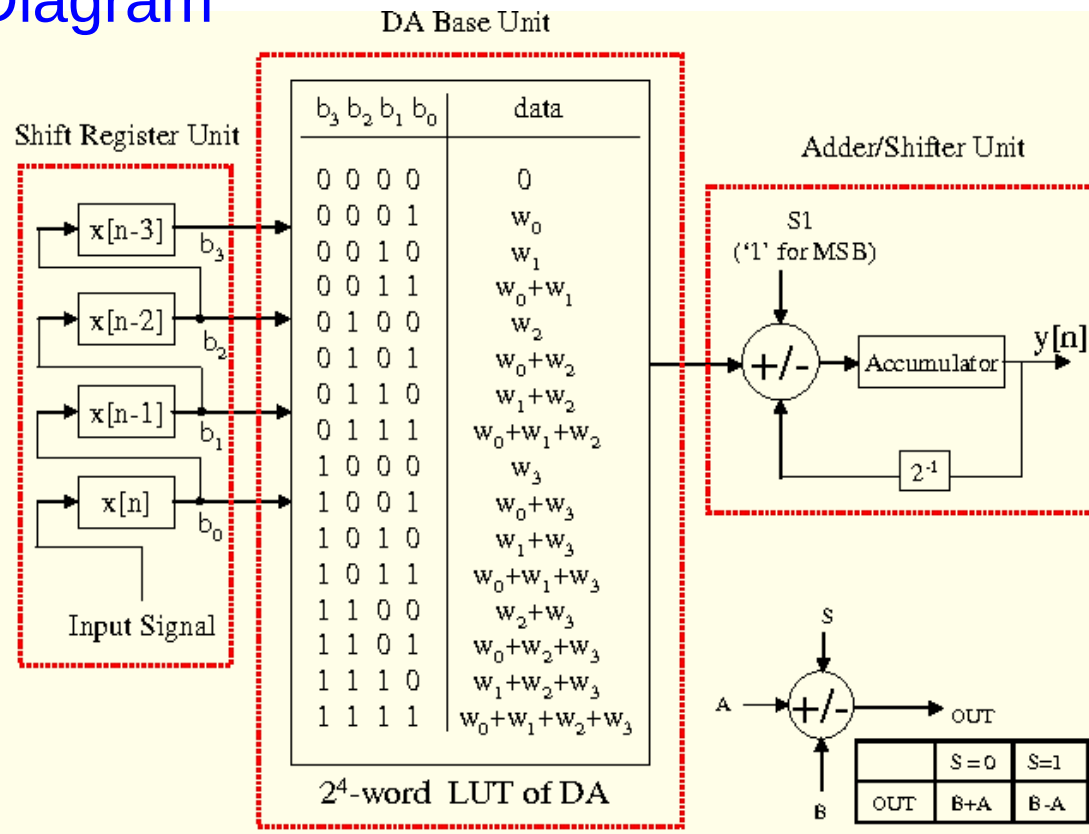


Source <http://www.spansion.com/amd/>



Distributed Arithmetic – An Example

* Block Diagram

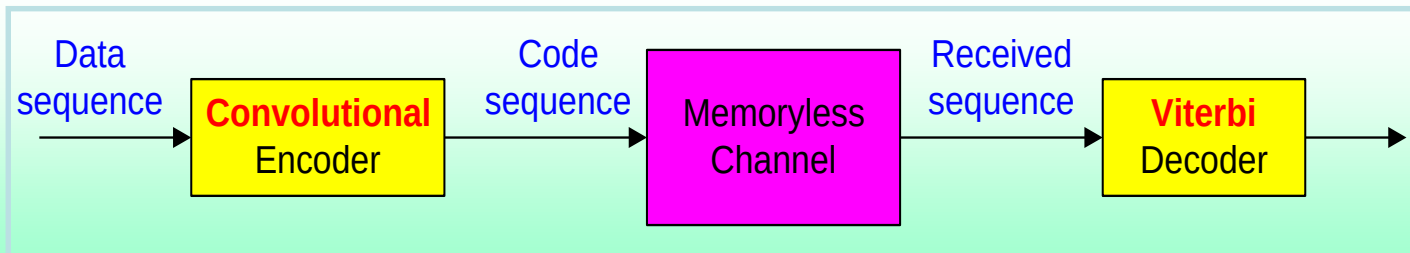


Source: H. Yoo and D. Anderson, "Hardware-efficient distributed arithmetic architecture for high-order digital filters," IEEE ICASSP, 2005.



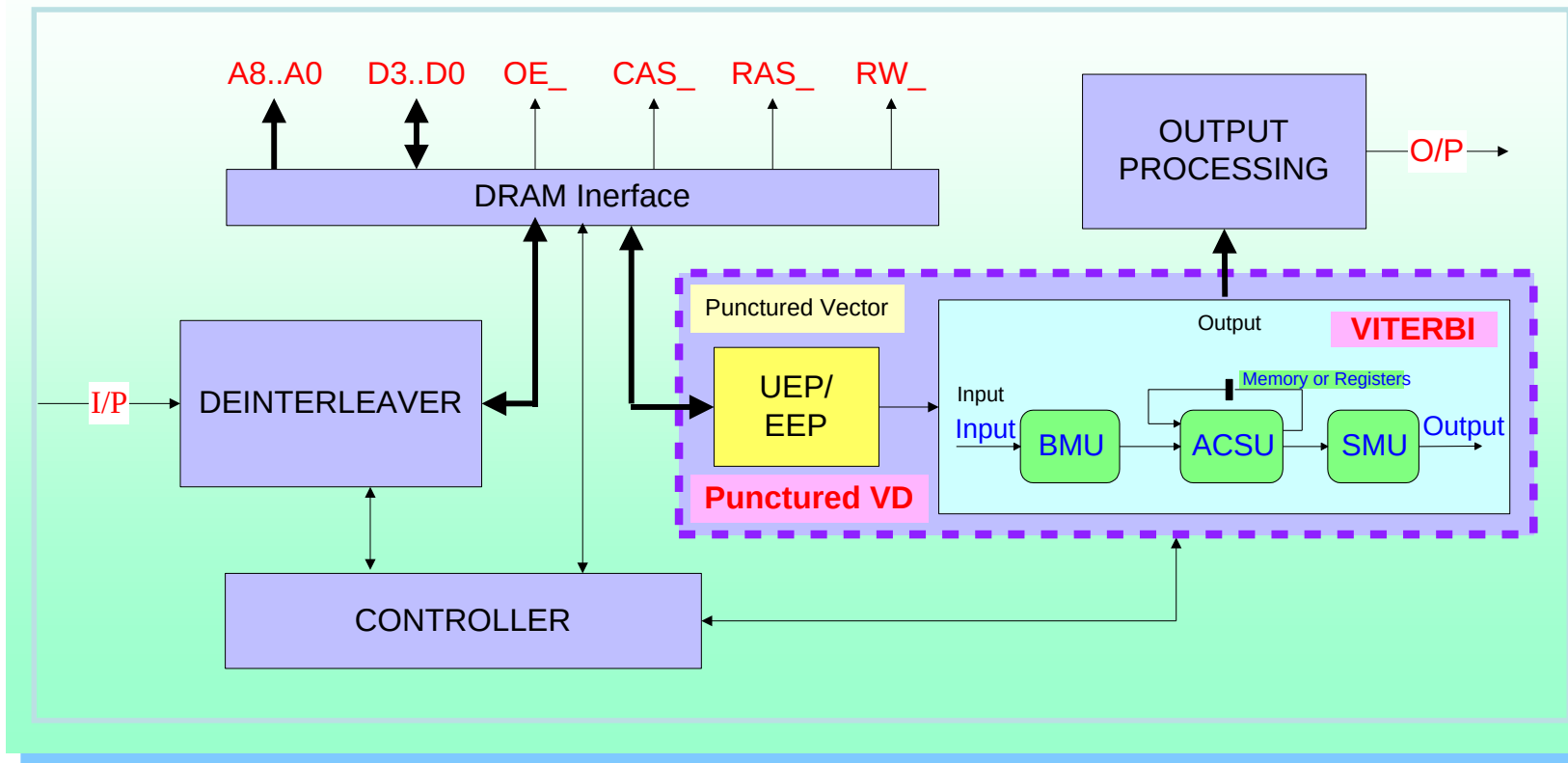
Convolution Code & Viterbi Decoder

- **Convolutional codes** were first introduced by Elias in 1955.
- In 1967, **Viterbi** proposed the well-known VA.
- Forney recognized that **VA** was a **maximum likelihood decoding algorithm** for convolutional codes.



VLSI Design for DAB System

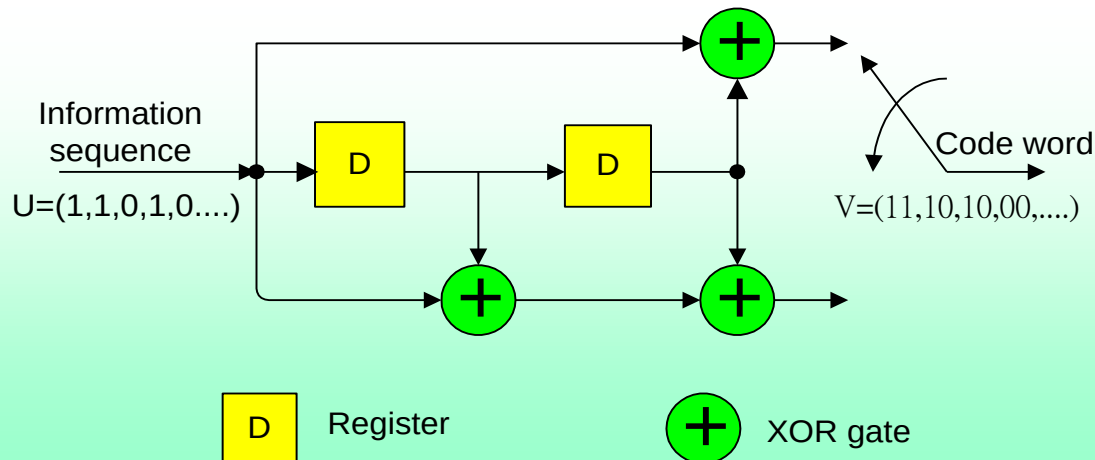
- An **external memory** is used for time deinterleaving and serves as a buffer for Viterbi decoding.



Convolutional Code

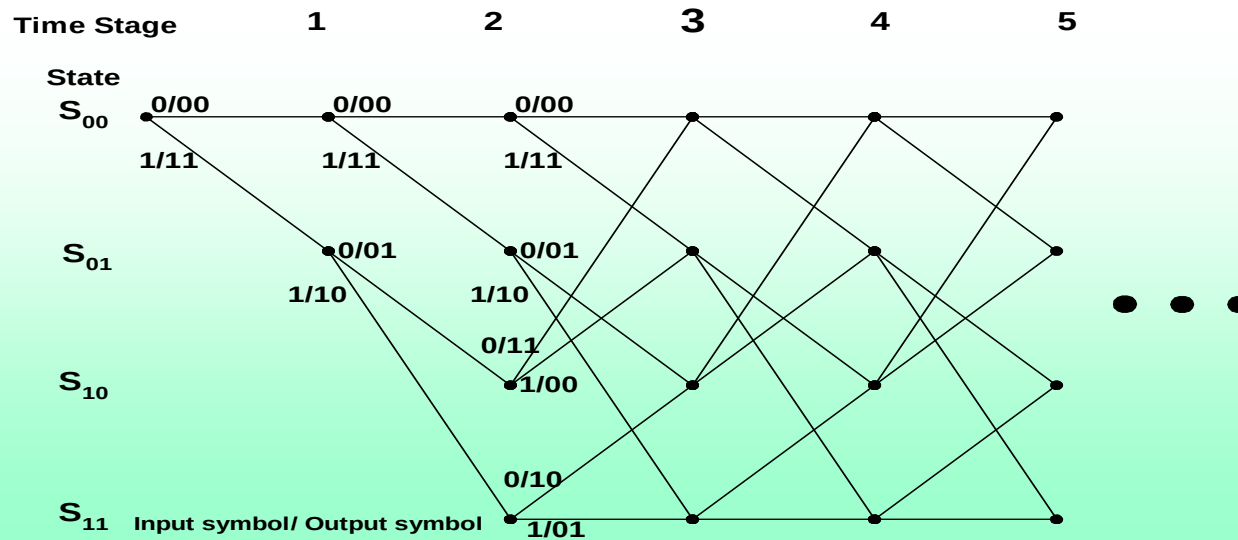
■ An (n, k, m) Convolutional Code

- ◆ k input bits, n output bits, m memory order
- ◆ Code rate $R = k/n$, Constraint length $K = m+1$
- ◆ It is composed of **shift-registers** and **modulo-2** operations.
- ◆ $(2, 1, 2)$ CC : Generator polynomial $g_1(D)=1+D^2$; $g_2(D)=1+D+D^2$



Trellis Diagram

- To aid in visualizing the transitions from states to states, it represents the finite-state diagram by a time-indexed equivalent.
- The **trellis diagram** of a (2, 1, 2) CC



Viterbi Algorithm (1/2)

■ Branch metric

- ◆ The **distance** of the trellis and the received codeword at each time step;
- ◆ It is denoted as $B_{i,j,t}$ for state i to state j at recursion t .

■ Partial path metric

- ◆ The **summation of path metric** at previous time stage $t-1$ and the branch metric at time t that a partial path goes through.

$$M_{i,t} = P_{i,t-1} + B_{i,j,t}$$

■ Survivor path

- ◆ The path with **the smallest path metric** when entering a state



Viterbi Algorithm (2/2)

■ Path metric

- ◆ The summation of the branch metrics that a survivor path goes through.
- ◆ It is denoted as $P_{j,t}$ for the state j at recursion t .

$$\begin{aligned} P_{i,t} &= \min_{\{i\}} [M_{i,t}] \\ &= \min_{\{i\}} [P_{i,t-1} + B_{i,j,t}] \end{aligned}$$

■ Global most-likely path

- ◆ The survivor path that has the **minimum path metric among all of the path metrics** at current time stage.



An Example of Viterbi Algorithm

